

Mathematical Foundations for Machine Learning_compressed.pdf

by Turnitin Student

Submission date: 26-Jul-2025 08:20PM (UTC+0530)

Submission ID: 2720776973

File name: Mathematical_Foundations_for_Machine_Learning_compressed.pdf (1.65M)

Word count: 26079

Character count: 125901

Mathematical Foundations for Machine Learning: A Programming-Enhanced Educational Framework for Undergraduate Education

Mateus Yonathan
Software Developer & Independent Researcher
<https://www.linkedin.com/in/siyoyo/>

Abstract

This paper presents a programming-enhanced educational framework that combines C# code, mathematical formulations, and visual descriptions to teach machine learning foundations. We present sixteen core mathematical domains through this approach, demonstrating how programming implementation enhances mathematical understanding. The framework addresses the gap between theoretical mathematics and practical machine learning applications, providing instructors with a transferable methodology for undergraduate education. Our approach shows how code implementation can serve as a bridge between abstract mathematical concepts and real-world applications, making complex topics more accessible to students.

Keywords: Mathematical foundations, Machine learning, Programming-enhanced learning, C# implementation, Educational framework, Undergraduate education

1 Language Choice Justification: Why C# for Mathematical Education

The choice of programming language in mathematical education involves trade-offs between type safety, development speed, and ecosystem maturity. C# offers strong static typing that can help catch mathematical errors during compilation [24], while providing a mature ecosystem suitable for educational applications [16].

Educational Criteria	C#	Python	C++	Julia	Java	MATLAB
Type Safety	Strong	Dynamic	Strong	Strong	Strong	Dynamic
Learning Curve	Moderate	Gentle	Steep	Moderate	Moderate	Gentle
Math Libraries	Growing	Extensive	Limited	Extensive	Limited	Extensive
IDE Support	Excellent	Good	Good	Good	Excellent	Good
Community Size	Large	Very Large	Large	Small	Large	Medium
Production Readiness	High	High	High	Medium	High	Medium
Research Ecosystem	Limited	Extensive	Limited	Growing	Limited	Extensive
Documentation	Excellent	Good	Variable	Good	Excellent	Good

The selection of C# for this educational framework is based on its balance of type safety and accessibility, rather than claims of inherent mathematical superiority. Each language offers distinct advantages for different educational contexts.

2 Linear Algebra

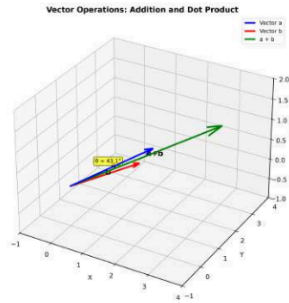
Linear algebra forms the mathematical backbone of machine learning, providing essential tools for representing and manipulating data in high-dimensional spaces [21]. From simple vector operations to complex matrix factorizations, these concepts enable efficient computation and meaningful data transformations [8].

2.1 Vectors: Operations, Dot Product, Cross Product, Norms

Listing 1: Vector Operations in C#

```
1 public class Vector
2 {
3     public double[] Components { get; set; }
4
5     public Vector(double[] components) => Components = components;
6
7     public static Vector operator +(Vector a, Vector b)
8     {
9         var result = new double[a.Components.Length];
10        for (int i = 0; i < a.Components.Length; i++)
11            result[i] = a.Components[i] + b.Components[i];
12        return new Vector(result);
13    }
14
15    public double DotProduct(Vector other)
16    {
17        double sum = 0;
18        for (int i = 0; i < Components.Length; i++)
19            sum += Components[i] * other.Components[i];
20        return sum;
21    }
22
23    public double Norm()
24    {
25        return Math.Sqrt(DotProduct(this));
26    }
27 }
```

Visual output: A 3D coordinate system showing two vectors originating from the origin, their sum as a diagonal vector, and the angle between them indicating their dot product relationship.



Mathematical formulation:

$$\mathbf{a} + \mathbf{b} = [a_1 + b_1, a_2 + b_2, \dots, a_n + b_n]$$
$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$$
$$\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{\sum_{i=1}^n a_i^2}$$

2.2 Matrices: Multiplication, Transpose, Inverse

Listing 2: Matrix Operations in C#

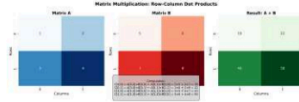
```
1 public class Matrix
2 {
3     public double[,] Data { get; set; }
4     public int Rows { get; }
5     public int Cols { get; }
6
7     public Matrix(int rows, int cols)
8     {
9         Rows = rows; Cols = cols;
10        Data = new double[rows, cols];
11    }
12
13    public static Matrix operator *(Matrix a, Matrix b)
14    {
15        var result = new Matrix(a.Rows, b.Cols);
16        for (int i = 0; i < a.Rows; i++)
17            for (int j = 0; j < b.Cols; j++)
18                for (int k = 0; k < a.Cols; k++)
19                    result.Data[i, j] += a.Data[i, k] * b.Data[k, j];
20        return result;
21    }
22
23    public Matrix Transpose()
24    {
25        var result = new Matrix(Cols, Rows);
```

```
26        for (int i = 0; i < Rows; i++)
27            for (int j = 0; j < Cols; j++)
28                result.Data[j, i] = Data[i, j];
29        return result;
30    }
31 }
```

Mathematical formulation:

$$(AB)_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$
$$A_{ij}^T = A_{ji}$$
$$AA^{-1} = A^{-1}A = I$$

Visual output: A grid representation showing matrix multiplication as row-column dot products, with color-coded elements demonstrating the multiplication process.



2.3 Matrix Factorizations: LU, QR, SVD

Listing 3: LU Decomposition in C#

```
1 public class LUDecomposition
2 {
3     public Matrix L { get; }
4     public Matrix U { get; }
5
6     public LUDecomposition(Matrix A)
7     {
8         int n = A.Rows;
9         L = new Matrix(n, n);
10        U = new Matrix(n, n);
11
12        for (int i = 0; i < n; i++)
13        {
14            L.Data[i, i] = 1;
15            for (int j = i + 1; j < n; j++)
16            {
17                double sum = 0;
18                for (int k = 0; k < i; k++)
19                    sum += L.Data[i, k] * U.Data[k, j];
20                U.Data[i, j] = A.Data[i, j] - sum;
21            }
22
23            for (int j = i + 1; j < n; j++)
24            {
25                double sum = 0;
26                for (int k = 0; k < i; k++)
27                    sum += L.Data[j, k] * U.Data[k, i];
28                L.Data[j, i] = (A.Data[j, i] - sum) / U.Data[i, i];
29            }
30        }
31    }
32 }
```

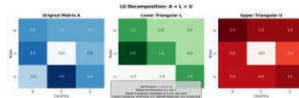
Mathematical formulation:

$$A = LU$$

$$A = QR$$

$$A = U\Sigma V^T$$

Visual output: A triangular matrix visualization showing the decomposition of a matrix into lower and upper triangular components.



2.4 Eigenvalues and Eigenvectors

Listing 4: Eigenvalue Computation in C#

```
1 public class EigenDecomposition
2 {
3     public double[] Eigenvalues { get; }
4     public Matrix Eigenvectors { get; }
5
6     public EigenDecomposition(Matrix A)
7     {
8         int n = A.Rows;
9         Eigenvalues = new double[n];
10        Eigenvectors = new Matrix(n, n);
11
12        // Power iteration method for dominant eigenvalue
13        Vector v = new Vector(new double[n]);
14        for (int i = 0; i < n; i++) v.Components[i] = 1;
15
16        for (int iter = 0; iter < 100; iter++)
17        {
18            Vector Av = Multiply(A, v);
19            double lambda = v.DotProduct(Av) / v.DotProduct(v);
20            v = Normalize(Av);
21        }
22
23        Eigenvalues[0] = lambda;
24        for (int i = 0; i < n; i++)
25            Eigenvectors.Data[i, 0] = v.Components[i];
26    }
27 }
```

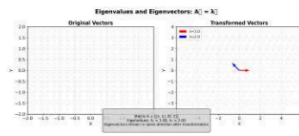
Mathematical formulation:

$$Av = \lambda v$$

$$\det(A - \lambda I) = 0$$

$$A = V\Lambda V^{-1}$$

Visual output: A geometric representation showing how eigenvectors remain in the same direction when multiplied by the matrix, with eigenvalues indicating the scaling factor.



2.5 Principal Component Analysis (PCA)

Listing 5: PCA Implementation in C#

```
1 public class PCA
2 {
3     public Matrix Components { get; }
```

```
4     public double[] ExplainedVariance { get; }
5
6     public PCA(Matrix data, int nComponents)
7     {
8         // Center the data
9         Matrix centered = CenterData(data);
```

```

10 // Compute covariance matrix
11 Matrix covariance = ComputeCovariance(centered);
12
13 // Find eigenvectors of covariance matrix
14 var eigenDecomp = new EigenDecomposition(covariance);
15
16 // Sort by eigenvalues (descending)
17 var sortedIndices = Enumerable.Range(0, eigenDecomp.Eigenvalues.Length)
18     .OrderByDescending(i => eigenDecomp.Eigenvalues[i])
19     .Take(nComponents)
20     .ToArray();
21
22 // Extract principal components
23 Components = new Matrix(data.Cols, nComponents);
24 ExplainedVariance = new double[nComponents];
25
26 for (int i = 0; i < nComponents; i++)
27 {
28     int idx = sortedIndices[i];
29     for (int j = 0; j < data.Cols; j++)
30     {
31         Components.Data[j, i] = eigenDecomp.Eigenvalues[idx] *
32             Data[j, idx];
33         ExplainedVariance[i] = eigenDecomp.Eigenvalues[idx];
34     }
35 }

```

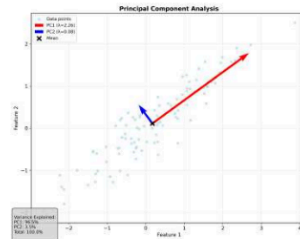
Mathematical formulation:

$$\Sigma = \frac{1}{n-1} X^T X$$

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

$$X_{reduced} = XW$$

Visual output: A 2D scatter plot showing data points projected onto the first two principal components, with arrows indicating the direction of maximum variance.



3 Calculus

Calculus provides the mathematical foundation for optimization in machine learning, enabling the computation of gradients and the understanding of how functions change [20]. From derivatives to optimization algorithms, these concepts are essential for training neural networks and understanding model behavior [15].

3.1 Derivatives

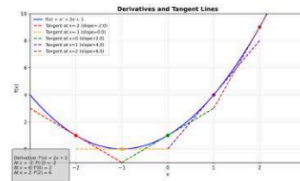
Listing 6: Derivative Computation in C#

```

22
1 public class Derivative
2 {
3     public static double ComputeDerivative(Func<double, double> f,
4         double x, double h = 1e-7)
5     {
6         return (f(x + h) - f(x)) / h;
7     }
8     public static double ComputeSecondDerivative(Func<double,
9         double> f, double x, double h = 1e-7)
10    {
11        return (f(x + h) - 2 * f(x) + f(x - h)) / (h * h);
12    }
13    public static Func<double, double> Differentiate(Func<double,
14        double> f)
15    {
16        return x => ComputeDerivative(f, x);
17    }
18 }

```

Visual output: A graph showing a function curve with a tangent line at a point, demonstrating the geometric interpretation of the derivative as the slope of the tangent.



Mathematical formulation:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

$$f''(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

3.2 Partial Derivatives

Listing 7: Partial Derivatives in C#

```
1 public class PartialDerivative
2 {
3     public static double ComputePartialDerivative(
4         Func<double, double, double> f,
5         double x, double y,
6         int variable, double h = 1e-7)
7     {
8         if (variable == 0) // with respect to x
9             return (f(x+h, y) - f(x, y)) / h;
10        else // with respect to y
11            return (f(x, y+h) - f(x, y)) / h;
12    }
13
14    public static double[] ComputeGradient(
15        Func<double, double, double> f,
16        double x, double y)
17    {
18        return new double[]
19        {
20            ComputePartialDerivative(f, x, y, 0),
21            ComputePartialDerivative(f, x, y, 1)
22        };
23    }
24 }
```

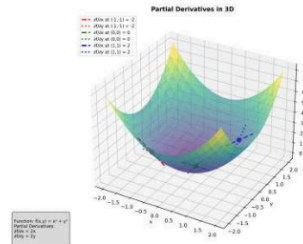
Mathematical formulation:

$$\frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x+h, y) - f(x, y)}{h}$$

$$\frac{\partial f}{\partial y} = \lim_{h \rightarrow 0} \frac{f(x, y+h) - f(x, y)}{h}$$

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

Visual output: A 3D surface plot showing a function of two variables with arrows indicating the direction of steepest ascent (gradient) at different points.



3.3 Chain Rule

Listing 8: Chain Rule Implementation in C#

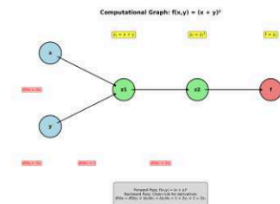
```
1 public class ChainRule
2 {
3     public static double ComputeChainRule(
4         Func<double, double> f,
5         Func<double, double> g,
6         double x)
7     {
8         double g_x = g(x);
9         double f_prime_g = ComputeDerivative(f, g_x);
10        double g_prime_x = ComputeDerivative(g, x);
11
12        return f_prime_g * g_prime_x;
13    }
14
15    public static Func<double, double> DifferentiateComposition(
16        Func<double, double> f,
17        Func<double, double> g)
18    {
19        return x => ComputeChainRule(f, g, x);
20    }
21 }
```

Mathematical formulation:

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$

$$\frac{d}{dx}[f(g(h(x)))] = f'(g(h(x))) \cdot g'(h(x)) \cdot h'(x)$$

Visual output: A computational graph showing the flow of derivatives through nested functions, with intermediate values and gradients labeled.



3.4 Gradient, Jacobian, Hessian

Listing 9: Gradient and Hessian in C#

```
1 public class GradientHessian
2 {
3     public static double[] ComputeGradient(
4         Func<double[], double> f,
5         double[] x, double h = 1e-7)
6     {
```

```
7         int n = x.Length;
8         double[] gradient = new double[n];
9
10        for (int i = 0; i < n; i++)
11        {
12            double[] x_plus = (double[])x.Clone();
13            double[] x_minus = (double[])x.Clone();
14            x_plus[i] += h;
15            x_minus[i] -= h;
```

```

17     gradient[i] = (f(x_plus) - f(x_minus)) / (2 * h);
18 }
19 return gradient;
20 }
21
22 public static double[,] ComputeHessian(
23     Func<double[], double> f,
24     double[] x, double h = 1e-7)
25 {
26     int n = x.Length;
27     double[,] hessian = new double[n, n];
28     for (int i = 0; i < n; i++)
29     {
30         for (int j = 0; j < n; j++)
31         {
32             double[] x_pp = (double[])x.Clone();
33             double[] x_pm = (double[])x.Clone();
34             double[] x_mp = (double[])x.Clone();
35             double[] x_mm = (double[])x.Clone();
36
37             x_pp[i] += h; x_pp[j] += h;
38             x_pm[i] += h; x_pm[j] -= h;
39             x_mp[i] -= h; x_mp[j] += h;
40             x_mm[i] -= h; x_mm[j] -= h;
41
42             hessian[i, j] = (f(x_pp) - f(x_pm) - f(x_mp) + f(
43                 x_mm)) / (4 * h * h);
44         }
45     }
46     return hessian;
47 }
48
49 }
50

```

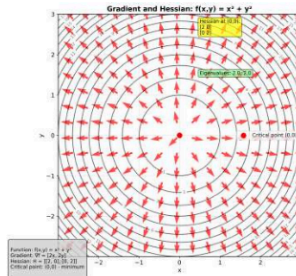
Mathematical formulation:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]$$

$$J_{ij} = \frac{\partial f_i}{\partial x_j}$$

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Visual output: A contour plot showing level curves of a function with arrows representing the gradient field, and a matrix visualization of the Hessian showing second-order derivatives.



3.5 Optimization using Gradient Descent, SGD, Adam

Listing 10: Optimization Algorithms in C#

```

1 public class Optimizer
2 {
3     public static double[] GradientDescent(
4         Func<double[], double> f,
5         Func<double[], double[]> gradient,
6         double[] x0, double learningRate = 0.01, int iterations =
7             1000)
8     {
9         double[] x = (double[])x0.Clone();
10        for (int i = 0; i < iterations; i++)
11        {
12            double[] grad = gradient(x);
13            for (int j = 0; j < x.Length; j++)
14                x[j] -= learningRate * grad[j];
15        }
16        return x;
17    }
18
19    public static double[] Adam(
20        Func<double[], double> f,
21        Func<double[], double[]> gradient,
22        double[] x0, double learningRate = 0.001, int iterations =
23            1000)
24    {
25        double[] x = (double[])x0.Clone();
26        double[] m = new double[x.Length]; // first moment
27        double[] v = new double[x.Length]; // second moment
28        double beta1 = 0.9, beta2 = 0.999, epsilon = 1e-8;
29        for (int t = 1; t <= iterations; t++)
30        {
31            double[] grad = gradient(x);
32
33            for (int i = 0; i < x.Length; i++)
34            {
35                m[i] = beta1 * m[i] + (1 - beta1) * grad[i];
36                v[i] = beta2 * v[i] + (1 - beta2) * grad[i] * grad[i];
37            }
38
39            double m_hat = m[i] / (1 - Math.Pow(beta1, t));

```

```

40        double v_hat = v[i] / (1 - Math.Pow(beta2, t));
41
42        x[i] -= learningRate * m_hat / (Math.Sqrt(v_hat) +
43            epsilon);
44    }
45
46    return x;
47 }
48

```

Mathematical formulation:

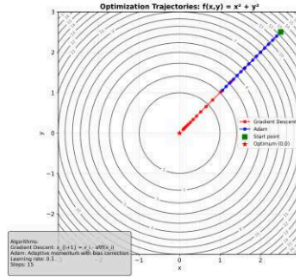
$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha \nabla f(\mathbf{x}_t) \quad (\text{Gradient Descent})$$

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla f(\mathbf{x}_t)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla f(\mathbf{x}_t))^2$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\alpha}{\sqrt{\mathbf{v}_t} + \epsilon} \mathbf{m}_t \quad (\text{Adam})$$

Visual output: A 2D plot showing optimization trajectories for different algorithms, with contour lines representing the objective function and arrows showing the path taken by each optimization method.



4 Probability and Statistics

Probability theory provides the mathematical foundation for understanding uncertainty in machine learning [18]. From probability distributions to statistical inference, these concepts enable the development of robust models that can handle real-world data variability [12].

4.1 Probability Distributions: Normal, Bernoulli, Categorical

Listing 11: Probability Distributions in C#

```
1 public class ProbabilityDistributions
2 {
3     public static double NormalPDF(double x, double mu, double
4         sigma)
5     {
6         double z = (x - mu) / sigma;
7         return Math.Exp(-0.5 * z * z) / (sigma * Math.Sqrt(2 * Math
8             .PI));
9     }
10    public static double BernoulliPMF(int k, double p)
11    {
12        return k == 1 ? p * Math.Pow(1 - p, k - 1) :
13            (1 - p) * Math.Pow(p, k - 1);
14    }
15    public static double CategoricalPMF(int k, double[] p)
16    {
17        return p[k];
18    }
19    public static double NormalCDF(double x, double mu, double
20        sigma)
21    {
22        return 0.5 * (1 + Erf((x - mu) / (sigma * Math.Sqrt(2)))));
23    }
24    private static double Erf(double x)
25    {
26        // Approximation of error function
27        double a1 = 0.254829592;
28        double a2 = -0.284497756;
29        double a3 = 1.421413741;
30        double a4 = -3.453152027;
31        double a5 = 1.061405429;
32        double p = 0.3275911;
33        double sign = x < 0 ? -1 : 1;
34        x = Math.Abs(x);
35    }
```

```
36 double x = 1.0 / (1.0 + p * x);
37 double y = 1.0 - (((a5 * E + a4) * E) + a3) * E + a2 * t
38 + a1) * t * Math.Exp(-x * x);
39 return sign * y;
40 }
41 }
42 }
```

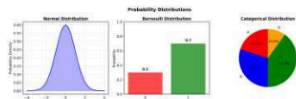
Mathematical formulation:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (\text{Normal})$$

$$P(X = k) = p^k (1 - p)^{1-k} \quad (\text{Bernoulli})$$

$$P(X = k) = p_k \quad (\text{Categorical})$$

Visual output: A plot showing different probability distributions: a bell-shaped normal distribution, a discrete Bernoulli distribution with two values, and a categorical distribution with multiple discrete probabilities.



4.2 Expectation and Variance

Listing 12: Expectation and Variance in C#

```
1 public class ExpectationVariance
2 {
3     public static double Expectation(Func<double, double> f, double
4         [] samples)
5     {
6         return samples.Average(x => f(x));
7     }
8     public static double Variance(double[] samples)
9     {
10        double mean = samples.Average();
11        return samples.Average(x => Math.Pow(x - mean, 2));
12    }
13 }
```

```
5 return samples.Average(x => f(x));
6 }
7 }
8 public static double Variance(double[] samples)
9 {
10    double mean = samples.Average();
11    return samples.Average(x => Math.Pow(x - mean, 2));
12 }
```

```

13 7
14 public static double Covariance(double[] x, double[] y)
15 {
16     double meanX = x.Average();
17     double meanY = y.Average();
18     double sum = 0;
19     for (int i = 0; i < x.Length; i++)
20         sum += (x[i] - meanX) * (y[i] - meanY);
21     return sum / (x.Length - 1);
22 }
23
24 1
25 public static double Correlation(double[] x, double[] y)
26 {
27     double cov = Covariance(x, y);
28     double stdX = Math.Sqrt(Variance(x));
29     double stdY = Math.Sqrt(Variance(y));
30     return cov / (stdX * stdY);
31 }
32 }
33
34 }

```

Mathematical formulation:

$$E[X] = \sum_i x_i P(X = x_i)$$

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$$

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

4.3 Bayes' Theorem

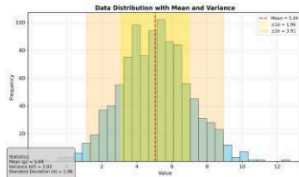
Listing 13: Bayes' Theorem in C#

```

1 1
2 public class BayesTheorem
3 {
4     public static double PosteriorProbability(
5         double prior, double likelihood, double evidence)
6     {
7         return (likelihood * prior) / evidence;
8     }
9     4
10    public static double[] PosteriorProbabilities(
11        double[] priors, double[] likelihoods)
12    {
13        double evidence = 0;
14        for (int i = 0; i < priors.Length; i++)
15            evidence += likelihoods[i] * priors[i];
16        double[] posteriors = new double[priors.Length];
17        for (int i = 0; i < priors.Length; i++)
18            posteriors[i] = (likelihoods[i] * priors[i]) / evidence;
19    }
20    return posteriors;
21 }
22
23 public static double NaiveBayesClassifier(
24     double[] classPriors, double[,] featureLikelihoods, double
25     [] features)
26 {
27     2
28     double[] posteriors = new double[classPriors.Length];
29     for (int c = 0; c < classPriors.Length; c++)
30     {
31         1.5
32         double likelihood = 1.0;
33         for (int f = 0; f < features.Length; f++)
34             likelihood *= featureLikelihoods[c, f];
35         posteriors[c] = likelihood * classPriors[c];
36     }
37     return Array.IndexOf(posteriors, posteriors.Max());
38 }
39 }

```

Visual output: A scatter plot showing the relationship between two random variables, with the mean and variance of each variable indicated, and correlation coefficient displayed.



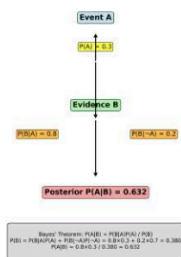
Mathematical formulation:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$P(B) = \sum_i P(B|A_i)P(A_i)$$

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{\sum_j P(B|A_j)P(A_j)}$$

Visual output: A Venn diagram showing the intersection of two events A and B, with conditional probabilities labeled, demonstrating how Bayes' theorem updates our beliefs based on new evidence.



4.4 Maximum Likelihood Estimation (MLE)

Listing 14: Maximum Likelihood Estimation in C#

```

1 public class MaximumLikelihood
2 {
3     public static double LogLikelihood(double[] data, double mu,
4         double sigma)
5     {
6         double logLikelihood = 0;
7         foreach (double x in data)
8         {
9             double z = (x - mu) / sigma;
10            logLikelihood += -0.5 * z * z - Math.Log(sigma * Math.
11                Sqrt(2 * Math.PI));
12        }
13        return logLikelihood;
14    }
15    public static double[] EstimateNormalParameters(double[] data)
16    {
17        double mu = data.Average();
18        double variance = data.Select(x => Math.Pow(x - mu, 2)).
19            Average();
20        double sigma = Math.Sqrt(variance);
21        return new double[] { mu, sigma };
22    }
23    public static double[] EstimateBernoulliParameter(double[] data)
24    {
25        double p = data.Average(); // MLE for Bernoulli is sample
26        return new double[] { p };
27    }
28    public static double[] EstimateCategoricalParameters(double[]
29        data, int numCategories)
30    {
31        double[] p = new double[numCategories];
32        for (int i = 0; i < numCategories; i++)
33        {
34            p[i] = data.Count(x => x == i) / (double)data.Length;
35        }
36    }
37 }
38

```

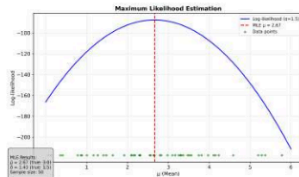
Mathematical formulation:

$$L(\theta) = \prod_{i=1}^n f(x_i | \theta)$$

$$\ell(\theta) = \sum_{i=1}^n \log f(x_i | \theta)$$

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \ell(\theta)$$

Visual output: A plot showing the likelihood function for different parameter values, with the maximum likelihood estimate marked at the peak of the function.



4.5 Entropy, KL Divergence

Listing 15: Entropy and KL Divergence in C#

```

1 public class InformationTheory
2 {
3     public static double Entropy(double[] probabilities)
4     {
5         double entropy = 0;
6         foreach (double p in probabilities)
7         {
8             if (p > 0)
9             {
10                entropy -= p * Math.Log(p, 2);
11            }
12        }
13        return entropy;
14    }
15    public static double KLDivergence(double[] p, double[] q)
16    {
17        double kl = 0;
18        for (int i = 0; i < p.Length; i++)
19        {
20            if (p[i] > 0 && q[i] > 0)
21            {
22                kl += p[i] * Math.Log(p[i] / q[i], 2);
23            }
24        }
25        return kl;
26    }
27    public static double CrossEntropy(double[] p, double[] q)
28    {
29        double crossEntropy = 0;
30        for (int i = 0; i < p.Length; i++)
31        {
32            if (p[i] > 0 && q[i] > 0)
33            {
34                crossEntropy -= p[i] * Math.Log(q[i], 2);
35            }
36        }
37        return crossEntropy;
38    }
39    public static double MutualInformation(double[] jointProb,
40        double[] pX, double[] pY)
41    {
42        double mi = 0;
43        for (int i = 0; i < pX.Length; i++)
44        {
45            for (int j = 0; j < pY.Length; j++)
46            {
47                if (jointProb[i, j] > 0 && pX[i] > 0 && pY[j] > 0)
48                {
49                    mi += jointProb[i, j] * Math.Log(jointProb[i, j] / (pX[i] * pY[j]), 2);
50                }
51            }
52        }
53        return mi;
54    }
55 }
56

```

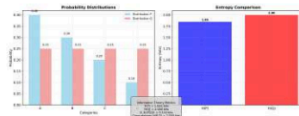
Mathematical formulation:

$$H(X) = - \sum_i p_i \log p_i$$

$$D_{KL}(P||Q) = \sum_i p_i \log \frac{p_i}{q_i}$$

$$H(P, Q) = - \sum_i p_i \log q_i$$

Visual output: A plot showing different probability distributions and their corresponding entropy values, with KL divergence measuring the difference between two distributions.



5 Distance Metrics

Distance metrics provide mathematical foundations for measuring similarity and dissimilarity between data points [1]. These measures are fundamental for clustering, nearest neighbor algorithms, and similarity-based learning methods.

5.1 Euclidean Distance

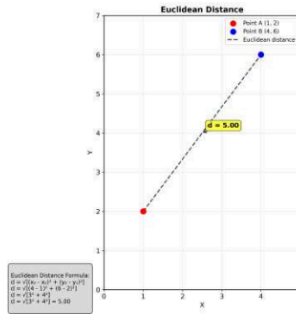
Listing 16: Euclidean Distance in C#

```
1 public class DistanceMetrics
2 {
3     public static double EuclideanDistance(double[] a, double[] b)
4     {
5         double sum = 0;
6         for (int i = 0; i < a.Length; i++)
7             sum += Math.Pow(a[i] - b[i], 2);
8         return Math.Sqrt(sum);
9     }
10
11     public static double EuclideanDistanceSquared(double[] a,
12     double[] b)
13     {
14         double sum = 0;
15         for (int i = 0; i < a.Length; i++)
16             sum += Math.Pow(a[i] - b[i], 2);
17         return sum;
18     }
19
20     public static double[] ComputeDistances(double[][] points,
21     double[] query)
22     {
23         double[] distances = new double[points.Length];
24         for (int i = 0; i < points.Length; i++)
25             distances[i] = EuclideanDistance(points[i], query);
26         return distances;
27     }
28 }
```

Mathematical formulation:

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$
$$d^2(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n (a_i - b_i)^2$$

Visual output: A 2D plane with two points connected by a straight line representing the Euclidean distance.



5.2 Manhattan Distance

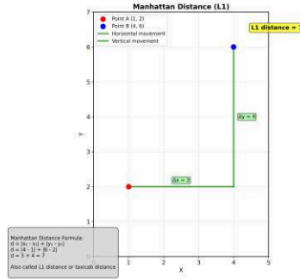
Listing 17: Manhattan Distance in C#

```
1 public class ManhattanDistance
2 {
3     public static double ManhattanDistance(double[] a, double[] b)
4     {
5         double sum = 0;
6         for (int i = 0; i < a.Length; i++)
7             sum += Math.Abs(a[i] - b[i]);
8         return sum;
9     }
10
11     public static double[] ManhattanDistances(double[][] points,
12     double[] query)
13     {
14         double[] distances = new double[points.Length];
15         for (int i = 0; i < points.Length; i++)
16             distances[i] = ManhattanDistance(points[i], query);
17         return distances;
18     }
19
20     public static int[] NearestNeighbors(double[][] points, double
21     [] query, int k)
22     {
23         var distances = ManhattanDistances(points, query);
24         return Enumerable.Range(0, distances.Length)
25             .OrderBy(i => distances[i])
26             .Take(k)
27             .ToArray();
28     }
29 }
```

Mathematical formulation:

$$d_{\text{Manhattan}}(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n |a_i - b_i|$$

Visual output: A grid showing the L1 path between two points, following only horizontal and vertical movements.



5.3 Cosine Similarity

Listing 18: Cosine Similarity in C#

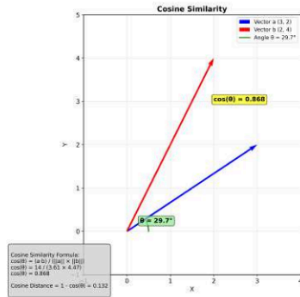
```
1 public class CosineSimilarity
2 {
3     public static double CosineSimilarity(double[] a, double[] b)
4     {
5         double dotProduct = 0;
6         double normA = 0;
7         double normB = 0;
8
9         for (int i = 0; i < a.Length; i++)
10         {
11             dotProduct += a[i] * b[i];
12             normA += a[i] * a[i];
13             normB += b[i] * b[i];
14         }
15
16         return dotProduct / (Math.Sqrt(normA) * Math.Sqrt(normB));
17     }
18
19     public static double CosineDistance(double[] a, double[] b)
20     {
21         return 1 - CosineSimilarity(a, b);
22     }
23
24     public static double[] ComputeSimilarities(double[][] vectors,
25         double[] query)
26     {
27         double[] similarities = new double[vectors.Length];
28         for (int i = 0; i < vectors.Length; i++)
29             similarities[i] = CosineSimilarity(vectors[i], query);
30         return similarities;
31     }
32 }
```

Mathematical formulation:

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

$$d_{\cosine}(\mathbf{a}, \mathbf{b}) = 1 - \cos(\theta)$$

Visual output: Two vectors from the origin with the angle between them highlighted, showing cosine similarity as the cosine of this angle.



5.4 Minkowski Distance

Listing 19: Minkowski Distance in C#

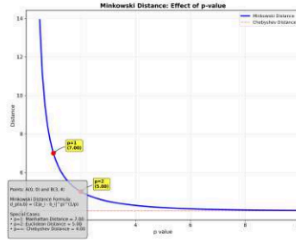
```
1 public class MinkowskiDistance
2 {
3     public static double MinkowskiDistance(double[] a, double[] b,
4         double p)
5     {
6         double sum = 0;
7         for (int i = 0; i < a.Length; i++)
8             sum += Math.Pow(Math.Abs(a[i] - b[i]), p);
9         return Math.Pow(sum, 1.0 / p);
10    }
11
12    public static double ChebyshevDistance(double[] a, double[] b)
13    {
14        double maxDiff = 0;
15        for (int i = 0; i < a.Length; i++)
16        {
17            double diff = Math.Abs(a[i] - b[i]);
18            if (diff > maxDiff) maxDiff = diff;
19        }
20        return maxDiff;
21    }
22
23    public static double[] ComputeDistances(double[][] points,
24        double[] query, double p)
25    {
26        double[] distances = new double[points.Length];
27        for (int i = 0; i < points.Length; i++)
28            distances[i] = MinkowskiDistance(points[i], query, p);
29        return distances;
30    }
31 }
```

Mathematical formulation:

$$d_p(\mathbf{a}, \mathbf{b}) = \left(\sum_{i=1}^n |a_i - b_i|^p \right)^{1/p}$$

$$d_\infty(\mathbf{a}, \mathbf{b}) = \max_i |a_i - b_i|$$

Visual output: A family of curves showing how different p-values affect the distance metric, from Manhattan (p=1) to Euclidean (p=2) to Chebyshev (p=infinity).



5.5 Hamming Distance

Listing 20: Hamming Distance in C#

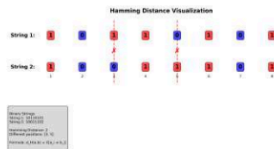
```
1 public class HammingDistance
2 {
3     public static int HammingDistance(int[] a, int[] b)
4     {
5         int distance = 0;
6         for (int i = 0; i < a.Length; i++)
7             if (a[i] != b[i]) distance++;
8         return distance;
9     }
10
11     public static int HammingDistance(string a, string b)
12     {
13         int distance = 0;
14         int minLength = Math.Min(a.Length, b.Length);
15         for (int i = 0; i < minLength; i++)
16             if (a[i] != b[i]) distance++;
17         distance += Math.Abs(a.Length - b.Length);
18         return distance;
19     }
20
21     public static double NormalizedHammingDistance(int[] a, int[] b)
22     {
23         return (double)HammingDistance(a, b) / a.Length;
24     }
25 }
```

Mathematical formulation:

$$d_{\text{Hamming}}(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n \mathbb{I}[a_i \neq b_i]$$

$$d_{\text{Hamming}}^{\text{norm}}(\mathbf{a}, \mathbf{b}) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[a_i \neq b_i]$$

Visual output: Two binary strings with highlighted positions where they differ, showing the Hamming distance as the count of mismatched positions.



6 Optimization

Optimization is the mathematical engine that drives machine learning algorithms [5]. From gradient descent to advanced optimization techniques, understanding these methods is crucial for training effective models efficiently [15].

6.1 Loss Functions: MSE, Cross-Entropy

Listing 21: Loss Functions in C#

```
1 public class LossFunctions
2 {
3     public static double MeanSquaredError(double[] predictions,
4     double[] targets)
5     {
6         double mse = 0;
7         for (int i = 0; i < predictions.Length; i++)
8             mse += Math.Pow(predictions[i] - targets[i], 2);
9         return mse / predictions.Length;
10    }
11
12    public static double[] MSEDerivative(double[] predictions,
13    double[] targets)
14    {
15        double[] derivative = new double[predictions.Length];
16        for (int i = 0; i < predictions.Length; i++)
17            derivative[i] = 2 * (predictions[i] - targets[i]) /
18            predictions.Length;
19    }
20
21    public static double CrossEntropyLoss(double[] predictions,
22    double[] targets)
23    {
24        double loss = 0;
25        for (int i = 0; i < predictions.Length; i++)
26        {
27            double p = Math.Max(1e-15, Math.Min(1 - 1e-15,
28            predictions[i]));
29            loss += targets[i] * Math.Log(p);
30        }
31        return -loss;
32    }
33
34    public static double[] CrossEntropyDerivative(double[]
35    predictions, double[] targets)
36    {
37        double[] derivative = new double[predictions.Length];
38        for (int i = 0; i < predictions.Length; i++)
39            derivative[i] = targets[i] / predictions[i] - 1;
40    }
41 }
```

```

34 {
35     double p = Math.Max(1e-15, Math.Min(1 - 1e-15,
36         predictions[i]));
37     derivative[i] = -targets[i] / p;
38 }
39 return derivative;
40 }

```

Mathematical formulation:

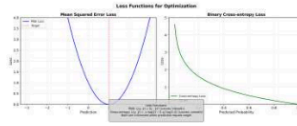
$$L_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{\partial L_{MSE}}{\partial \hat{y}_i} = \frac{2}{n} (\hat{y}_i - y_i)$$

$$L_{CE} = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

$$\frac{\partial L_{CE}}{\partial \hat{y}_i} = - \frac{y_i}{\hat{y}_i}$$

Visual output: A plot showing MSE as a parabolic curve and cross-entropy as a logarithmic curve, both minimized at the target value.



6.2 Optimization Algorithms: Gradient Descent Family

Listing 22: Advanced Optimization in C#

```

1 public class AdvancedOptimizers
2 {
3     public static double[] MomentumGradientDescent(Func<double[], double> f, double[] x0, double learningRate = 0.01,
4         double momentum = 0.9)
5     {
6         double[] x = (double[])x0.Clone();
7         double[] velocity = new double[x.Length];
8         for (int iter = 0; iter < 1000; iter++)
9         {
10             double[] grad = Gradient(f, x);
11             for (int i = 0; i < x.Length; i++)
12             {
13                 velocity[i] = momentum * velocity[i] - learningRate * grad[i];
14                 x[i] += velocity[i];
15             }
16             return x;
17         }
18     }
19
20     public static double[] RMSprop(Func<double[], double> f, double[] x0, double alpha = 0.001, double beta = 0.9)
21     {
22         double[] x = (double[])x0.Clone();
23         double[] v = new double[x.Length];
24         for (int iter = 0; iter < 1000; iter++)
25         {
26             double[] grad = Gradient(f, x);
27             for (int i = 0; i < x.Length; i++)
28             {
29                 v[i] = beta * v[i] + (1 - beta) * grad[i] * grad[i];
30                 x[i] -= alpha * grad[i] / (Math.Sqrt(v[i]) + 1e-8);
31             }
32             return x;
33         }
34     }
35
36     public static double[] AdaGrad(Func<double[], double> f, double[] x0, double alpha = 0.01)
37     {
38         double[] x = (double[])x0.Clone();
39         double[] g = new double[x.Length];
40         for (int iter = 0; iter < 1000; iter++)
41         {
42             double[] grad = Gradient(f, x);
43             for (int i = 0; i < x.Length; i++)
44             {
45                 g[i] += grad[i] * grad[i];
46                 x[i] -= alpha * grad[i] / (Math.Sqrt(g[i]) + 1e-8);
47             }
48             return x;
49         }
50     }
51 }

```

```

47 g[i] += grad[i] * grad[i];
48 x[i] -= alpha * grad[i] / (Math.Sqrt(g[i]) + 1e-8);
49 }
50 return x;
51 }
52 }
53 }

```

Mathematical formulation:

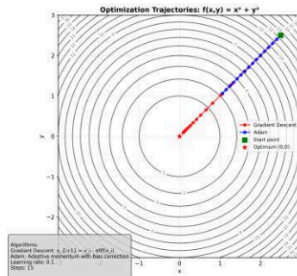
$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} - \alpha \nabla f(\mathbf{x}_t) \quad (\text{Momentum})$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_t$$

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) (\nabla f(\mathbf{x}_t))^2 \quad (\text{RMSprop})$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha \frac{\nabla f(\mathbf{x}_t)}{\sqrt{v_t} + \epsilon}$$

Visual output: A contour plot showing optimization trajectories for different algorithms, with momentum showing overshooting and adaptive methods showing smoother convergence.



6.3 Constrained Optimization

Listing 23: Constrained Optimization in C#

```

1 public class ConstrainedOptimization
2 {
3     public static double[] ProjectedGradientDescent(Func<double[],
4         double> f, double[] x0, Func<double[], double[]>
5         projection, double learningRate = 0.01)
6     {
7         double[] x = (double[])x0.Clone();
8         for (int iter = 0; iter < 1000; iter++)
9         {
10             double[] grad = Gradient(f, x);
11             for (int i = 0; i < x.Length; i++)
12                 x[i] -= learningRate * grad[i];
13             x = projection(x);
14             return x;
15         }
16     }
17     public static double[] L2Projection(double[] x, double radius)
18     {
19         double norm = Math.Sqrt(x.Select(xi => xi * xi).Sum());
20         if (norm <= radius) return x;
21         double scale = radius / norm;
22         return x.Select(xi => xi * scale).ToArray();
23     }
24     public static double[] SimplexProjection(double[] x)
25     {
26         double[] sorted = (double[])x.Clone();
27         Array.Sort(sorted);
28         Array.Reverse(sorted);
29         double sum = 0;
30         int k = 0;
31         for (int i = 0; i < sorted.Length; i++)
32         {
33             sum += sorted[i];
34             if (sorted[i] * (i + 1) - sum) / (i + 1) > 0)
35                 k = i + 1;
36         }
37         double[] result = new double[x.Length];
38         for (int i = 0; i < x.Length; i++)
39             result[i] = sorted[i] * k / sorted.Length;
40         return result;
41     }
42 }

```

```

42 }
43
44 double theta = (sum - 1) / k;
45 return x.Select(xi => Math.Max(0, xi - theta)).ToArray();
46 }
47 }

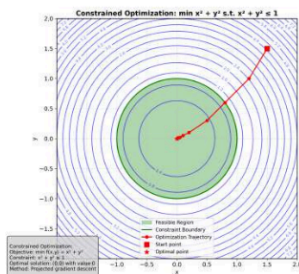
```

Mathematical formulation:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{x} \in C$$

$$\mathbf{x}_{t+1} = P_C(\mathbf{x}_t - \alpha \nabla f(\mathbf{x}_t))$$

Visual output: A 2D plot showing the feasible region as a circle or simplex, with optimization trajectory constrained to stay within the region.



6.4 Stochastic Optimization

Listing 24: Stochastic Optimization in C#

```

1 public class StochasticOptimization
2 {
3     public static double[] StochasticGradientDescent(Func<double[],
4         double> f, double[] batchFunctions, double[] x0, double
5         learningRate = 0.01)
6     {
7         double[] x = (double[])x0.Clone();
8         Random rng = new Random();
9         for (int epoch = 0; epoch < 100; epoch++)
10         {
11             // Shuffle batch functions
12             var shuffled = batchFunctions.OrderBy(f => rng.Next()).
13                 ToArray();
14             for (int i = 0; i < shuffled.Length; i++)
15             {
16                 double[] grad = Gradient(shuffled[i], x);
17                 for (int j = 0; j < x.Length; j++)
18                     x[j] -= learningRate * grad[j];
19             }
20             return x;
21         }
22     }
23     public static double[] MiniBatchGradientDescent(Func<double[],
24         double> f, double[] batchFunctions, double[] x0, int batchSize =
25         32, double learningRate = 0.01)
26     {
27         double[] x = (double[])x0.Clone();
28         Random rng = new Random();
29         for (int epoch = 0; epoch < 100; epoch++)
30         {
31             var shuffled = batchFunctions.OrderBy(f => rng.Next()).
32                 ToArray();
33             for (int i = 0; i < shuffled.Length; i += batchSize)
34             {
35                 int end = Math.Min(i + batchSize, shuffled.Length);
36                 double[] avgGrad = new double[x.Length];

```

```

37         for (int j = i; j < end; j++)
38         {
39             double[] grad = Gradient(shuffled[j], x);
40             for (int k = 0; k < x.Length; k++)
41                 avgGrad[k] += grad[k];
42         }
43         for (int k = 0; k < x.Length; k++)
44         {
45             avgGrad[k] /= (end - i);
46             x[k] -= learningRate * avgGrad[k];
47         }
48         return x;
49     }
50 }
51 }
52 }
53 }

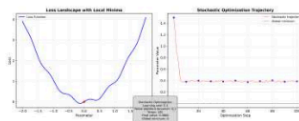
```

Mathematical formulation:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha \nabla f_t(\mathbf{x}_t) \quad (\text{SGD})$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha \frac{1}{|B|} \sum_{i \in B} \nabla f_i(\mathbf{x}_t) \quad (\text{Mini-batch})$$

Visual output: A noisy optimization trajectory showing how stochastic methods explore the loss landscape with random sampling.



7 Linear Models

Linear models form the foundation of supervised learning [9], providing interpretable and efficient approaches to regression and classification problems. These models establish the mathematical basis for understanding more complex machine learning algorithms [3].

7.1 Linear Regression

Listing 25: Linear Regression in C#

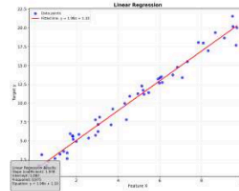
```
1 public class LinearRegression
2 {
3     public double[] Weights { get; private set; }
4     public double Bias { get; private set; }
5
6     public LinearRegression(int inputSize)
7     {
8         Weights = new double[inputSize];
9         Bias = 0;
10    }
11
12    public double Predict(double[] features)
13    {
14        double prediction = Bias;
15        for (int i = 0; i < features.Length; i++)
16            prediction += Weights[i] * features[i];
17        return prediction;
18    }
19
20    public void Fit(double[][] X, double[] y, double learningRate =
21        0.01, int epochs = 1000)
22    {
23        int n = X.Length;
24        int d = X[0].Length;
25        for (int epoch = 0; epoch < epochs; epoch++)
26        {
27            double[] gradWeights = new double[d];
28            double gradBias = 0;
29            for (int i = 0; i < n; i++)
30            {
31                double prediction = Predict(X[i]);
32                double error = prediction - y[i];
33                for (int j = 0; j < d; j++)
34                {
35                    gradWeights[j] += error * X[i][j];
36                    gradBias += error;
37                }
38            }
39            for (int j = 0; j < d; j++)
```

```
41         Weights[j] -= learningRate * gradWeights[j] / n;
42         Bias -= learningRate * gradBias / n;
43     }
44 }
45 }
```

Mathematical formulation:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$
$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{ij}$$

Visual output: A scatter plot of data points with a fitted straight line showing the linear relationship between features and target.



7.2 Logistic Regression

Listing 26: Logistic Regression in C#

```
1 public class LogisticRegression
2 {
3     public double[] Weights { get; private set; }
4     public double Bias { get; private set; }
5
6     public LogisticRegression(int inputSize)
7     {
8         Weights = new double[inputSize];
9         Bias = 0;
10    }
11
12    public double Predict(double[] features)
13    {
14        double logit = Bias;
15        for (int i = 0; i < features.Length; i++)
16            logit += Weights[i] * features[i];
17        return Sigmoid(logit);
18    }
19
20    private double Sigmoid(double x)
21    {
22        return 1.0 / (1.0 + Math.Exp(-x));
23    }
24
25    public void Fit(double[][] X, int[] y, double learningRate =
26        0.01, int epochs = 1000)
27    {
28        int n = X.Length;
```

```
29
30        for (int epoch = 0; epoch < epochs; epoch++)
31        {
32            double[] gradWeights = new double[d];
33            double gradBias = 0;
34            for (int i = 0; i < n; i++)
35            {
36                double prediction = Predict(X[i]);
37                double error = prediction - y[i];
38                for (int j = 0; j < d; j++)
39                {
40                    gradWeights[j] += error * X[i][j];
41                    gradBias += error;
42                }
43            }
44            for (int j = 0; j < d; j++)
45            {
46                Weights[j] -= learningRate * gradWeights[j] / n;
47                Bias -= learningRate * gradBias / n;
48            }
49        }
50    }
```

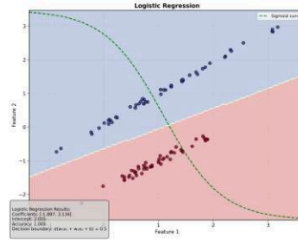
Mathematical formulation:

$$P(y = 1|x) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$L(\mathbf{w}, b) = - \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Visual output: A scatter plot with two classes of points and a sigmoid curve showing the decision boundary.



7.3 Softmax Regression

Listing 27: Softmax Regression in C#

```
1 public class SoftmaxRegression
2 {
3     public double[,] Weights { get; private set; }
4     public double[] Biases { get; private set; }
5     public int NumClasses { get; }
6
7     public SoftmaxRegression(int inputSize, int numClasses)
8     {
9         Weights = new double[inputSize, numClasses];
10        Biases = new double[numClasses];
11        NumClasses = numClasses;
12    }
13
14    public double[] Predict(double[] features)
15    {
16        double[] logits = new double[NumClasses];
17        for (int k = 0; k < NumClasses; k++)
18        {
19            logits[k] = Biases[k];
20            for (int i = 0; i < features.Length; i++)
21                logits[k] += Weights[i, k] * features[i];
22        }
23        return Softmax(logits);
24    }
25
26    private double[] Softmax(double[] logits)
27    {
28        double maxLogit = logits.Max();
29        double[] expLogits = logits.Select(x => Math.Exp(x - maxLogit)).ToArray();
30        double sum = expLogits.Sum();
31        return expLogits.Select(x => x / sum).ToArray();
32    }
33
34    public void Fit(double[][] X, int[] y, double learningRate = 0.01, int epochs = 1000)
35    {
36        int n = X.Length;
37        int d = X[0].Length;
38        for (int epoch = 0; epoch < epochs; epoch++)
39        {
40            double[,] gradWeights = new double[d, NumClasses];
41            double[] gradBiases = new double[NumClasses];
42            for (int i = 0; i < n; i++)
43            {
44                double[] predictions = Predict(X[i]);
45                int trueClass = y[i];
46                for (int k = 0; k < NumClasses; k++)
47                {
48                    double error = predictions[k] - (k == trueClass ? 1 : 0);
49                }
50            }
51        }
52    }
53 }
```

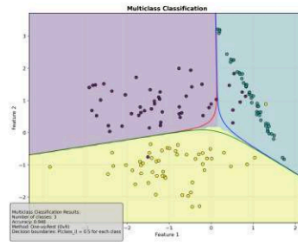
```
54 gradBiases[k] += error;
55
56 for (int j = 0; j < d; j++)
57     gradWeights[j, k] += error * X[i][j];
58 }
59
60 for (int k = 0; k < NumClasses; k++)
61 {
62     Biases[k] -= learningRate * gradBiases[k] / n;
63     for (int j = 0; j < d; j++)
64         Weights[j, k] -= learningRate * gradWeights[j, k] / n;
65 }
66 }
67 }
68 }
69 }
```

Mathematical formulation:

$$P(y = k|x) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

$$L(\mathbf{W}, \mathbf{b}) = - \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik})$$

Visual output: A multi-class scatter plot with decision boundaries showing how the model separates different classes.



7.4 Ridge and Lasso Regression

Listing 28: Regularized Regression in C#

```
1 public class RegularizedRegression
2 {
3     public static double[] RidgeRegression(double[][] X, double[] y,
4     , double lambda = 0.1)
5     {
6         int n = X.Length;
7         int d = X[0].Length;
```

```

7 // Add regularization term to normal equations
8 double[] XTX = new double[d, d];
9 double[] XTy = new double[d];
10
11 for (int i = 0; i < d; i++)
12 {
13     for (int j = 0; j < d; j++)
14     {
15         for (int k = 0; k < n; k++)
16             XTX[i, j] += X[k][i] * X[k][j];
17         if (i == j) XTX[i, j] += lambda;
18     }
19     for (int k = 0; k < n; k++)
20         XTy[i] += X[k][i] * y[k];
21 }
22
23 return SolveLinearSystem(XTX, XTy);
24
25
26 public static double[] LassoRegression(double[][] X, double[] y,
27 double lambda = 0.1, int maxIter = 1000)
28 {
29     int n = X.Length;
30     int d = X[0].Length;
31     double[] weights = new double[d];
32     for (int iter = 0; iter < maxIter; iter++)
33     {
34         for (int j = 0; j < d; j++)
35         {
36             double numerator = 0;
37             double denominator = 0;
38             for (int i = 0; i < n; i++)
39             {
40                 double residual = y[i];
41                 for (int k = 0; k < d; k++)
42                     if (k != j) residual -= weights[k] * X[i][k];
43             }
44         }
45     }

```

```

46         numerator += X[i][j] * residual;
47         denominator += X[i][j] * X[i][j];
48     }
49     double z = numerator / denominator;
50     weights[j] = Math.Sign(z) * Math.Max(0, Math.Abs(z)
51         - lambda / denominator);
52 }
53
54 }
55
56 return weights;
57
58 }

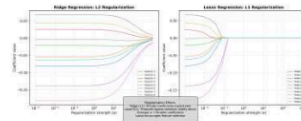
```

Mathematical formulation:

$$L_{Ridge}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2$$

$$L_{Lasso}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1$$

Visual output: A coefficient plot showing how regularization shrinks coefficients, with Lasso producing sparse solutions (many zeros).



8 Neural Networks

Neural networks represent the cutting edge of machine learning [10], combining mathematical concepts from linear algebra, calculus, and optimization. Understanding their mathematical foundations is essential for designing effective deep learning architectures [14].

8.1 Feedforward Neural Networks

Listing 29: Feedforward Neural Network in C#

```

33
34 public double[] Forward(double[] input)
35 {
36     double[] current = input;
37     for (int layer = 0; layer < Weights.Length; layer++)
38     {
39         int inputSize = Weights[layer].GetLength(0);
40         int outputSize = Weights[layer].GetLength(1);
41         double[] output = new double[outputSize];
42         for (int j = 0; j < outputSize; j++)
43         {
44             output[j] = Biases[layer][j];
45             for (int i = 0; i < inputSize; i++)
46                 output[j] += current[i] * Weights[layer][i, j];
47         }
48         current = ApplyActivation(output);
49     }
50     return current;
51 }
52
53 private double[] ApplyActivation(double[] x)
54 {
55     return x.Select(xi => ReLU(xi)).ToArray();
56 }
57
58 private double ReLU(double x)
59 {
60     return Math.Max(0, x);
61 }
62
63 }
64
65 }
66

```

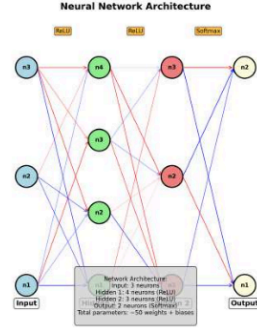
Mathematical formulation:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

$$\sigma(x) = \max(0, x) \quad (\text{ReLU})$$

Visual output: A network diagram showing layers of neurons connected by weighted edges, with activation functions applied at each layer.



8.2 Backpropagation

Listing 30: Backpropagation in C#

```

1 public class Backpropagation
2 {
3     public static void Train(NeuralNetwork network, double[][] X,
4         double[] y, double learningRate = 0.01, int epochs =
5         1000)
6     {
7         int n = X.Length;
8         for (int epoch = 0; epoch < epochs; epoch++)
9         {
10             for (int i = 0; i < n; i++)
11             {
12                 // Forward pass
13                 var activations = ForwardPass(network, X[i]);
14                 var zValues = GetZValues(network, X[i]);
15                 // Backward pass
16                 var deltas = BackwardPass(network, activations,
17                     zValues, y[i]);
18                 // Update weights and biases
19                 UpdateParameters(network, X[i], activations, deltas,
20                     learningRate);
21             }
22         }
23     }
24     private static double[] ForwardPass(NeuralNetwork network,
25         double[] input)
26     {
27         double[] current = input;
28         var activations = new List<double[]> { current };
29         for (int layer = 0; layer < network.Weights.Length; layer++)
30         {
31             int inputSize = network.Weights[layer].GetLength(0);
32             int outputSize = network.Weights[layer].GetLength(1);
33             double[] output = new double[outputSize];
34             for (int j = 0; j < outputSize; j++)
35             {
36                 output[j] = network.Biases[layer][j];
37                 for (int i = 0; i < inputSize; i++)
38                     output[j] += current[i] * network.Weights[layer][i, j];
39             }
40             current = output.Select(x => ReLU(x)).ToArray();
41             activations.Add(current);
42         }
43         return current;
44     }
45     private static double[] BackwardPass(NeuralNetwork network,
46         double[] activations, double[] zValues, double target)
47     {
48         int numLayers = network.Weights.Length;
49         var deltas = new double[numLayers][];
50         // Output layer delta
51         deltas[numLayers - 1] = new double[activations.Length];
52         for (int j = 0; j < activations.Length; j++)
53         {
54             deltas[numLayers - 1][j] = (activations[j] - target) *
55                 ReLUDerivative(zValues[j]);
56         }
57         // Hidden layer deltas
58         for (int layer = numLayers - 2; layer >= 0; layer--)
59         {
60             int nextLayerSize = network.Weights[layer + 1].
61                 GetLength(1);
62             deltas[layer] = new double[nextLayerSize];
63             for (int j = 0; j < nextLayerSize; j++)
64             {
65                 double error = 0;
66                 for (int k = 0; k < deltas[layer + 1].Length; k++)
67                     error += deltas[layer + 1][k] * network.Weights
68                         [layer + 1][j, k];
69                 deltas[layer][j] = error * ReLUDerivative(zValues[j]);
70             }
71         }
72         return deltas;
73     }
74     private static double ReLUDerivative(double x)
75     {
76         return x > 0 ? 1 : 0;
77     }
78 }

```

Mathematical formulation:

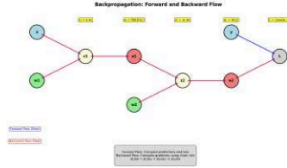
$$\delta^{(L)} = \nabla_a C \odot \sigma'(\mathbf{z}^{(L)})$$

$$\delta^{(l)} = ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$$

$$\frac{\partial C}{\partial w_{ij}^{(l)}} = a_j^{(l-1)} \delta_i^{(l)}$$

$$\frac{\partial C}{\partial b_i^{(l)}} = \delta_i^{(l)}$$

Visual output: A computational graph showing the forward and backward flow of gradients through the network layers.



8.3 Activation Functions: Sigmoid, ReLU, Tanh

Listing 31: Activation Functions in C#

```
1 public class ActivationFunctions
2 {
3     public static double Sigmoid(double x)
4     {
5         return 1.0 / (1.0 + Math.Exp(-x));
6     }
7
8     public static double SigmoidDerivative(double x)
9     {
10        double s = Sigmoid(x);
11        return s * (1 - s);
12    }
13
14    public static double ReLU(double x)
15    {
16        return Math.Max(0, x);
17    }
18
19    public static double ReLUDerivative(double x)
20    {
21        return x > 0 ? 1 : 0;
22    }
23
24    public static double Tanh(double x)
25    {
26        return Math.Tanh(x);
27    }
28
29    public static double TanhDerivative(double x)
30    {
31        double t = Math.Tanh(x);
32        return 1 - t * t;
33    }
34
35    public static double LeakyReLU(double x, double alpha = 0.01)
36    {
37        return x > 0 ? x : alpha * x;
38    }
39
40    public static double LeakyReLUDerivative(double x, double alpha = 0.01)
41    {
42        return x > 0 ? 1 : alpha;
43    }
44 }
```

Mathematical formulation:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (\text{Sigmoid})$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

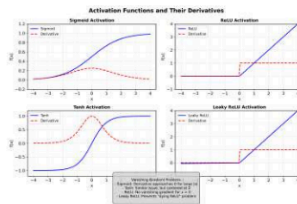
$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh'(x) = 1 - \tanh^2(x)$$

Visual output: A plot showing the shapes of different activation functions and their derivatives, highlighting the vanishing gradient problem in sigmoid and the benefits of ReLU.



8.4 Convolutional Neural Networks

Listing 32: Convolutional Neural Network in C#

```
1 public class ConvolutionalLayer
2 {
3     public int InputChannels { get; }
4     public int OutputChannels { get; }
5     public int KernelSize { get; }
6     public double[,] Kernels { get; }
7     public double[] Biases { get; }
8
9     public ConvolutionalLayer(int inputChannels, int outputChannels,
10        int kernelSize)
11    {
12        InputChannels = inputChannels;
13        OutputChannels = outputChannels;
14        KernelSize = kernelSize;
15        Kernels = new double[outputChannels, inputChannels,
16            kernelSize, kernelSize];
17        Biases = new double[outputChannels];
18        InitializeParameters();
19    }
20
21    private void InitializeParameters()
22    {
23        Random rng = new Random();
24        double std = Math.Sqrt(2.0 / (InputChannels * KernelSize *
25            KernelSize));
26        for (int oc = 0; oc < OutputChannels; oc++)
27        {
28            for (int ic = 0; ic < InputChannels; ic++)
29            {
30                for (int i = 0; i < KernelSize; i++)
31                {
32                    for (int j = 0; j < KernelSize; j++)
33                        Kernels[oc, ic, i, j] = (rng.NextDouble() -
34                            0.5) * 2 * std;
35                }
36            }
37            Biases[oc] = 0;
38        }
39    }
40 }
```

```

38 public double[,] Convolve(double[,] input, double[,] kernel)
39 {
40     int inputHeight = input.GetLength(0);
41     int inputWidth = input.GetLength(1);
42     int outputHeight = inputHeight - KernelSize + 1;
43     int outputWidth = inputWidth - KernelSize + 1;
44     double[,] output = new double[outputHeight, outputWidth];
45
46     for (int i = 0; i < outputHeight; i++)
47     {
48         for (int j = 0; j < outputWidth; j++)
49         {
50             double sum = 0;
51             for (int ki = 0; ki < KernelSize; ki++)
52             {
53                 for (int kj = 0; kj < KernelSize; kj++)
54                 {
55                     sum += input[i + ki, j + kj] * kernel[ki, kj];
56                 }
57             }
58             output[i, j] = sum;
59         }
60     }
61     return output;
62 }
63
64 public double[,] Forward(double[,] input)
65 {
66     int inputHeight = input.GetLength(0);
67     int inputWidth = input.GetLength(1);
68     int outputHeight = inputHeight - KernelSize + 1;
69     int outputWidth = inputWidth - KernelSize + 1;
70     double[,] output = new double[outputHeight, outputWidth,
71     OutputChannels];
72
73     for (int oc = 0; oc < OutputChannels; oc++)
74     {
75         for (int ic = 0; ic < InputChannels; ic++)
76         {
77             double[,] inputChannel = new double[inputHeight,
78             inputWidth];
79             for (int i = 0; i < inputHeight; i++)
80             {
81                 for (int j = 0; j < inputWidth; j++)
82                 {
83                     inputChannel[i, j] = input[i, j, ic];
84                 }
85             }
86             double[,] kernel = new double[KernelSize,
87             KernelSize];
88             for (int i = 0; i < KernelSize; i++)
89             {
90                 for (int j = 0; j < KernelSize; j++)
91                 {
92                     kernel[i, j] = kernel[oc, ic, i, j];
93                 }
94             }
95             double[,] convolved = Convolve(inputChannel, kernel);
96         }
97     }
98     // Add bias and apply activation
99     for (int i = 0; i < outputHeight; i++)
100     {
101         for (int j = 0; j < outputWidth; j++)
102         {
103             output[i, j, oc] = ReLU(output[i, j, oc] +
104             Biases[oc]);
105         }
106     }
107     return output;
108 }

```

```

87 double[,] convolved = Convolve(inputChannel, kernel);
88 }
89 for (int i = 0; i < outputHeight; i++)
90 {
91     for (int j = 0; j < outputWidth; j++)
92     {
93         output[i, j, oc] += convolved[i, j];
94     }
95 }
96 // Add bias and apply activation
97 for (int i = 0; i < outputHeight; i++)
98 {
99     for (int j = 0; j < outputWidth; j++)
100     {
101         output[i, j, oc] = ReLU(output[i, j, oc] +
102         Biases[oc]);
103     }
104 }
105 return output;
106 }

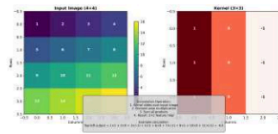
```

Mathematical formulation:

$$(I * K)_{ij} = \sum_{m=0}^{I-1} \sum_{n=0}^{K-1} I_{i+m, j+n} K_{m,n}$$

$$\text{ReLU}(x) = \max(0, x)$$

Visual output: A diagram showing how a kernel slides over an input image, performing element-wise multiplication and summation to produce feature maps.



9 Clustering

Clustering algorithms provide fundamental tools for unsupervised learning and data exploration [1]. These methods group similar data points together, enabling pattern discovery and data organization without predefined labels.

9.1 K-Means

Listing 33: K-Means Clustering in C#

```

1 public class KMeans
2 {
3     public double[][] Centroids { get; private set; }
4     public int[] Labels { get; private set; }
5     public int K { get; }
6
7     public KMeans(int k)
8     {
9         K = k;
10        Centroids = new double[k][];
11        Labels = new int[0];
12    }
13
14    public void Fit(double[][] data, int maxIterations = 100)
15    {
16        int n = data.Length;
17        int d = data[0].Length;
18        Labels = new int[n];
19
20        // Initialize centroids randomly
21        Random rng = new Random();
22        for (int k = 0; k < K; k++)
23        {
24            Centroids[k] = new double[d];

```

```

25 int randomIndex = rng.Next(n);
26 for (int j = 0; j < d; j++)
27     Centroids[k][j] = data[randomIndex][j];
28 }
29
30 for (int iteration = 0; iteration < maxIterations;
31 iteration++)
32 {
33     bool converged = true;
34
35     // Assignment step
36     for (int i = 0; i < n; i++)
37     {
38         int bestCluster = 0;
39         double bestDistance = double.MaxValue;
40         for (int k = 0; k < K; k++)
41         {
42             double distance = EuclideanDistance(data[i],
43             Centroids[k]);
44             if (distance < bestDistance)
45             {
46                 bestDistance = distance;
47                 bestCluster = k;
48             }
49         }

```

```

50         if (Labels[i] != bestCluster)
51         {
52             Labels[i] = bestCluster;
53             converged = false;
54         }
55     }
56
57     if (converged) break;
58
59     // Update step
60     for (int k = 0; k < K; k++)
61     {
62         var clusterPoints = new List<double[]>();
63         for (int i = 0; i < n; i++)
64         {
65             if (Labels[i] == k)
66                 clusterPoints.Add(data[i]);
67         }
68         if (clusterPoints.Count > 0)
69         {
70             for (int j = 0; j < d; j++)
71             {
72                 double sum = 0;
73                 foreach (var point in clusterPoints)
74                     sum += point[j];
75                 Centroids[k][j] = sum / clusterPoints.Count;
76             }
77         }
78     }
79
80 }
81
82 private double EuclideanDistance(double[] a, double[] b)
83 {
84     double sum = 0;
85     for (int i = 0; i < a.Length; i++)
86         sum += Math.Pow(a[i] - b[i], 2);
87     return Math.Sqrt(sum);
88 }
89
90 public double Inertia()
91 {
92     double inertia = 0;
93     for (int i = 0; i < Labels.Length; i++)
94     {
95

```

```

96         inertia += Math.Pow(EuclideanDistance(data[i],
97             Centroids[Labels[i]], 2);
98     }
99     return inertia;
100 }

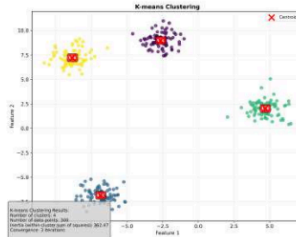
```

Mathematical formulation:

$$\min_{C, z} \sum_{i=1}^n \sum_{k=1}^K z_{ik} \|x_i - c_k\|^2$$

$$\text{s.t. } \sum_{k=1}^K z_{ik} = 1, \quad z_{ik} \in \{0, 1\}$$

Visual output: A scatter plot showing data points colored by cluster assignment, with centroids marked as larger points.



9.2 DBSCAN

Listing 34: DBSCAN Clustering in C#

```

1 public class DBSCAN
2 {
3     public int[] Labels { get; private set; }
4     public double Epsilon { get; }
5     public int MinPoints { get; }
6
7     public DBSCAN(double epsilon, int minPoints)
8     {
9         Epsilon = epsilon;
10        MinPoints = minPoints;
11    }
12
13    public void Fit(double[][] data)
14    {
15        int n = data.Length;
16        Labels = new int[n];
17        for (int i = 0; i < n; i++)
18            Labels[i] = -1; // Unassigned
19
20        int clusterId = 0;
21        for (int i = 0; i < n; i++)
22        {
23            if (Labels[i] != -1) continue;
24
25            var neighbors = GetNeighbors(data, i);
26            if (neighbors.Count < MinPoints)
27            {
28                Labels[i] = -1; // Noise
29            }
30            else
31            {
32                clusterId++;
33                ExpandCluster(data, i, neighbors, clusterId);
34            }
35        }
36    }
37
38    private List<int> GetNeighbors(double[][] data, int pointIndex)
39    {
40

```

```

41        var neighbors = new List<int>();
42        for (int i = 0; i < data.Length; i++)
43        {
44            if (i != pointIndex && EuclideanDistance(data[
45                pointIndex], data[i]) <= Epsilon)
46                neighbors.Add(i);
47        }
48        return neighbors;
49    }
50
51    private void ExpandCluster(double[][] data, int pointIndex,
52        List<int> neighbors, int clusterId)
53    {
54        Labels[pointIndex] = clusterId;
55        for (int i = 0; i < neighbors.Count; i++)
56        {
57            int neighborIndex = neighbors[i];
58            if (Labels[neighborIndex] == -1)
59            {
60                Labels[neighborIndex] = clusterId;
61                var newNeighbors = GetNeighbors(data, neighborIndex);
62                if (newNeighbors.Count >= MinPoints)
63                {
64                    foreach (var newNeighbor in newNeighbors)
65                    {
66                        if (!neighbors.Contains(newNeighbor))
67                            neighbors.Add(newNeighbor);
68                    }
69                }
70            }
71        }
72    }
73
74    private double EuclideanDistance(double[] a, double[] b)
75    {
76        double sum = 0;
77        for (int i = 0; i < a.Length; i++)
78            sum += Math.Pow(a[i] - b[i], 2);
79        return Math.Sqrt(sum);
80    }
81
82 }

```

```

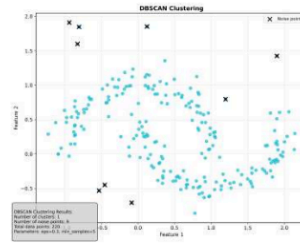
83 public int GetNumberOfClusters()
84 {
85     return Labels.Max() + 1;
86 }
87
88 public int GetNumberOfNoisePoints()
89 {
90     return Labels.Count(l => l == -1);
91 }
92
93 }

```

Mathematical formulation:

Core point: $|N_e(p)| \geq \text{MinPts}$
 Border point: $|N_e(p)| < \text{MinPts}$ but $p \in N_e(q)$ for some core q
 Noise point: Neither core nor border

Visual output: A scatter plot showing clusters as connected components, with noise points isolated and core points densely connected.



9.3 Hierarchical Clustering

Listing 35: Hierarchical Clustering in C#

```

1 public class HierarchicalClustering
2 {
3     public double[][] LinkageMatrix { get; private set; }
4     public int[] Labels { get; private set; }
5
6     public void Fit(double[][] data, int nClusters)
7     {
8         int n = data.Length;
9         var clusters = new List<List<int>>();
10
11         // Initialize each point as its own cluster
12         for (int i = 0; i < n; i++)
13             clusters.Add(new List<int> { i });
14
15         var linkageMatrix = new List<double>[]();
16
17         while (clusters.Count > nClusters)
18         {
19             double minDistance = double.MaxValue;
20             int cluster1 = -1, cluster2 = -1;
21
22             // Find the closest pair of clusters
23             for (int i = 0; i < clusters.Count; i++)
24             {
25                 for (int j = i + 1; j < clusters.Count; j++)
26                 {
27                     double distance = CompleteLinkage(data,
28                                                         clusters[i], clusters[j]);
29                     if (distance < minDistance)
30                     {
31                         minDistance = distance;
32                         cluster1 = i;
33                         cluster2 = j;
34                     }
35                 }
36             }
37
38             // Merge clusters
39             clusters[cluster1].AddRange(clusters[cluster2]);
40             clusters.RemoveAt(cluster2);
41
42             linkageMatrix.Add(new double[] { cluster1, cluster2,
43                                             minDistance, clusters[cluster1].Count });
44
45             LinkageMatrix = linkageMatrix.ToArray();
46
47             // Assign labels
48             Labels = new int[n];
49             for (int i = 0; i < clusters.Count; i++)
50             {
51                 foreach (int pointIndex in clusters[i])
52                     Labels[pointIndex] = i;
53             }
54
55             private double CompleteLinkage(double[][] data, List<int>
56                 cluster1, List<int> cluster2)
57             {
58                 double maxDistance = 0;
59                 foreach (int i in cluster1)
60                 {
61                     foreach (int j in cluster2)
62                     {
63                         double distance = EuclideanDistance(data[i], data[j]);
64                         if (distance > maxDistance)
65                             maxDistance = distance;
66                     }
67                 }
68                 return maxDistance;
69             }
70
71             private double SingleLinkage(double[][] data, List<int>
72                 cluster1, List<int> cluster2)
73             {
74                 double minDistance = double.MaxValue;
75                 foreach (int i in cluster1)
76                 {
77                     foreach (int j in cluster2)
78                     {
79                         double distance = EuclideanDistance(data[i], data[j]);
80                         if (distance < minDistance)
81                             minDistance = distance;
82                     }
83                 }
84                 return minDistance;
85             }
86
87             private double EuclideanDistance(double[] a, double[] b)
88             {
89                 double sum = 0;
90                 for (int i = 0; i < a.Length; i++)
91                     sum += Math.Pow(a[i] - b[i], 2);
92                 return Math.Sqrt(sum);
93             }
94
95         }
96     }
97 }

```

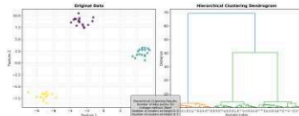
Mathematical formulation:

$$d_{complete}(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y)$$

$$d_{single}(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$$

$$d_{average}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

Visual output: A dendrogram showing the hierarchical structure of clusters, with height representing the distance at which clusters were merged.



9.4 Gaussian Mixture Models

Listing 36: Gaussian Mixture Models in C#

```
1 public class GaussianMixtureModel
2 {
3     public double[] Weights { get; private set; }
4     public double[][] Means { get; private set; }
5     public double[][] Covariances { get; private set; }
6     public int[] Labels { get; private set; }
7     public int K { get; }
8
9     public GaussianMixtureModel(int k)
10    {
11        K = k;
12        Weights = new double[k];
13        Means = new double[k][d];
14        Covariances = new double[k][d][d];
15        for (int i = 0; i < k; i++)
16            Weights[i] = 1.0 / k;
17    }
18
19    public void Fit(Double[][] data, int maxIterations = 100)
20    {
21        int n = data.Length;
22        int d = data[0].Length;
23        Labels = new int[n];
24
25        // Initialize parameters
26        Random rng = new Random();
27        for (int k = 0; k < K; k++)
28        {
29            Means[k] = new double[d];
30            int randomIndex = rng.Next(n);
31            for (int j = 0; j < d; j++)
32                Means[k][j] = data[randomIndex][j];
33            Covariances[k] = new double[d][d];
34            for (int i = 0; i < d; i++)
35                Covariances[k][i][i] = 1.0;
36        }
37
38        for (int iteration = 0; iteration < maxIterations; iteration++)
39        {
40            // E-step: Compute responsibilities
41            double[,] responsibilities = new double[n, K];
42            for (int i = 0; i < n; i++)
43            {
44                double sum = 0;
45                for (int k = 0; k < K; k++)
46                {
47                    responsibilities[i, k] = Weights[k] *
48                        GaussianPDF(data[i], Means[k],
49                            Covariances[k]);
50                    sum += responsibilities[i, k];
51                }
52                for (int k = 0; k < K; k++)
53                    responsibilities[i, k] /= sum;
54            }
55
56            // M-step: Update parameters
57            for (int k = 0; k < K; k++)
58            {
59                double Nk = 0;
60                for (int i = 0; i < n; i++)
61                    Nk += responsibilities[i, k];
62                Weights[k] = Nk / n;
63
64                // Update mean
65                for (int j = 0; j < d; j++)
66                {
67                    Means[k][j] = 0;
68                    for (int i = 0; i < n; i++)
69                        Means[k][j] += responsibilities[i, k] *
70                            data[i][j];
71                    Means[k][j] /= Nk;
72                }
73
74                // Update covariance
75
```

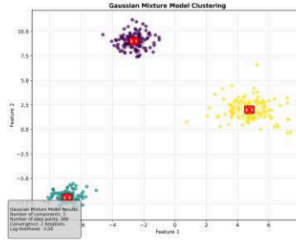
```
76
77         for (int i = 0; i < d; i++)
78         {
79             for (int j = 0; j < d; j++)
80             {
81                 Covariances[k][i, j] = 0;
82                 for (int m = 0; m < Nk; m++)
83                     Covariances[k][i, j] +=
84                         responsibilities[m, k] *
85                         (data[m][i] - Means[k][i]) * (data[
86                             m][j] - Means[k][j]);
87                 Covariances[k][i, j] /= Nk;
88             }
89         }
90     }
91
92     // Assign labels based on maximum responsibility
93     for (int i = 0; i < n; i++)
94     {
95         int bestCluster = 0;
96         double maxResponsibility = 0;
97         for (int k = 0; k < K; k++)
98         {
99             double resp = Weights[k] * GaussianPDF(data[i],
100                 Means[k], Covariances[k]);
101             if (resp > maxResponsibility)
102             {
103                 maxResponsibility = resp;
104                 bestCluster = k;
105             }
106             Labels[i] = bestCluster;
107         }
108     }
109
110     private double GaussianPDF(double[] x, double[] mean, double[,]
111         covariance)
112     {
113         int d = x.Length;
114         double det = Determinant(covariance);
115         double[,] inv = Inverse(covariance);
116         double[] diff = new double[d];
117         for (int i = 0; i < d; i++)
118             diff[i] = x[i] - mean[i];
119
120         double exponent = 0;
121         for (int i = 0; i < d; i++)
122         {
123             for (int j = 0; j < d; j++)
124                 exponent += diff[i] * inv[i, j] * diff[j];
125         }
126         return Math.Exp(-0.5 * exponent) / Math.Sqrt(Math.Pow(2 *
127             Math.PI, d) * det);
128     }
129 }
```

Mathematical formulation:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

Visual output: A scatter plot showing data points colored by cluster, with ellipses representing the Gaussian components and their covariance structure.



10 Dimensionality Reduction

Dimensionality reduction techniques address the curse of dimensionality by finding lower-dimensional representations of high-dimensional data [22]. These methods preserve important data characteristics while reducing computational complexity and improving model performance [11].

10.1 PCA: Projection, Eigen Decomposition, Variance Explanation

Listing 37: PCA Implementation in C#

```

1 public class PCA
2 {
3     public double[] ExplainedVariance { get; private set; }
4     public double[] ExplainedVarianceRatio { get; private set; }
5     public double[] Components { get; private set; }
6     public double[] Mean { get; private set; }
7
8     public PCA(int nComponents)
9     {
10         NComponents = nComponents;
11     }
12
13     public void Fit(double[][] X)
14     {
15         int n = X.Length;
16         int d = X[0].Length;
17
18         // Center the data
19         Mean = new double[d];
20         for (int j = 0; j < d; j++)
21         {
22             for (int i = 0; i < n; i++)
23                 Mean[j] += X[i][j];
24             Mean[j] /= n;
25         }
26
27         double[,] centered = new double[n, d];
28         for (int i = 0; i < n; i++)
29         {
30             for (int j = 0; j < d; j++)
31                 centered[i, j] = X[i][j] - Mean[j];
32         }
33
34         // Compute covariance matrix
35         double[,] covariance = new double[d, d];
36         for (int i = 0; i < d; i++)
37         {
38             for (int j = 0; j < d; j++)
39             {
40                 for (int k = 0; k < n; k++)
41                     covariance[i, j] += centered[k, i] * centered[k, j];
42             }
43             covariance[i, i] /= (n - 1);
44         }
45
46         // Eigen decomposition
47         var eigenDecomposition = new EigenDecomposition(covariance);
48
49         // Sort by eigenvalues (descending)
50         var sortedIndices = Enumerable.Range(0, d)
51             .OrderByDescending(i => eigenDecomposition.Eigenvalues[i])
52             .Take(NComponents)
53             .ToArray();
54
55         Components = new double[NComponents, d];
56         ExplainedVariance = new double[NComponents];
57         ExplainedVarianceRatio = new double[NComponents];
58
59         double totalVariance = eigenDecomposition.Eigenvalues.Sum();
60
61         for (int i = 0; i < NComponents; i++)
62         {
63             int idx = sortedIndices[i];
64             ExplainedVariance[i] = eigenDecomposition.Eigenvalues[idx];
65             ExplainedVarianceRatio[i] = eigenDecomposition.Eigenvalues[idx] / totalVariance;
66
67             for (int j = 0; j < d; j++)
68                 Components[i, j] = eigenDecomposition.Eigenvalues[idx, j];
69         }
70
71         public double[][] Transform(double[][] X)
72         {
73             int n = X.Length;
74             double[][] transformed = new double[n][NComponents];
75
76             for (int i = 0; i < n; i++)
77             {
78                 transformed[i] = new double[NComponents];
79                 for (int j = 0; j < NComponents; j++)
80                 {
81                     transformed[i][j] = 0;
82                     for (int k = 0; k < X[0].Length; k++)
83                         transformed[i][j] += (X[i][k] - Mean[k]) * Components[j, k];
84                 }
85             }
86
87             return transformed;
88         }
89
90         public double[][] InverseTransform(double[][] X)
91         {
92             int n = X.Length;
93             double[][] reconstructed = new double[n][d];
94
95             for (int i = 0; i < n; i++)
96             {
97                 reconstructed[i] = new double[d];
98                 for (int j = 0; j < Mean.Length; j++)
99                 {
100                     reconstructed[i][j] = Mean[j];
101                     for (int k = 0; k < NComponents; k++)
102                         reconstructed[i][j] += X[i][k] * Components[k, j];
103                 }
104             }
105
106             return reconstructed;
107         }
108     }

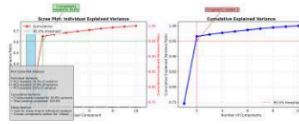
```

109

Mathematical formulation:

$$\begin{aligned}\mathbf{X}_{centered} &= \mathbf{X} - \bar{\mathbf{X}} \\ \Sigma &= \frac{1}{n-1} \mathbf{X}_{centered}^T \mathbf{X}_{centered} \\ \Sigma &= \mathbf{U} \Lambda \mathbf{U}^T \\ \mathbf{X}_{reduced} &= \mathbf{X}_{centered} \mathbf{U}_k\end{aligned}$$

Visual output: A scree plot showing explained variance ratio for each principal component, with cumulative variance reaching 95



10.2 t-SNE

Listing 38: t-SNE Implementation in C#

```

1 public class TSNE
2 {
3     public double Perplexity { get; }
4     public int NComponents { get; }
5     public double LearningRate { get; }
6
7     public TSNE(int nComponents = 2, double perplexity = 30.0,
8                 double learningRate = 200.0)
9     {
10         NComponents = nComponents;
11         Perplexity = perplexity;
12         LearningRate = learningRate;
13     }
14
15     public double[][] FitTransform(double[][] X, int maxIterations
16                                     = 1000)
17     {
18         int n = X.Length;
19         int d = X[0].Length;
20
21         // Compute pairwise distances
22         double[,] distances = new double[n, n];
23         for (int i = 0; i < n; i++)
24         {
25             for (int j = i + 1; j < n; j++)
26             {
27                 double dist = EuclideanDistance(X[i], X[j]);
28                 distances[i, j] = dist;
29                 distances[j, i] = dist;
30             }
31         }
32
33         // Compute P (joint probabilities in high-dimensional space)
34         double[,] P = new double[n, n];
35         for (int i = 0; i < n; i++)
36         {
37             double[] beta = BinarySearchBeta(distances, i,
38                                               Perplexity);
39             double sum = 0;
40             for (int j = 0; j < n; j++)
41             {
42                 if (i != j)
43                 {
44                     P[i, j] = Math.Exp(-distances[i, j] * beta[i]);
45                     sum += P[i, j];
46                 }
47             }
48             for (int j = 0; j < n; j++)
49                 P[i, j] /= sum;
50         }
51
52         // Symmetrize P
53         for (int i = 0; i < n; i++)
54         {
55             for (int j = 0; j < n; j++)
56                 P[i, j] = (P[i, j] + P[j, i]) / (2 * n);
57         }
58
59         // Initialize Y randomly
60         Random rng = new Random();
61         double[,] Y = new double[n, NComponents];
62         for (int i = 0; i < n; i++)
63         {
64             for (int j = 0; j < NComponents; j++)
65                 Y[i, j] = (rng.NextDouble() - 0.5) * 0.0001;
66         }
67
68         // Gradient descent
69         double[,] dY = new double[n, NComponents];
70         double[,] dY2 = new double[n, NComponents];
71
72         double[,] gains = new double[n, NComponents];
73         for (int iter = 0; iter < maxIterations; iter++)
74         {
75             // Compute Q (joint probabilities in low-dimensional
76             // space)
77             double[,] Q = new double[n, n];
78             double sumQ = 0;
79             for (int i = 0; i < n; i++)
80             {
81                 for (int j = i + 1; j < n; j++)
82                 {
83                     double dist = 0;
84                     for (int k = 0; k < NComponents; k++)
85                         dist += Math.Pow(Y[i, k] - Y[j, k], 2);
86                     Q[i, j] = 1 / (1 + dist);
87                     Q[j, i] = Q[i, j];
88                     sumQ += 2 * Q[i, j];
89                 }
90             }
91
92             for (int i = 0; i < n; i++)
93             {
94                 for (int j = 0; j < n; j++)
95                 {
96                     if (i != j)
97                     {
98                         double diff = Y[i, d] - Y[j, d];
99                         dY[i, d] += 4 * (P[i, j] - Q[i, j]) *
100                             diff;
101                     }
102                 }
103             }
104
105             // Update Y
106             for (int i = 0; i < n; i++)
107             {
108                 for (int d = 0; d < NComponents; d++)
109                 {
110                     gains[i, d] = (Math.Sign(dY[i, d]) != Math.Sign
111                                     (LY[i, d])) ?
112                         Math.Max(gains[i, d] *
113                                 0.8, 0.01);
114                     LY[i, d] = 0.5 * LY[i, d] - LearningRate *
115                         gains[i, d] * dY[i, d];
116                     Y[i, d] += LY[i, d];
117                 }
118             }
119
120             // Convert to array format
121             double[][] result = new double[n][];
122             for (int i = 0; i < n; i++)
123             {
124                 result[i] = new double[NComponents];
125                 for (int j = 0; j < NComponents; j++)
126                     result[i][j] = Y[i, j];
127             }
128
129             return result;
130         }
131     }
132 }

```

```

138 private double[] BinarySearchBeta(double[,] distances, int i,
139 double perplexity)
140 {
141     int n = distances.GetLength(0);
142     double[] beta = new double[n];
143     for (int j = 0; j < n; j++)
144     {
145         if (i != j)
146         {
147             double betaMin = -double.MaxValue;
148             double betaMax = double.MaxValue;
149             double beta_i = 1.0;
150             for (int iter = 0; iter < 50; iter++)
151             {
152                 double sum = 0;
153                 double sumDist = 0;
154                 for (int k = 0; k < n; k++)
155                 {
156                     if (k != i)
157                     {
158                         double p = Math.Exp(-distances[i, k] *
159                             beta_i);
160                         sum += p;
161                         sumDist += distances[i, k] * p;
162                     }
163                 }
164                 double entropy = Math.Log(sum) * beta_i +
165                     sumDist / sum;
166                 double entropyDiff = entropy - Math.Log(
167                     perplexity);
168                 if (Math.Abs(entropyDiff) < 1e-5) break;
169                 if (entropyDiff > 0)
170                 {
171                     betaMin = beta_i;
172                     if (betaMax == double.MaxValue)
173                         beta_i *= 2;
174                     else
175                         beta_i = (beta_i + betaMax) / 2;
176                 }
177             }
178         }
179     }
180     return beta;
181 }

```

```

179 }
180 else
181 {
182     betaMax = beta_i;
183     if (betaMin == -double.MaxValue)
184         beta_i /= 2;
185     else
186         beta_i = (beta_i + betaMin) / 2;
187 }
188 }
189 beta[j] = beta_i;
190 }
191 return beta;
192 }
193 private double EuclideanDistance(double[] a, double[] b)
194 {
195     double sum = 0;
196     for (int i = 0; i < a.Length; i++)
197         sum += Math.Pow(a[i] - b[i], 2);
198     return Math.Sqrt(sum);
199 }
200 }
201 }
202 }
203 }
204 }

```

Mathematical formulation:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}}$$

Visual output: A 2D scatter plot showing data points positioned to preserve local structure and cluster relationships from the high-dimensional space.

10.3 UMAP

Listing 39: UMAP Implementation in C#

```

1 public class UMAP
2 {
3     public int NNeighbors { get; }
4     public double MinDist { get; }
5     public int NComponents { get; }
6
7     public UMAP(int nNeighbors = 15, double minDist = 0.1, int
8         nComponents = 2)
9     {
10         NNeighbors = nNeighbors;
11         MinDist = minDist;
12         NComponents = nComponents;
13     }
14
15     public double[][] FitTransform(double[][] X, int maxIterations
16         = 200)
17     {
18         int n = X.Length;
19
20         // Compute k-nearest neighbors
21         var knnGraph = ComputeKNN(X);
22
23         // Compute fuzzy simplicial set
24         var fuzzySet = ComputeFuzzySimplicialSet(X, knnGraph);
25
26         // Initialize embedding
27         Random rng = new Random();
28         double[,] embedding = new double[n, NComponents];
29         for (int i = 0; i < n; i++)
30         {
31             for (int j = 0; j < NComponents; j++)
32                 embedding[i, j] = (rng.NextDouble() - 0.5) * 10;
33         }
34
35         // Stochastic gradient descent
36         for (int iter = 0; iter < maxIterations; iter++)
37         {
38             double learningRate = 1.0 - (double)iter /
39                 maxIterations;
40             for (int i = 0; i < n; i++)
41             {
42                 for (int j = 0; j < n; j++)

```

```

41 {
42     if (i != j && fuzzySet[i, j] > 0)
43     {
44         double[] grad = ComputeGradient(embedding,
45             i, j, fuzzySet[i, j], learningRate);
46         for (int d = 0; d < NComponents; d++)
47             embedding[i, d] += grad[d];
48     }
49 }
50 }
51
52 // Convert C# array format
53 double[][] result = new double[n][];
54 for (int i = 0; i < n; i++)
55 {
56     result[i] = new double[NComponents];
57     for (int j = 0; j < NComponents; j++)
58         result[i][j] = embedding[i, j];
59 }
60 return result;
61 }
62
63 private List<int>[] ComputeKNN(double[][] X)
64 {
65     int n = X.Length;
66     var knnGraph = new List<int>[n];
67
68     for (int i = 0; i < n; i++)
69     {
70         var distances = new List<(int index, double distance)
71             >();
72         for (int j = 0; j < n; j++)
73         {
74             if (i != j)
75             {
76                 double dist = EuclideanDistance(X[i], X[j]);
77                 distances.Add((j, dist));
78             }
79         }
80         knnGraph[i] = distances
81             .OrderBy(x => x.distance)
82             .Take(NNeighbors)
83             .Select(x => x.index)
84             .ToList();
85     }
86 }

```

```

86     }
87     return knnGraph;
88 }
89
90 private double[,] ComputeFuzzySimplicialSet(double[][] X, List<
91     int>[] knnGraph)
92 {
93     int n = X.Length;
94     double[,] fuzzySet = new double[n, n];
95     for (int i = 0; i < n; i++)
96     {
97         double maxDist = 0;
98         foreach (int j in knnGraph[i])
99         {
100             double dist = EuclideanDistance(X[i], X[j]);
101             if (dist > maxDist) maxDist = dist;
102         }
103
104         foreach (int j in knnGraph[i])
105         {
106             double dist = EuclideanDistance(X[i], X[j]);
107             fuzzySet[i, j] = Math.Exp(-dist / maxDist);
108         }
109     }
110 }
111
112 return fuzzySet;
113 }
114
115 private double[] ComputeGradient(double[,] embedding, int i,
116     int j, double weight, double learningRate)
117 {
118     double[] grad = new double[NComponents];
119     double dist = 0;
120     for (int d = 0; d < NComponents; d++)
121     {
122         double diff = embedding[i, d] - embedding[j, d];
123         dist += diff * diff;
124     }
125
126     dist = Math.Sqrt(dist);
127     if (dist > 0)
128     {
129         double factor = -2 * weight * learningRate / (1 + dist
130             * dist);
131         for (int d = 0; d < NComponents; d++)
132         {
133             double diff = embedding[i, d] - embedding[j, d];
134             grad[d] = factor * diff / dist;
135         }
136     }
137     return grad;
138 }
139
140 private double EuclideanDistance(double[] a, double[] b)
141 {

```

```

143     double sum = 0;
144     for (int i = 0; i < a.Length; i++)
145         sum += Math.Pow(a[i] - b[i], 2);
146     return Math.Sqrt(sum);
147 }
148

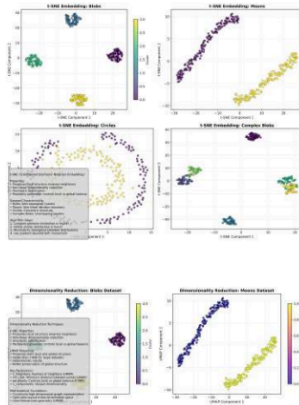
```

Mathematical formulation:

$$d_{low}(y_i, y_j) = \frac{1}{1 + a \cdot \text{dist}(y_i, y_j)^{2b}}$$

$$\frac{\partial C}{\partial y_i} = \sum_{j \neq i} \frac{\partial C}{\partial d_{low}(y_i, y_j)} \cdot \frac{\partial d_{low}(y_i, y_j)}{\partial y_i}$$

Visual output: A 2D embedding showing data points with preserved local and global structure, with clusters and manifolds clearly visible.



11 Time Series and Signal Processing

Time series analysis provides mathematical tools for understanding temporal patterns and sequential data [4]. These techniques are essential for forecasting, signal processing, and analyzing data that evolves over time [13].

11.1 Discrete Fourier Transform and FFT

Listing 40: Fast Fourier Transform in C#

```

1 public class FFT
2 {
3     public static Complex[] Forward(double[] signal)
4     {
5         int n = signal.Length;
6         Complex[] complexSignal = new Complex[n];
7         for (int i = 0; i < n; i++)
8             complexSignal[i] = new Complex(signal[i], 0);
9         return FFTRecursive(complexSignal);
10    }
11
12    private static Complex[] FFTRecursive(Complex[] signal)
13    {
14        int n = signal.Length;
15        if (n == 1) return signal;
16
17        if (n % 2 != 0)
18            throw new ArgumentException("Signal length must be a
19                power of 2");
20
21        Complex[] even = new Complex[n / 2];
22        Complex[] odd = new Complex[n / 2];
23        for (int i = 0; i < n / 2; i++)
24        {
25            even[i] = signal[2 * i];
26            odd[i] = signal[2 * i + 1];
27        }
28        Complex[] evenFFT = FFTRecursive(even);
29        Complex[] oddFFT = FFTRecursive(odd);
30
31        Complex[] result = new Complex[n];
32        for (int k = 0; k < n / 2; k++)
33        {
34            double angle = -2 * Math.PI * k / n;
35            Complex w = new Complex(Math.Cos(angle), Math.Sin(angle)

```

```

    });
40
    result[k] = evenFFT[k] + w * oddFFT[k];
41
    result[k + n / 2] = evenFFT[k] - w * oddFFT[k];
42
    }
43
    return result;
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
}

public static double[] Inverse(Complex[] spectrum)
{
    int n = spectrum.Length;
    Complex[] conjugated = new Complex[n];
    for (int i = 0; i < n; i++)
        conjugated[i] = Complex.Conjugate(spectrum[i]);
    Complex[] inverseFFT = FFTRecursive(conjugated);
    double[] result = new double[n];
    for (int i = 0; i < n; i++)
        result[i] = inverseFFT[i].Real / n;
    return result;
}

public static double[] PowerSpectrum(Complex[] spectrum)
{
    int n = spectrum.Length;
    double[] power = new double[n];
    for (int i = 0; i < n; i++)
        power[i] = spectrum[i].Magnitude * spectrum[i].Magnitude;
    return power;
}

public static double[] FrequencyBins(int n, double samplingRate)
{
    double[] frequencies = new double[n];
    for (int i = 0; i < n; i++)
        frequencies[i] = i * samplingRate / n;
    return frequencies;
}

public struct Complex
{
    public double Real { get; set; }
    public double Imaginary { get; set; }

    public Complex(double real, double imaginary)
    {
        Real = real;
        Imaginary = imaginary;
    }

    public double Magnitude => Math.Sqrt(Real * Real + Imaginary * Imaginary);
    public static Complex operator +(Complex a, Complex b)
    {
        return new Complex(a.Real + b.Real, a.Imaginary + b.Imaginary);
    }
}

```

```

101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
}

public static Complex operator -(Complex a, Complex b)
{
    return new Complex(a.Real - b.Real, a.Imaginary - b.Imaginary);
}

public static Complex operator *(Complex a, Complex b)
{
    return new Complex(a.Real * b.Real - a.Imaginary * b.Imaginary,
        a.Real * b.Imaginary + a.Imaginary * b.Real);
}

public static Complex Conjugate(Complex c)
{
    return new Complex(c.Real, -c.Imaginary);
}

```

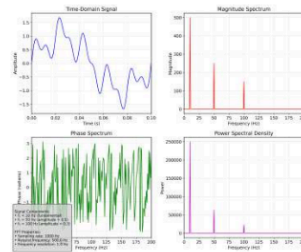
Mathematical formulation:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}$$

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i2\pi kn/N}$$

$$P_k = |X_k|^2$$

Visual output: A time-domain signal and its corresponding frequency spectrum showing magnitude and phase components.



11.2 Basic Convolution (1D and 2D)

Listing 41: Convolution Implementation in C#

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
}

public class Convolution
{
    public static double[] Convolve1D(double[] signal, double[] kernel)
    {
        int signalLength = signal.Length;
        int kernelLength = kernel.Length;
        int outputLength = signalLength + kernelLength - 1;
        double[] output = new double[outputLength];
        for (int i = 0; i < outputLength; i++)
        {
            output[i] = 0;
            for (int j = 0; j < kernelLength; j++)
            {
                int signalIndex = i - j;
                if (signalIndex >= 0 && signalIndex < signalLength)
                    output[i] += signal[signalIndex] * kernel[j];
            }
        }
    }
}

```

```

22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
}

return output;
}

public static double[,] Convolve2D(double[,] image, double[,] kernel)
{
    int imageHeight = image.GetLength(0);
    int imageWidth = image.GetLength(1);
    int kernelHeight = kernel.GetLength(0);
    int kernelWidth = kernel.GetLength(1);
    int outputHeight = imageHeight - kernelHeight + 1;
    int outputWidth = imageWidth - kernelWidth + 1;
    double[,] output = new double[outputHeight, outputWidth];
    for (int i = 0; i < outputHeight; i++)
    {
        for (int j = 0; j < outputWidth; j++)
        {
            output[i, j] = 0;
            for (int ki = 0; ki < kernelHeight; ki++)
            {
                for (int kj = 0; kj < kernelWidth; kj++)
                {
                    output[i, j] += image[i + ki, j + kj] *

```

```

        kernel[kl, kj];
    }
    }
    }
    }
    return output;
}
2 public static double[] GaussianKernel1D(double sigma, int size)
{
    double[] kernel = new double[size];
    int center = size / 2;
    double sum = 0;
    for (int i = 0; i < size; i++)
    {
        double k = i - center;
        kernel[i] = Math.Exp(-(k * k) / (2 * sigma * sigma));
        sum += kernel[i];
    }
    // Normalize
    for (int i = 0; i < size; i++)
        kernel[i] /= sum;
    return kernel;
}
2 public static double[,] GaussianKernel2D(double sigma, int size)
{
    double[,] kernel = new double[size, size];
    int center = size / 2;
    double sum = 0;
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            double k = i - center;
            double y = j - center;
            kernel[i, j] = Math.Exp(-(k * k + y * y) / (2 * sigma * sigma));
            sum += kernel[i, j];
        }
    }
    // Normalize
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
            kernel[i, j] /= sum;
    }
    return kernel;
}
}

```

```

101 public static double[] SobelXKernel()
102 {
103     return new double[] { -1, 0, 1, -2, 0, 2, -1, 0, 1 };
104 }
105
106 public static double[] SobelYKernel()
107 {
108     return new double[] { -1, -2, -1, 0, 0, 0, 1, 2, 1 };
109 }
110
111 }

```

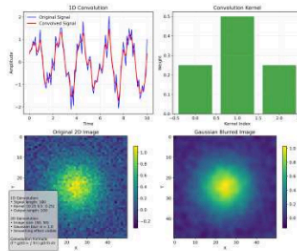
Mathematical formulation:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m]$$

$$(f * g)[i, j] = \sum_m \sum_n f[m, n]g[i - m, j - n]$$

Visual output: A 1D signal and kernel with their convolution result, and a 2D image with a Gaussian blur kernel showing the smoothing effect.



11.3 Autocorrelation and Cross-Correlation

Listing 42: Correlation Analysis in C#

```

1 public class Correlation
2 {
3     2 public static double[] Autocorrelation(double[] signal)
4     {
5         int n = signal.Length;
6         double[] autocorr = new double[n];
7         for (int lag = 0; lag < n; lag++)
8         {
9             autocorr[lag] = 0;
10            for (int i = 0; i < n - lag; i++)
11            {
12                autocorr[lag] += signal[i] * signal[i + lag];
13            }
14            autocorr[lag] /= (n - lag);
15        }
16        return autocorr;
17    }
18
19    public static double[] CrossCorrelation(double[] signal1,
20    double[] signal2)
21    {
22        int n1 = signal1.Length;
23        int n2 = signal2.Length;
24        int maxLag = Math.Max(n1, n2);
25        double[] crosscorr = new double[2 * maxLag - 1];
26        for (int lag = -maxLag + 1; lag < maxLag; lag++)
27        {
28            crosscorr[lag + maxLag - 1] = 0;
29        }
30    }

```

```

31    int count = 0;
32
33    for (int i = 0; i < n1; i++)
34    {
35        int j = i + lag;
36        if (j >= 0 && j < n2)
37        {
38            crosscorr[lag + maxLag - 1] += signal1[i] *
39            signal2[j];
40            count++;
41        }
42    }
43    if (count > 0)
44        crosscorr[lag + maxLag - 1] /= count;
45    }
46
47    return crosscorr;
48
49    2 public static double[] NormalizedAutocorrelation(double[]
50    signal)
51    {
52        double[] autocorr = Autocorrelation(signal);
53        double variance = autocorr[0];
54
55        for (int i = 0; i < autocorr.Length; i++)
56            autocorr[i] /= variance;
57
58        return autocorr;
59    }
60
61    public static int FindPeak(double[] signal, int minLag = 0)
62    {
63        int peakIndex = minLag;

```

```

64 double peakValue = signal[minLag];
65
66 for (int i = minLag + 1; i < signal.Length; i++)
67 {
68     if (signal[i] > peakValue)
69     {
70         peakValue = signal[i];
71         peakIndex = i;
72     }
73 }
74
75 return peakIndex;
76 }
77 }

```

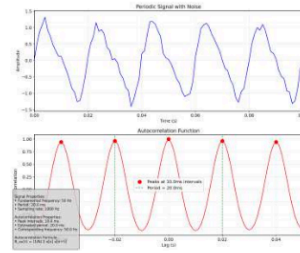
Mathematical formulation:

$$R_{xx}(\tau) = \frac{1}{N} \sum_{n=0}^{N-1} x[n]x[n+\tau]$$

$$R_{xy}(\tau) = \frac{1}{N} \sum_{n=0}^{N-1} x[n]y[n+\tau]$$

$$\rho_{xx}(\tau) = \frac{R_{xx}(\tau)}{R_{xx}(0)}$$

Visual output: A plot showing the autocorrelation function with peaks at periodic intervals, indicating the fundamental frequency of the signal.



11.4 Digital Filters

Listing 43: Digital Filters in C#

```

1 public class DigitalFilters
2 {
3     public static double[] MovingAverage(double[] signal, int
4         windowSize)
5     {
6         int n = signal.Length;
7         double[] filtered = new double[n];
8         for (int i = 0; i < n; i++)
9         {
10             double sum = 0;
11             int count = 0;
12             for (int j = Math.Max(0, i - windowSize / 2);
13                  j < Math.Min(n, i + windowSize / 2 + 1);
14                  j++)
15             {
16                 sum += signal[j];
17                 count++;
18             }
19             filtered[i] = sum / count;
20         }
21         return filtered;
22     }
23
24     public static double[] ExponentialSmoothing(double[] signal,
25         double alpha)
26     {
27         int n = signal.Length;
28         double[] filtered = new double[n];
29         filtered[0] = signal[0];
30         for (int i = 1; i < n; i++)
31         {
32             filtered[i] = alpha * signal[i] + (1 - alpha) *
33                 filtered[i - 1];
34         }
35         return filtered;
36     }
37
38     public static double[] ButterworthLowPass(double[] signal,
39         double cutoffFreq, double samplingRate)
40     {
41         int n = signal.Length;
42         Complex[] spectrum = FFT.Forward(signal);
43         for (int i = 0; i < n; i++)
44         {
45             double freq = i * samplingRate / n;
46             double filterResponse = 1.0 /
47                 Math.Sqrt(1 + Math.Pow(freq / cutoffFreq, 4));
48             spectrum[i] = new Complex(
49                 spectrum[i].Real *
50                 filterResponse,
51                 spectrum[i].Imaginary *
52                 filterResponse);
53         }
54         return FFT.Inverse(spectrum);
55     }
56
57     public static double[] MedianFilter(double[] signal, int
58         windowSize)
59     {
60         int n = signal.Length;
61         double[] filtered = new double[n];
62         for (int i = 0; i < n; i++)
63         {
64             var window = new List<double>();
65             for (int j = Math.Max(0, i - windowSize / 2);
66                  j < Math.Min(n, i + windowSize / 2 + 1);
67                  j++)
68             {
69                 window.Add(signal[j]);
70             }
71             window.Sort();
72             filtered[i] = window[window.Count / 2];
73         }
74         return filtered;
75     }
76 }

```

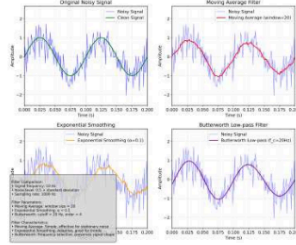
Mathematical formulation:

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n-k] \quad (\text{Moving Average})$$

$$y[n] = \alpha x[n] + (1 - \alpha)y[n-1] \quad (\text{Exponential Smoothing})$$

$$H(f) = \frac{1}{\sqrt{1 + (f/f_c)^4}} \quad (\text{Butterworth})$$

Visual output: A noisy signal with different filtered versions showing smoothing effects, frequency response curves, and time-domain comparisons.



12 Information Theory

Information theory provides mathematical frameworks for quantifying information, uncertainty, and data compression [7]. These concepts are essential for understanding entropy, mutual information, and efficient data representation in machine learning [13].

12.1 Entropy

Listing 44: Entropy Calculations in C#

```
1 public class InformationTheory
2 {
3     public static double ShannonEntropy(double[] probabilities)
4     {
5         double entropy = 0;
6         for (int i = 0; i < probabilities.Length; i++)
7         {
8             if (probabilities[i] > 0)
9                 entropy += probabilities[i] * Math.Log(
10                     probabilities[i], 2);
11         }
12         return entropy;
13     }
14     public static double CrossEntropy(double[] p, double[] q)
15     {
16         double crossEntropy = 0;
17         for (int i = 0; i < p.Length; i++)
18         {
19             if (p[i] > 0 && q[i] > 0)
20                 crossEntropy += p[i] * Math.Log(q[i], 2);
21         }
22         return crossEntropy;
23     }
24     public static double KLDivergence(double[] p, double[] q)
25     {
26         double kl = 0;
27         for (int i = 0; i < p.Length; i++)
28         {
29             if (p[i] > 0 && q[i] > 0)
30                 kl += p[i] * Math.Log(p[i] / q[i], 2);
31         }
32         return kl;
33     }
34     public static double MutualInformation(double[,] jointProb,
35         double[] pX, double[] pY)
36     {
37         double mi = 0;
38         for (int i = 0; i < jointProb.GetLength(0); i++)
39         {
40             for (int j = 0; j < jointProb.GetLength(1); j++)
41             {
42                 if (jointProb[i, j] > 0 && pX[i] > 0 && pY[j] > 0)
43                 {
44                     mi += jointProb[i, j] * Math.Log(jointProb[i, j] / (pX[i] * pY[j]), 2);
45                 }
46             }
47         }
48         return mi;
49     }
50 }
51 public static double ConditionalEntropy(double[,] jointProb,
52     double[] pX)
```

```
53 {
54     double conditionalEntropy = 0;
55     for (int i = 0; i < jointProb.GetLength(0); i++)
56     {
57         if (pX[i] > 0)
58         {
59             double conditionalEntropyForX = 0;
60             for (int j = 0; j < jointProb.GetLength(1); j++)
61             {
62                 if (jointProb[i, j] > 0)
63                 {
64                     double conditionalProb = jointProb[i, j] /
65                         pX[i];
66                     conditionalEntropyForX += conditionalProb *
67                         Math.Log(conditionalProb, 2);
68                 }
69             }
70             conditionalEntropy += pX[i] *
71                 conditionalEntropyForX;
72         }
73     }
74 }
```

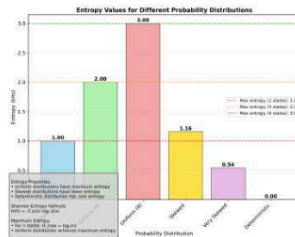
Mathematical formulation:

$$H(X) = - \sum_i p(x_i) \log_2 p(x_i)$$

$$H(X|Y) = - \sum_{i,j} p(x_i, y_j) \log_2 p(x_i|y_j)$$

$$I(X; Y) = \sum_{i,j} p(x_i, y_j) \log_2 \frac{p(x_i, y_j)}{p(x_i)p(y_j)}$$

Visual output: A bar chart showing entropy values for different probability distributions, with maximum entropy for uniform distributions.



12.2 Information Gain

Listing 45: Information Gain in C#

```

1 public class DecisionTreeMetrics
2 {
3     public static double InformationGain(double[] parentEntropy,
4         double[][] childEntropies, double[] childWeights)
5     {
6         double parentEntropyValue = ShannonEntropy(parentEntropy);
7         double weightedChildEntropy = 0;
8
9         for (int i = 0; i < childEntropies.Length; i++)
10         {
11             weightedChildEntropy += childWeights[i] *
12                 ShannonEntropy(childEntropies[i]);
13         }
14
15         return parentEntropyValue - weightedChildEntropy;
16     }
17
18     public static double GiniImpurity(double[] probabilities)
19     {
20         double gini = 1;
21         for (int i = 0; i < probabilities.Length; i++)
22         {
23             gini -= probabilities[i] * probabilities[i];
24         }
25         return gini;
26     }
27
28     public static double GiniGain(double[] parentProbabilities,
29         double[][] childProbabilities, double[] childWeights)
30     {
31         double parentGini = GiniImpurity(parentProbabilities);
32         double weightedChildGini = 0;
33
34         for (int i = 0; i < childProbabilities.Length; i++)
35         {
36             weightedChildGini += childWeights[i] * GiniImpurity(
37                 childProbabilities[i]);
38         }
39
40         return parentGini - weightedChildGini;
41     }
42
43     public static double[] CalculateClassProbabilities(int[] labels)
44     {
45         var classCounts = new Dictionary<int, int>();
46         foreach (int label in labels)
47         {
48             if (classCounts.ContainsKey(label))
49                 classCounts[label]++;
50             else
51                 classCounts[label] = 1;
52         }
53
54         double[] probabilities = new double[classCounts.Count];
55         int totalSamples = labels.Length;
56         int i = 0;
57
58         foreach (var kvp in classCounts)
59         {
60             probabilities[i] = (double)kvp.Value / totalSamples;
61             i++;
62         }
63
64         return probabilities;
65     }
66
67     public static double[] CalculateConditionalProbabilities(int[]
68         labels, int[] featureValues, int featureValue)
69     {
70         var classCounts = new Dictionary<int, int>();
71         foreach (int label in labels)
72         {
73             if (classCounts.ContainsKey(label))
74                 classCounts[label]++;
75             else
76                 classCounts[label] = 1;
77         }
78
79         double[] probabilities = new double[classCounts.Count];
80         int totalSamples = labels.Length;
81         int i = 0;
82
83         foreach (var kvp in classCounts)
84         {
85             probabilities[i] = (double)kvp.Value / totalSamples;
86             i++;
87         }
88
89         return probabilities;
90     }
91 }

```

```

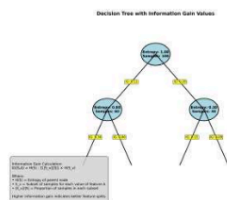
65     {
66         var classCounts = new Dictionary<int, int>();
67         int totalCount = 0;
68
69         for (int i = 0; i < labels.Length; i++)
70         {
71             if (featureValues[i] == featureValue)
72             {
73                 if (classCounts.ContainsKey(labels[i]))
74                     classCounts[labels[i]]++;
75                 else
76                     classCounts[labels[i]] = 1;
77             }
78             totalCount++;
79         }
80         double[] probabilities = new double[classCounts.Count];
81         int j = 0;
82
83         foreach (var kvp in classCounts)
84         {
85             probabilities[j] = (double)kvp.Value / totalCount;
86             j++;
87         }
88
89         return probabilities;
90     }
91 }

```

Mathematical formulation:

$$\begin{aligned}
IG(S, A) &= H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \\
Gini(S) &= 1 - \sum_{i=1}^c p_i^2 \\
IG_{Gini}(S, A) &= Gini(S) \\
&\quad - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} Gini(S_v)
\end{aligned}$$

Visual output: A decision tree showing information gain values at each split, with higher gains indicating better feature splits.



12.3 Channel Capacity

Listing 46: Channel Capacity in C#

```

1 public class ChannelCapacity
2 {
3     public static double BinarySymmetricChannelCapacity(double p)
4     {
5         if (p == 0 || p == 1) return 1;
6         return 1 + p * Math.Log(p, 2) + (1 - p) * Math.Log(1 - p,
7             2);
8     }
9     public static double GaussianChannelCapacity(double snr)
10    {
11        return 0.5 * Math.Log(1 + snr, 2);
12    }
13    public static double[,] ComputeTransitionMatrix(double[]
14        inputProbs, double errorProb)
15    {
16        int n = inputProbs.Length;
17        double[,] transitionMatrix = new double[n, n];
18        for (int i = 0; i < n; i++)
19        {
20            for (int j = 0; j < n; j++)
21            {
22                if (i == j)
23                    transitionMatrix[i, j] = 1 - errorProb;
24                else
25                    transitionMatrix[i, j] = errorProb / (n - 1);
26            }
27        }
28        return transitionMatrix;
29    }
30    public static double[] ComputeOutputProbabilities(double[]
31        inputProbs, double[,] transitionMatrix)
32    {
33        int n = inputProbs.Length;
34        double[] outputProbs = new double[n];
35        for (int j = 0; j < n; j++)
36        {
37            outputProbs[j] = 0;
38            for (int i = 0; i < n; i++)
39            {
40                outputProbs[j] += inputProbs[i] * transitionMatrix[
41                    i, j];
42            }
43        }
44        return outputProbs;
45    }
46    public static double ComputeChannelCapacity(double[,]
47        transitionMatrix, int maxIterations = 1000)
48    {
49        int n = transitionMatrix.GetLength(0);
50        double[] inputProbs = new double[n];
51        for (int i = 0; i < n; i++)
52            inputProbs[i] = 1.0 / n; // Start with uniform
53            distribution
54        double capacity = 0;
55        for (int iter = 0; iter < maxIterations; iter++)
56        {
57            double[] outputProbs = ComputeOutputProbabilities(
58                inputProbs, transitionMatrix);
59            // Update input probabilities using Blahut-Arimoto
60            // algorithm
61            double[] newInputProbs = new double[n];

```

```

65        double sum = 0;
66        for (int i = 0; i < n; i++)
67        {
68            double product = 1;
69            for (int j = 0; j < n; j++)
70            {
71                if (transitionMatrix[i, j] > 0 && outputProbs[j]
72                    > 0)
73                {
74                    product *= Math.Pow(transitionMatrix[i, j] /
75                        outputProbs[j], transitionMatrix[i, j]);
76                }
77                newInputProbs[i] = inputProbs[i] * product;
78                sum += newInputProbs[i];
79            }
80            inputProbs = newInputProbs;
81            // Compute mutual information
82            capacity = 0;
83            for (int i = 0; i < n; i++)
84            {
85                for (int j = 0; j < n; j++)
86                {
87                    if (inputProbs[i] > 0 && transitionMatrix[i, j]
88                        > 0 && outputProbs[j] > 0)
89                    {
90                        capacity += inputProbs[i] *
91                            transitionMatrix[i, j] *
92                            Math.Log(transitionMatrix[i, j] /
93                                outputProbs[j], 2);
94                    }
95                }
96            }
97        }
98        return capacity;
99    }
100 }
101 }
102 }
103 }

```

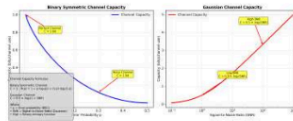
Mathematical formulation:

$$C = \max_{p(x)} I(X; Y)$$

$$C_{BSC} = 1 - H(p)$$

$$C_{Gaussian} = \frac{1}{2} \log_2(1 + \text{SNR})$$

Visual output: A plot showing channel capacity as a function of error probability or SNR, with maximum capacity at optimal input distributions.



12.4 Data Compression

Listing 47: Data Compression in C#

```

1 public class DataCompression
2 {
3     public static Dictionary<char, int> HuffmanEncoding(string
4         text)
5     {
6         // Count frequencies
7         var frequencies = new Dictionary<char, int>();
8         foreach (char c in text)

```

```

8     {
9         if (frequencies.ContainsKey(c))
10            frequencies[c]++;
11        else
12            frequencies[c] = 1;
13    }
14    // Build Huffman tree
15    var nodes = new List<HuffmanNode>();
16    foreach (var kvp in frequencies)
17    {
18

```

```

19         nodes.Add(new HuffmanNode { Symbol = kvp.Key, Frequency
20             = kvp.Value });
21     }
22     while (nodes.Count > 1)
23     {
24         nodes.Sort((a, b) => a.Frequency.CompareTo(b.Frequency));
25     }
26     var left = nodes[0];
27     var right = nodes[1];
28     nodes.RemoveRange(0, 2);
29     var parent = new HuffmanNode
30     {
31         Frequency = left.Frequency + right.Frequency,
32         Left = left,
33         Right = right
34     };
35     nodes.Add(parent);
36 }
37 // Generate codes
38 var codes = new Dictionary<char, string>();
39 GenerateCodes(nodes[0], "", codes);
40 return codes;
41 }
42 private static void GenerateCodes(HuffmanNode node, string code
43     , Dictionary<char, string> codes)
44 {
45     if (node.Symbol.HasValue)
46     {
47         codes[node.Symbol.Value] = code;
48     }
49     else
50     {
51         GenerateCodes(node.Left, code + "0", codes);
52         GenerateCodes(node.Right, code + "1", codes);
53     }
54 }
55 public static string CompressText(string text, Dictionary<char,
56     string> codes)
57 {
58     string compressed = "";
59     foreach (char c in text)
60     {
61         compressed += codes[c];
62     }
63     return compressed;
64 }
65 public static double CompressionRatio(string original, string
66     compressed)
67 {
68     return (double)compressed.Length / (original.Length * 8);
69     // Assuming 8 bits per character
70 }
71 public static double EntropyRate(string text)
72 {
73     var frequencies = new Dictionary<char, int>();

```

```

74     foreach (char c in text)
75     {
76         if (frequencies.ContainsKey(c))
77             frequencies[c]++;
78         else
79             frequencies[c] = 1;
80     }
81     double[] probabilities = new double[frequencies.Count];
82     int total = text.Length;
83     int i = 0;
84     foreach (var kvp in frequencies)
85     {
86         probabilities[i] = (double)kvp.Value / total;
87         i++;
88     }
89     return ShannonEntropy(probabilities);
90 }
91 }
92 public class HuffmanNode
93 {
94     public char? Symbol { get; set; }
95     public int Frequency { get; set; }
96     public HuffmanNode Left { get; set; }
97     public HuffmanNode Right { get; set; }
98 }

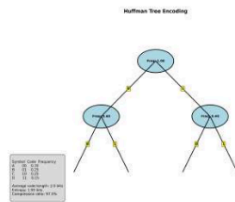
```

Mathematical formulation:

$$L(C) = \sum_i p(x_i) l(c(x_i))$$

$$H(X) \leq L(C) < H(X) + 1$$

Visual output: A Huffman tree showing the binary encoding of each symbol, with shorter codes for more frequent symbols.



13 Evaluation Metrics

Evaluation metrics provide quantitative measures for assessing machine learning model performance [19]. Understanding these metrics is crucial for model selection, comparison, and ensuring reliable model deployment [9].

13.1 Accuracy, Precision, Recall, F1-Score

Listing 48: Classification Metrics in C#

```

1 public class ClassificationMetrics
2 {
3     public static double Accuracy(int[] predictions, int[] targets)
4     {
5         int correct = 0;
6         for (int i = 0; i < predictions.Length; i++)
7         {
8             if (predictions[i] == targets[i])
9                 correct++;
10        }
11        return (double)correct / predictions.Length;
12    }

```

```

13     public static double Precision(int[] predictions, int[] targets
14         , int positiveClass = 1)
15     {
16         int truePositives = 0;
17         int falsePositives = 0;
18         for (int i = 0; i < predictions.Length; i++)
19         {
20             if (predictions[i] == positiveClass)
21             {
22                 if (targets[i] == positiveClass)
23                     truePositives++;
24                 else
25                     falsePositives++;
26             }

```

```

27     }
28 }
29
30 return truePositives + falsePositives > 0 ?
31 (double>truePositives / (truePositives + falsePositives
32 ) : 0;
33
34 public static double Recall(int[] predictions, int[] targets,
35 int positiveClass)
36 {
37     int truePositives = 0;
38     int falseNegatives = 0;
39     for (int i = 0; i < predictions.Length; i++)
40     {
41         if (targets[i] == positiveClass)
42         {
43             if (predictions[i] == positiveClass)
44                 truePositives++;
45             else
46                 falseNegatives++;
47         }
48     }
49     return truePositives + falseNegatives > 0 ?
50 (double>truePositives / (truePositives + falseNegatives
51 ) : 0;
52
53 public static double F1Score(int[] predictions, int[] targets,
54 int positiveClass = 1)
55 {
56     double precision = Precision(predictions, targets,
57 positiveClass);
58     double recall = Recall(predictions, targets, positiveClass);
59     return precision + recall > 0 ?
60 2 * precision * recall / (precision + recall) : 0;
61
62 public static double[] ConfusionMatrix(int[] predictions, int[]
63 targets, int numClasses)
64 {
65     double[,] matrix = new double[numClasses, numClasses];
66     for (int i = 0; i < predictions.Length; i++)
67     {
68         matrix[targets[i], predictions[i]]++;
69     }
70     double[] flattened = new double[numClasses * numClasses];
71     for (int i = 0; i < numClasses; i++)
72     {
73         for (int j = 0; j < numClasses; j++)
74         {
75             flattened[i * numClasses + j] = matrix[i, j];
76         }
77     }
78     return flattened;
79 }
80
81 public static double MacroF1(int[] predictions, int[] targets,
82 int numClasses)
83 {
84     double sumF1 = 0;
85     for (int c = 0; c < numClasses; c++)
86     {
87         sumF1 += F1Score(predictions, targets, c);
88     }
89     return sumF1 / numClasses;
90
91 public static double MicroF1(int[] predictions, int[] targets,
92 int numClasses)
93 {
94

```

```

96     int totalTP = 0;
97     int totalFP = 0;
98     int totalFN = 0;
99     for (int c = 0; c < numClasses; c++)
100     {
101         int tp = 0, fp = 0, fn = 0;
102         for (int i = 0; i < predictions.Length; i++)
103         {
104             if (predictions[i] == c && targets[i] == c)
105                 tp++;
106             else if (predictions[i] == c && targets[i] != c)
107                 fp++;
108             else if (predictions[i] != c && targets[i] == c)
109                 fn++;
110         }
111         totalTP += tp;
112         totalFP += fp;
113         totalFN += fn;
114     }
115     double microPrecision = totalTP > 0 ?
116 (double)totalTP / (totalTP + totalFP) : 0;
117     double microRecall = totalTP > 0 ?
118 (double)totalTP / (totalTP + totalFN) : 0;
119     return microPrecision + microRecall > 0 ?
120 2 * microPrecision * microRecall / (microPrecision +
121 microRecall) : 0;
122 }
123

```

Mathematical formulation:

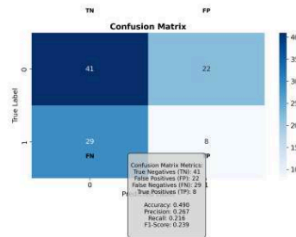
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Visual output: A confusion matrix heatmap showing true positives, false positives, true negatives, and false negatives with color intensity indicating counts.



13.2 ROC Curve and AUC

Listing 49: ROC and AUC in C#

```

1 public class ROCCurve
2 {
3     public static (double[], double[]) ComputeROC(double[] scores,
4 int[] targets, int positiveClass = 1)
5 {
6     var sortedIndices = Enumerable.Range(0, scores.Length)
7 .OrderByDescending(i => scores[i])
8 .ToArray();
9     int totalPositive = targets.Count(t => t == positiveClass);

```

```

10     int totalNegative = targets.Length - totalPositive;
11
12     var fpr = new List<double>();
13     var tpr = new List<double>();
14
15     int tp = 0, fp = 0;
16
17     for (int i = 0; i < sortedIndices.Length; i++)
18     {
19         int idx = sortedIndices[i];
20         if (targets[idx] == positiveClass)
21             tp++;
22         else

```

```

23         fpr++;
24         tpr.Add((double)tp / totalPositive);
25         fpr.Add((double)fp / totalNegative);
26     }
27     return (fpr.ToArray(), tpr.ToArray());
28 }
29
30 public static double ComputeAUC(double[] fpr, double[] tpr)
31 {
32     double auc = 0;
33     for (int i = 1; i < fpr.Length; i++)
34     {
35         auc += (fpr[i] - fpr[i - 1]) * (tpr[i] + tpr[i - 1]) / 2;
36     }
37     return auc;
38 }
39
40 public static double ComputeAUCFromScores(double[] scores, int
41 [] targets, int positiveClass = 1)
42 {
43     var (fpr, tpr) = ComputeROC(scores, targets, positiveClass);
44     return ComputeAUC(fpr, tpr);
45 }
46
47 public static double FindOptimalThreshold(double[] scores, int
48 [] targets, int positiveClass = 1)
49 {
50     var (fpr, tpr) = ComputeROC(scores, targets, positiveClass);
51     var sortedScores = scores.OrderByDescending(s => s).ToArray();
52
53     double bestThreshold = sortedScores[0];
54     double bestF1 = 0;
55     for (int i = 0; i < sortedScores.Length; i++)
56     {
57         double threshold = sortedScores[i];
58         int[] predictions = scores.Select(s => s >= threshold ?
59             positiveClass : 1 - positiveClass).ToArray();
60         double f1 = F1Score(predictions, targets, positiveClass);
61
62         if (f1 > bestF1)
63         {
64             bestF1 = f1;
65             bestThreshold = threshold;
66         }
67     }
68     return bestThreshold;
69 }
70
71 public static double PrecisionRecallAUC(double[] scores, int[]
72 targets, int positiveClass = 1)
73 {
74     var sortedIndices = Enumerable.Range(0, scores.Length)
75         .OrderByDescending(i => scores[i])
76         .ToArray();
77
78     int totalPositive = targets.Count(t => t == positiveClass);
79     var precision = new List<double>();
80     var recall = new List<double>();
81
82

```

```

83     int tp = 0, fp = 0;
84
85     for (int i = 0; i < sortedIndices.Length; i++)
86     {
87         int idx = sortedIndices[i];
88         if (targets[idx] == positiveClass)
89             tp++;
90         else
91             fp++;
92
93         recall.Add((double)tp / totalPositive);
94         precision.Add(tp + fp > 0 ? (double)tp / (tp + fp) : 1);
95     }
96
97     double auc = 0;
98     for (int i = 1; i < precision.Count; i++)
99     {
100         auc += (recall[i] - recall[i - 1]) * (precision[i] +
101             precision[i - 1]) / 2;
102     }
103     return auc;
104 }
105

```

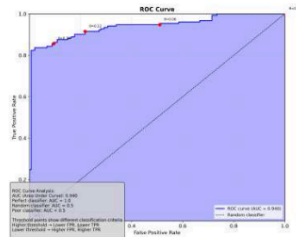
Mathematical formulation:

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

$$AUC = \int_0^1 TPR(FPR) dFPR$$

Visual output: An ROC curve showing true positive rate vs false positive rate, with AUC as the area under the curve and a diagonal line representing random classification.



13.3 Regression Metrics

Listing 50: Regression Metrics in C#

```

1 public class RegressionMetrics
2 {
3     public static double MeanSquaredError(double[] predictions,
4     double[] targets)
5     {
6         double mae = 0;
7         for (int i = 0; i < predictions.Length; i++)
8         {
9             mae += Math.Pow(predictions[i] - targets[i], 2);
10        }
11        return mae / predictions.Length;
12    }
13
14    public static double RootMeanSquaredError(double[] predictions,
15    double[] targets)
16    {
17        return Math.Sqrt(MeanSquaredError(predictions, targets));
18    }
19

```

```

18    public static double MeanAbsoluteError(double[] predictions,
19    double[] targets)
20    {
21        double mae = 0;
22        for (int i = 0; i < predictions.Length; i++)
23        {
24            mae += Math.Abs(predictions[i] - targets[i]);
25        }
26        return mae / predictions.Length;
27    }
28
29    public static double R2Score(double[] predictions, double[]
30    targets)
31    {
32        double meanTarget = targets.Average();
33        double ssRes = 0;
34        double ssTot = 0;
35        for (int i = 0; i < predictions.Length; i++)
36        {
37            ssRes += Math.Pow(predictions[i] - targets[i], 2);
38            ssTot += Math.Pow(targets[i] - meanTarget, 2);
39        }
40    }
41

```

```

38     }
39     return 1 - ssRes / ssTot;
40 }
41
42 public static double ExplainedVarianceScore(double[]
43     predictions, double[] targets)
44 {
45     double meanTarget = targets.Average();
46     double meanPred = predictions.Average();
47     double numerator = 0;
48     double denominator = 0;
49     for (int i = 0; i < predictions.Length; i++)
50     {
51         numerator += (predictions[i] - meanPred) * (targets[i]
52             - meanTarget);
53         denominator += Math.Pow(targets[i] - meanTarget, 2);
54     }
55     return denominator > 0 ? 1 - numerator * numerator /
56         denominator : 0;
57 }
58
59 public static double MeanAbsolutePercentageError(double[]
60     predictions, double[] targets)
61 {
62     double mape = 0;
63     int validCount = 0;
64     for (int i = 0; i < predictions.Length; i++)
65     {
66         if (targets[i] != 0)
67         {
68             mape += Math.Abs(predictions[i] - targets[i]) /
69                 targets[i];
70             validCount++;
71         }
72     }
73     return validCount > 0 ? mape / validCount * 100 : 0;
74 }
75
76 public static double HuberLoss(double[] predictions, double[]
77     targets, double delta = 1.0)
78 {
79     double loss = 0;
80     for (int i = 0; i < predictions.Length; i++)
81     {
82         double error = Math.Abs(predictions[i] - targets[i]);
83         if (error <= delta)
84         {
85             loss += 0.5 * error * error;
86         }
87         else
88         {

```

```

89             loss += delta * error - 0.5 * delta * delta;
90         }
91     }
92     return loss / predictions.Length;
93 }
94 }

```

Mathematical formulation:

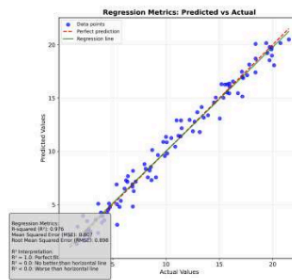
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Visual output: A scatter plot of predicted vs actual values with a perfect prediction line, showing residuals and R-squared value.



13.4 Clustering Metrics

Listing 51: Clustering Metrics in C#

```

1 public class ClusteringMetrics
2 {
3     public static double SilhouetteScore(double[][] data, int[]
4         labels)
5     {
6         int n = data.Length;
7         double totalSilhouette = 0;
8         for (int i = 0; i < n; i++)
9         {
10             double a = AverageDistance(data[i], data, labels,
11                 labels[i], true);
12             double b = AverageDistance(data[i], data, labels,
13                 labels[i], false);
14             if (a < b)
15                 totalSilhouette += 1 - a / b;
16             else if (a > b)
17                 totalSilhouette += b / a - 1;
18             // else silhouette = 0
19         }
20         return totalSilhouette / n;
21     }
22
23     private static double AverageDistance(double[] point, double
24         [][] data, int[] labels, int cluster, bool sameCluster)
25     {
26         double sum = 0;
27         int count = 0;

```

```

28     for (int i = 0; i < data.Length; i++)
29     {
30         if ((sameCluster && labels[i] == cluster && i != Array.
31             IndexOf(data, point)) ||
32             (!sameCluster && labels[i] != cluster))
33         {
34             sum += EuclideanDistance(point, data[i]);
35             count++;
36         }
37     }
38     return count > 0 ? sum / count : 0;
39 }
40
41 public static double CalinskiHarabaszIndex(double[][] data, int[]
42     labels)
43 {
44     int n = data.Length;
45     int k = labels.Distinct().Count();
46     if (k <= 1) return 0;
47     double[] overallMean = new double[data[0].Length];
48     for (int i = 0; i < n; i++)
49     {
50         for (int j = 0; j < data[i].Length; j++)
51             overallMean[j] += data[i][j];
52     }
53     for (int j = 0; j < overallMean.Length; j++)
54         overallMean[j] /= n;
55     double betweenClusterSS = 0;
56     double withinClusterSS = 0;
57     var clusters = labels.Distinct().ToArray();

```

```

61 foreach (int cluster in clusters)
62 {
63     var clusterPoints = data.Where((_, i) => labels[i] ==
64         cluster).ToArray();
65     int clusterSize = clusterPoints.Length;
66     if (clusterSize == 0) continue;
67     // Compute cluster mean
68     double[] clusterMean = new double[clusterPoints[0].Length];
69     for (int i = 0; i < clusterSize; i++)
70     {
71         for (int j = 0; j < clusterPoints[i].Length; j++)
72             clusterMean[j] += clusterPoints[i][j];
73     }
74     for (int j = 0; j < clusterMean.Length; j++)
75         clusterMean[j] /= clusterSize;
76     betweenClusterSS += clusterSize * Math.Pow(
77         EuclideanDistance(clusterMean, overallMean), 2);
78     // Compute cluster diameter
79     double diameter = 0;
80     for (int i = 0; i < clusterSize; i++)
81     {
82         withinClusterSS += Math.Pow(EuclideanDistance(
83             clusterPoints[i], clusterMean), 2);
84     }
85     return betweenClusterSS / withinClusterSS * (n - k) / (k -
86         1);
87 }
88 public static double DaviesBouldinIndex(double[][] data, int[]
89     labels)
90 {
91     var clusters = labels.Distinct().ToArray();
92     int k = clusters.Length;
93     if (k <= 1) return 0;
94     double[] clusterMeans = new double[k];
95     double[] clusterDiameters = new double[k];
96     for (int i = 0; i < k; i++)
97     {
98         var clusterPoints = data.Where((_, idx) => labels[idx]
99             == clusters[i]).ToArray();
100         int clusterSize = clusterPoints.Length;
101         if (clusterSize == 0) continue;
102         // Compute cluster mean
103         double[] mean = new double[clusterPoints[0].Length];
104         for (int j = 0; j < clusterSize; j++)
105         {
106             for (int d = 0; d < clusterPoints[j].Length; d++)
107                 mean[d] += clusterPoints[j][d];
108         }
109         for (int d = 0; d < mean.Length; d++)
110             mean[d] /= clusterSize;
111         clusterMeans[i] = EuclideanDistance(mean, new double[
112             mean.Length]); // Distance from origin
113         // Compute cluster diameter
114         double diameter = 0;
115         for (int j = 0; j < clusterSize; j++)
116         {
117             diameter += EuclideanDistance(clusterPoints[j],
118                 mean);
119         }
120     }
121 }

```

```

123 clusterDiameters[i] = diameter / clusterSize;
124 }
125 double totalIndex = 0;
126 for (int i = 0; i < k; i++)
127 {
128     double maxRatio = 0;
129     for (int j = 0; j < k; j++)
130     {
131         if (i != j)
132         {
133             double distance = Math.Abs(clusterMeans[i] -
134                 clusterMeans[j]);
135             double ratio = (clusterDiameters[i] +
136                 clusterDiameters[j]) / distance;
137             if (ratio > maxRatio)
138                 maxRatio = ratio;
139         }
140     }
141     totalIndex += maxRatio;
142 }
143 return totalIndex / k;
144 }
145 private static double EuclideanDistance(double[] a, double[] b)
146 {
147     double sum = 0;
148     for (int i = 0; i < a.Length; i++)
149         sum += Math.Pow(a[i] - b[i], 2);
150     return Math.Sqrt(sum);
151 }
152 }
153 }

```

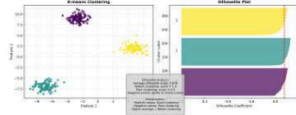
Mathematical formulation:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

$$CH = \frac{\text{tr}(B_k)}{\text{tr}(W_k)} \times \frac{n-k}{k-1}$$

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{\sigma_i + \sigma_j}{d(c_i, c_j)}$$

Visual output: A silhouette plot showing individual silhouette scores for each point, with positive values indicating good clustering and negative values indicating poor clustering.



14 Discrete Mathematics

Discrete mathematics provides the logical and combinatorial foundations for computer science and machine learning [17]. These concepts are essential for algorithm design, logic programming, and understanding computational structures [6].

14.1 Set Theory: Union, Intersection, Complement

Listing 52: Set Operations in C#

```

1 public class SetOperations
2 {
3     public static HashSet<T> Union<T>(HashSet<T> set1, HashSet<T>
4         set2)
5     {
6         var result = new HashSet<T>(set1);
7         result.UnionWith(set2);
8     }

```

```

7     return result;
8 }
9 public static HashSet<T> Intersection<T>(HashSet<T> set1,
10     HashSet<T> set2)
11 {
12     var result = new HashSet<T>(set1);
13     result.IntersectWith(set2);
14     return result;
15 }

```

```

16 public static HashSet<T> Complement<T>(HashSet<T> set, HashSet<
17 T> universalSet)
18 {
19     var result = new HashSet<T>(universalSet);
20     result.ExceptWith(set);
21     return result;
22 }
23
24 public static HashSet<T> Difference<T>(HashSet<T> set1, HashSet<
25 T> set2)
26 {
27     var result = new HashSet<T>(set1);
28     result.ExceptWith(set2);
29     return result;
30 }
31
32 public static bool IsSubset<T>(HashSet<T> subset, HashSet<T>
33 superset)
34 {
35     return subset.IsSubsetOf(superset);
36 }
37
38 public static bool IsProperSubset<T>(HashSet<T> subset, HashSet<
39 T> superset)
40 {
41     return subset.IsProperSubsetOf(superset);
42 }
43
44 public static HashSet<T> SymmetricDifference<T>(HashSet<T> set1
45 , HashSet<T> set2)
46 {
47     var result = new HashSet<T>(set1);
48     result.SymmetricExceptWith(set2);
49     return result;
50 }
51
52 public static int Cardinality<T>(HashSet<T> set)
53 {
54     return set.Count;
55 }
56
57 public static HashSet<HashSet<T>> PowerSet<T>(HashSet<T> set)
58 {
59     var elements = set.ToArray();
60     var powerSet = new HashSet<HashSet<T>>();
61     int n = elements.Length;
62     for (int i = 0; i < (1 << n); i++)
63     {
64         var subset = new HashSet<T>();
65         for (int j = 0; j < n; j++)
66         {

```

```

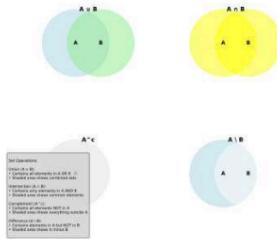
64         if ((i & (1 << j)) != 0)
65             subset.Add(elements[j]);
66         powerSet.Add(subset);
67     }
68     return powerSet;
69 }
70
71 }
72

```

Mathematical formulation:

$$\begin{aligned}
 A \cup B &= \{x : x \in A \text{ or } x \in B\} \\
 A \cap B &= \{x : x \in A \text{ and } x \in B\} \\
 A^c &= \{x : x \notin A\} \\
 A \setminus B &= \{x : x \in A \text{ and } x \notin B\}
 \end{aligned}$$

Visual output: Venn diagrams showing set operations with overlapping circles representing different sets and shaded regions indicating the result of each operation.



14.2 Logic: AND, OR, NOT, Implication

Listing 53: Logical Operations in C#

```

1 public class LogicOperations
2 {
3     public static bool AND(bool a, bool b)
4     {
5         return a & b;
6     }
7
8     public static bool OR(bool a, bool b)
9     {
10        return a || b;
11    }
12
13    public static bool NOT(bool a)
14    {
15        return !a;
16    }
17
18    public static bool Implication(bool p, bool q)
19    {
20        return p || q; // p -> q is equivalent to !p || q
21    }
22
23    public static bool Biconditional(bool p, bool q)
24    {
25        return p == q; // p <=> q is equivalent to (p -> q) && (q -> p)
26    }
27
28    public static bool XOR(bool a, bool b)
29    {
30        return a != b;
31    }
32
33    public static bool NAND(bool a, bool b)
34    {

```

```

35        return !(a & b);
36    }
37
38    public static bool NOR(bool a, bool b)
39    {
40        return !(a || b);
41    }
42
43    public static bool[] TruthTable(Func<bool, bool, bool> operation)
44    {
45        bool[] results = new bool[4];
46        results[0] = operation(false, false);
47        results[1] = operation(false, true);
48        results[2] = operation(true, false);
49        results[3] = operation(true, true);
50        return results;
51    }
52
53    public static bool IsTautology(Func<bool, bool, bool> operation)
54    {
55        var truthTable = TruthTable(operation);
56        return truthTable.All(result => result);
57    }
58
59    public static bool IsContradiction(Func<bool, bool, bool> operation)
60    {
61        var truthTable = TruthTable(operation);
62        return truthTable.All(result => !result);
63    }
64
65    public static bool IsSatisfiable(Func<bool, bool, bool> operation)
66    {
67        var truthTable = TruthTable(operation);
68        return truthTable.Any(result => result);
69    }

```

70

Mathematical formulation:

$p \wedge q = \text{true}$ if both p and q are true

$p \vee q = \text{true}$ if at least one of p or q is true

$\neg p = \text{true}$ if p is false

$p \rightarrow q = \neg p \vee q$

Visual output: A truth table showing all possible combinations of input values and their corresponding output for each logical operation.

Truth Table AND (&)			
p	q	Result	
T	T	T	
T	F	F	
F	T	F	
F	F	F	

Truth Table OR ()			
p	q	Result	
T	T	T	
T	F	T	
F	T	T	
F	F	F	

Truth Table XOR (^)			
p	q	Result	
T	T	F	
T	F	T	
F	T	T	
F	F	F	

Truth Table NOT (!)			
p	q	Result	
T		F	
F		T	

14.3 Boolean Algebra

Listing 54: Boolean Algebra in C#

```

1 public class BooleanAlgebra
2 {
3     public static bool ApplyBooleanFunction(bool[] inputs, bool[]
4         function)
5     {
6         int index = 0;
7         for (int i = 0; i < inputs.Length; i++)
8         {
9             if (inputs[i])
10                 index += (1 << i);
11         }
12         return function[index];
13     }
14     public static bool[] MinimizeBooleanFunction(bool[] function)
15     {
16         // Simple implementation of Quine-McCluskey algorithm
17         int numVars = (int) Math.Log(function.Length, 2);
18         var minterms = new List<int>();
19         for (int i = 0; i < function.Length; i++)
20         {
21             if (function[i])
22             {
23                 minterms.Add(i);
24             }
25         }
26         // Group by number of 1s
27         var groups = new Dictionary<int, List<string>>();
28         foreach (int minterm in minterms)
29         {
30             string binary = Convert.ToString(minterm, 2).PadLeft(
31                 numVars, '0');
32             int ones = binary.Count(c => c == '1');
33             if (!groups.ContainsKey(ones))
34                 groups[ones] = new List<string>();
35             groups[ones].Add(binary);
36         }
37         // Find prime implicants
38         var primeImplicants = new List<string>();
39         var used = new HashSet<string>();
40         for (int i = 0; i < numVars; i++)
41         {
42             var newGroups = new Dictionary<int, List<string>>();
43             var newUsed = new HashSet<string>();
44             foreach (var group in groups)
45             {
46                 if (group.Key + 1 < groups.Count)
47                 {
48                     var nextGroup = groups[group.Key + 1];
49                     foreach (string term1 in group.Value)
50                     {
51                         foreach (string term2 in nextGroup)
52                         {
53                             string combined = CombineTerms(term1,
54                                 term2);
55                             if (combined != null)
56                             {
57                                 int ones = combined.Count(c => c ==
58                                     '1');
59                                 if (!newGroups.ContainsKey(ones))
60                                     newGroups[ones] = new List<
61                                         string>();
62                                 newGroups[ones].Add(combined);
63                                 newUsed.Add(term1);
64                                 newUsed.Add(term2);
65                             }
66                         }
67                     }
68                 }
69             }
70             // Add unused terms as prime implicants
71             foreach (var group in groups)
72             {
73                 foreach (string term in group.Value)
74                 {
75                     if (!newUsed.Contains(term))
76                         primeImplicants.Add(term);
77                 }
78             }
79             groups = newGroups;
80             used = newUsed;
81         }
82         return primeImplicants.ToArray();
83     }
84     private static string CombineTerms(string term1, string term2)
85     {
86         int differences = 0;
87         int diffPos = -1;
88         for (int i = 0; i < term1.Length; i++)
89         {
90             if (term1[i] != term2[i])
91             {
92                 differences++;
93                 diffPos = i;
94             }
95         }
96         if (differences == 1)
97         {
98             return term1.Substring(0, diffPos) + "-" + term1.
99                 Substring(diffPos + 1);
100         }
101         return null;
102     }
103     public static bool DeMorganLaw1(bool a, bool b)
104     {
105         // !(a && b) == !a || !b
106         bool left = !(a && b);
107         bool right = !a || !b;
108         return left == right;
109     }
110     public static bool DeMorganLaw2(bool a, bool b)
111     {
112         // !(a || b) == !a && !b
113         bool left = !(a || b);
114         bool right = !a && !b;
115         return left == right;
116     }
117     public static bool DistributiveLaw1(bool a, bool b, bool c)
118     {
119         // a && (b || c) == (a && b) || (a && c)
120         bool left = a && (b || c);
121         bool right = (a && b) || (a && c);
122         return left == right;
123     }
124     public static bool DistributiveLaw2(bool a, bool b, bool c)
125     {
126         // a || (b && c) == (a || b) && (a || c)
127         bool left = a || (b && c);
128         bool right = (a || b) && (a || c);
129         return left == right;
130     }
131     public static bool AssociativeLaw1(bool a, bool b, bool c)
132     {
133         // (a && b) && c == a && (b && c)
134         bool left = (a && b) && c;
135         bool right = a && (b && c);
136         return left == right;
137     }
138     public static bool AssociativeLaw2(bool a, bool b, bool c)
139     {
140         // (a || b) || c == a || (b || c)
141         bool left = (a || b) || c;
142         bool right = a || (b || c);
143         return left == right;
144     }
145     public static bool CommutativeLaw1(bool a, bool b)
146     {
147         // a && b == b && a
148         bool left = a && b;
149         bool right = b && a;
150         return left == right;
151     }
152     public static bool CommutativeLaw2(bool a, bool b)
153     {
154         // a || b == b || a
155         bool left = a || b;
156         bool right = b || a;
157         return left == right;
158     }
159     public static bool AbsorptionLaw1(bool a, bool b)
160     {
161         // a && (a || b) == a
162         bool left = a && (a || b);
163         bool right = a;
164         return left == right;
165     }
166     public static bool AbsorptionLaw2(bool a, bool b)
167     {
168         // a || (a && b) == a
169         bool left = a || (a && b);
170         bool right = a;
171         return left == right;
172     }
173     public static bool IdentityLaw1(bool a)
174     {
175         // a && 1 == a
176         bool left = a && 1;
177         bool right = a;
178         return left == right;
179     }
180     public static bool IdentityLaw2(bool a)
181     {
182         // a || 0 == a
183         bool left = a || 0;
184         bool right = a;
185         return left == right;
186     }
187     public static bool NegationLaw1(bool a)
188     {
189         // a && !a == 0
190         bool left = a && !a;
191         bool right = 0;
192         return left == right;
193     }
194     public static bool NegationLaw2(bool a)
195     {
196         // a || !a == 1
197         bool left = a || !a;
198         bool right = 1;
199         return left == right;
200     }
201     public static bool DoubleNegationLaw(bool a)
202     {
203         // !!a == a
204         bool left = !!a;
205         bool right = a;
206         return left == right;
207     }
208     public static bool ContradictionLaw1(bool a)
209     {
210         // a && !a == 0
211         bool left = a && !a;
212         bool right = 0;
213         return left == right;
214     }
215     public static bool ContradictionLaw2(bool a)
216     {
217         // a || !a == 1
218         bool left = a || !a;
219         bool right = 1;
220         return left == right;
221     }
222     public static bool TautologyLaw1(bool a)
223     {
224         // a && a == a
225         bool left = a && a;
226         bool right = a;
227         return left == right;
228     }
229     public static bool TautologyLaw2(bool a)
230     {
231         // a || a == a
232         bool left = a || a;
233         bool right = a;
234         return left == right;
235     }
236     public static bool IdempotentLaw1(bool a)
237     {
238         // a && a == a
239         bool left = a && a;
240         bool right = a;
241         return left == right;
242     }
243     public static bool IdempotentLaw2(bool a)
244     {
245         // a || a == a
246         bool left = a || a;
247         bool right = a;
248         return left == right;
249     }
250     public static bool DeMorganLaw3(bool a, bool b, bool c)
251     {
252         // !(a && b && c) == !a || !b || !c
253         bool left = !(a && b && c);
254         bool right = !a || !b || !c;
255         return left == right;
256     }
257     public static bool DeMorganLaw4(bool a, bool b, bool c)
258     {
259         // !(a || b || c) == !a && !b && !c
260         bool left = !(a || b || c);
261         bool right = !a && !b && !c;
262         return left == right;
263     }
264     public static bool DeMorganLaw5(bool a, bool b, bool c)
265     {
266         // !(!a && !b && !c) == a || b || c
267         bool left = !(!a && !b && !c);
268         bool right = a || b || c;
269         return left == right;
270     }
271     public static bool DeMorganLaw6(bool a, bool b, bool c)
272     {
273         // !(!a || !b || !c) == a && b && c
274         bool left = !(!a || !b || !c);
275         bool right = a && b && c;
276         return left == right;
277     }
278     public static bool DeMorganLaw7(bool a, bool b, bool c)
279     {
280         // !(a && b) && c == (!a || !b) && c
281         bool left = !(a && b) && c;
282         bool right = (!a || !b) && c;
283         return left == right;
284     }
285     public static bool DeMorganLaw8(bool a, bool b, bool c)
286     {
287         // !(a || b) || c == (!a && !b) || c
288         bool left = !(a || b) || c;
289         bool right = (!a && !b) || c;
290         return left == right;
291     }
292     public static bool DeMorganLaw9(bool a, bool b, bool c)
293     {
294         // !(!a && !b) && c == (a || b) && c
295         bool left = !(!a && !b) && c;
296         bool right = (a || b) && c;
297         return left == right;
298     }
299     public static bool DeMorganLaw10(bool a, bool b, bool c)
300     {
301         // !(!a || !b) || c == (a && b) || c
302         bool left = !(!a || !b) || c;
303         bool right = (a && b) || c;
304         return left == right;
305     }
306     public static bool DeMorganLaw11(bool a, bool b, bool c)
307     {
308         // !(a && b && c) == !a || !b || !c
309         bool left = !(a && b && c);
310         bool right = !a || !b || !c;
311         return left == right;
312     }
313     public static bool DeMorganLaw12(bool a, bool b, bool c)
314     {
315         // !(a || b || c) == !a && !b && !c
316         bool left = !(a || b || c);
317         bool right = !a && !b && !c;
318         return left == right;
319     }
320     public static bool DeMorganLaw13(bool a, bool b, bool c)
321     {
322         // !(!a && !b && !c) == a || b || c
323         bool left = !(!a && !b && !c);
324         bool right = a || b || c;
325         return left == right;
326     }
327     public static bool DeMorganLaw14(bool a, bool b, bool c)
328     {
329         // !(!a || !b || !c) == a && b && c
330         bool left = !(!a || !b || !c);
331         bool right = a && b && c;
332         return left == right;
333     }
334     public static bool DeMorganLaw15(bool a, bool b, bool c)
335     {
336         // !(a && b) && c == (!a || !b) && c
337         bool left = !(a && b) && c;
338         bool right = (!a || !b) && c;
339         return left == right;
340     }
341     public static bool DeMorganLaw16(bool a, bool b, bool c)
342     {
343         // !(a || b) || c == (!a && !b) || c
344         bool left = !(a || b) || c;
345         bool right = (!a && !b) || c;
346         return left == right;
347     }
348     public static bool DeMorganLaw17(bool a, bool b, bool c)
349     {
350         // !(!a && !b) && c == (a || b) && c
351         bool left = !(!a && !b) && c;
352         bool right = (a || b) && c;
353         return left == right;
354     }
355     public static bool DeMorganLaw18(bool a, bool b, bool c)
356     {
357         // !(!a || !b) || c == (a && b) || c
358         bool left = !(!a || !b) || c;
359         bool right = (a && b) || c;
360         return left == right;
361     }
362     public static bool DeMorganLaw19(bool a, bool b, bool c)
363     {
364         // !(a && b && c) == !a || !b || !c
365         bool left = !(a && b && c);
366         bool right = !a || !b || !c;
367         return left == right;
368     }
369     public static bool DeMorganLaw20(bool a, bool b, bool c)
370     {
371         // !(a || b || c) == !a && !b && !c
372         bool left = !(a || b || c);
373         bool right = !a && !b && !c;
374         return left == right;
375     }
376     public static bool DeMorganLaw21(bool a, bool b, bool c)
377     {
378         // !(!a && !b && !c) == a || b || c
379         bool left = !(!a && !b && !c);
380         bool right = a || b || c;
381         return left == right;
382     }
383     public static bool DeMorganLaw22(bool a, bool b, bool c)
384     {
385         // !(!a || !b || !c) == a && b && c
386         bool left = !(!a || !b || !c);
387         bool right = a && b && c;
388         return left == right;
389     }
390     public static bool DeMorganLaw23(bool a, bool b, bool c)
391     {
392         // !(a && b) && c == (!a || !b) && c
393         bool left = !(a && b) && c;
394         bool right = (!a || !b) && c;
395         return left == right;
396     }
397     public static bool DeMorganLaw24(bool a, bool b, bool c)
398     {
399         // !(a || b) || c == (!a && !b) || c
400         bool left = !(a || b) || c;
401         bool right = (!a && !b) || c;
402         return left == right;
403     }
404     public static bool DeMorganLaw25(bool a, bool b, bool c)
405     {
406         // !(!a && !b) && c == (a || b) && c
407         bool left = !(!a && !b) && c;
408         bool right = (a || b) && c;
409         return left == right;
410     }
411     public static bool DeMorganLaw26(bool a, bool b, bool c)
412     {
413         // !(!a || !b) || c == (a && b) || c
414         bool left = !(!a || !b) || c;
415         bool right = (a && b) || c;
416         return left == right;
417     }
418     public static bool DeMorganLaw27(bool a, bool b, bool c)
419     {
420         // !(a && b && c) == !a || !b || !c
421         bool left = !(a && b && c);
422         bool right = !a || !b || !c;
423         return left == right;
424     }
425     public static bool DeMorganLaw28(bool a, bool b, bool c)
426     {
427         // !(a || b || c) == !a && !b && !c
428         bool left = !(a || b || c);
429         bool right = !a && !b && !c;
430         return left == right;
431     }
432     public static bool DeMorganLaw29(bool a, bool b, bool c)
433     {
434         // !(!a && !b && !c) == a || b || c
435         bool left = !(!a && !b && !c);
436         bool right = a || b || c;
437         return left == right;
438     }
439     public static bool DeMorganLaw30(bool a, bool b, bool c)
440     {
441         // !(!a || !b || !c) == a && b && c
442         bool left = !(!a || !b || !c);
443         bool right = a && b && c;
444         return left == right;
445     }
446     public static bool DeMorganLaw31(bool a, bool b, bool c)
447     {
448         // !(a && b) && c == (!a || !b) && c
449         bool left = !(a && b) && c;
450         bool right = (!a || !b) && c;
451         return left == right;
452     }
453     public static bool DeMorganLaw32(bool a, bool b, bool c)
454     {
455         // !(a || b) || c == (!a && !b) || c
456         bool left = !(a || b) || c;
457         bool right = (!a && !b) || c;
458         return left == right;
459     }
460     public static bool DeMorganLaw33(bool a, bool b, bool c)
461     {
462         // !(!a && !b) && c == (a || b) && c
463         bool left = !(!a && !b) && c;
464         bool right = (a || b) && c;
465         return left == right;
466     }
467     public static bool DeMorganLaw34(bool a, bool b, bool c)
468     {
469         // !(!a || !b) || c == (a && b) || c
470         bool left = !(!a || !b) || c;
471         bool right = (a && b) || c;
472         return left == right;
473     }
474     public static bool DeMorganLaw35(bool a, bool b, bool c)
475     {
476         // !(a && b && c) == !a || !b || !c
477         bool left = !(a && b && c);
478         bool right = !a || !b || !c;
479         return left == right;
480     }
481     public static bool DeMorganLaw36(bool a, bool b, bool c)
482     {
483         // !(a || b || c) == !a && !b && !c
484         bool left = !(a || b || c);
485         bool right = !a && !b && !c;
486         return left == right;
487     }
488     public static bool DeMorganLaw37(bool a, bool b, bool c)
489     {
490         // !(!a && !b && !c) == a || b || c
491         bool left = !(!a && !b && !c);
492         bool right = a || b || c;
493         return left == right;
494     }
495     public static bool DeMorganLaw38(bool a, bool b, bool c)
496     {
497         // !(!a || !b || !c) == a && b && c
498         bool left = !(!a || !b || !c);
499         bool right = a && b && c;
500         return left == right;
501     }
502     public static bool DeMorganLaw39(bool a, bool b, bool c)
503     {
504         // !(a && b) && c == (!a || !b) && c
505         bool left = !(a && b) && c;
506         bool right = (!a || !b) && c;
507         return left == right;
508     }
509     public static bool DeMorganLaw40(bool a, bool b, bool c)
510     {
511         // !(a || b) || c == (!a && !b) || c
512         bool left = !(a || b) || c;
513         bool right = (!a && !b) || c;
514         return left == right;
515     }
516     public static bool DeMorganLaw41(bool a, bool b, bool c)
517     {
518         // !(!a && !b) && c == (a || b) && c
519         bool left = !(!a && !b) && c;
520         bool right = (a || b) && c;
521         return left == right;
522     }
523     public static bool DeMorganLaw42(bool a, bool b, bool c)
524     {
525         // !(!a || !b) || c == (a && b) || c
526         bool left = !(!a || !b) || c;
527         bool right = (a && b) || c;
528         return left == right;
529     }
530     public static bool DeMorganLaw43(bool a, bool b, bool c)
531     {
532         // !(a && b && c) == !a || !b || !c
533         bool left = !(a && b && c);
534         bool right = !a || !b || !c;
535         return left == right;
536     }
537     public static bool DeMorganLaw44(bool a, bool b, bool c)
538     {
539         // !(a || b || c) == !a && !b && !c
540         bool left = !(a || b || c);
541         bool right = !a && !b && !c;
542         return left == right;
543     }
544     public static bool DeMorganLaw45(bool a, bool b, bool c)
545     {
546         // !(!a && !b && !c) == a || b || c
547         bool left = !(!a && !b && !c);
548         bool right = a || b || c;
549         return left == right;
550     }
551     public static bool DeMorganLaw46(bool a, bool b, bool c)
552     {
553         // !(!a || !b || !c) == a && b && c
554         bool left = !(!a || !b || !c);
555         bool right = a && b && c;
556         return left == right;
557     }
558     public static bool DeMorganLaw47(bool a, bool b, bool c)
559     {
560         // !(a && b) && c == (!a || !b) && c
561         bool left = !(a && b) && c;
562         bool right = (!a || !b) && c;
563         return left == right;
564     }
565     public static bool DeMorganLaw48(bool a, bool b, bool c)
566     {
567         // !(a || b) || c == (!a && !b) || c
568         bool left = !(a || b) || c;
569         bool right = (!a && !b) || c;
570         return left == right;
571     }
572     public static bool DeMorganLaw49(bool a, bool b, bool c)
573     {
574         // !(!a && !b) && c == (a || b) && c
575         bool left = !(!a && !b) && c;
576         bool right = (a || b) && c;
577         return left == right;
578     }
579     public static bool DeMorganLaw50(bool a, bool b, bool c)
580     {
581         // !(!a || !b) || c == (a && b) || c
582         bool left = !(!a || !b) || c;
583         bool right = (a && b) || c;
584         return left == right;
585     }
586     public static bool DeMorganLaw51(bool a, bool b, bool c)
587     {
588         // !(a && b && c) == !a || !b || !c
589         bool left = !(a && b && c);
590         bool right = !a || !b || !c;
591         return left == right;
592     }
593     public static bool DeMorganLaw52(bool a, bool b, bool c)
594     {
595         // !(a || b || c) == !a && !b && !c
596         bool left = !(a || b || c);
597         bool right = !a && !b && !c;
598         return left == right;
599     }
600     public static bool DeMorganLaw53(bool a, bool b, bool c)
601     {
602         // !(!a && !b && !c) == a || b || c
603         bool left = !(!a && !b && !c);
604         bool right = a || b || c;
605         return left == right;
606     }
607     public static bool DeMorganLaw54(bool a, bool b, bool c)
608     {
609         // !(!a || !b || !c) == a && b && c
610         bool left = !(!a || !b || !c);
611         bool right = a && b && c;
612         return left == right;
613     }
614     public static bool DeMorganLaw55(bool a, bool b, bool c)
615     {
616         // !(a && b) && c == (!a || !b) && c
617         bool left = !(a && b) && c;
618         bool right = (!a || !b) && c;
619         return left == right;
620     }
621     public static bool DeMorganLaw56(bool a, bool b, bool c)
622     {
623         // !(a || b) || c == (!a && !b) || c
624         bool left = !(a || b) || c;
625         bool right = (!a && !b) || c;
626         return left == right;
627     }
628     public static bool DeMorganLaw57(bool a, bool b, bool c)
629     {
630         // !(!a && !b) && c == (a || b) && c
631         bool left = !(!a && !b) && c;
632         bool right = (a || b) && c;
633         return left == right;
634     }
635     public static bool DeMorganLaw58(bool a, bool b, bool c)
636     {
637         // !(!a || !b) || c == (a && b) || c
638         bool left = !(!a || !b) || c;
639         bool right = (a && b) || c;
640         return left == right;
641     }
642     public static bool DeMorganLaw59(bool a, bool b, bool c)
643     {
644         // !(a && b && c) == !a || !b || !c
645         bool left = !(a && b && c);
646         bool right = !a || !b || !c;
647         return left == right;
648     }
649     public static bool DeMorganLaw60(bool a, bool b, bool c)
650     {
651         // !(a || b || c) == !a && !b && !c
652         bool left = !(a || b || c);
653         bool right = !a && !b && !c;
654         return left == right;
655     }
656     public static bool DeMorganLaw61(bool a, bool b, bool c)
657     {
658         // !(!a && !b && !c) == a || b || c
659         bool left = !(!a && !b && !c);
660         bool right = a || b || c;
661         return left == right;
662     }
663     public static bool DeMorganLaw62(bool a, bool b, bool c)
664     {
665         // !(!a || !b || !c) == a && b && c
666         bool left = !(!a || !b || !c);
667         bool right = a && b && c;
668         return left == right;
669     }
670     public static bool DeMorganLaw63(bool a, bool b, bool c)
671     {
672         // !(a && b) && c == (!a || !b) && c
673         bool left = !(a && b) && c;
674         bool right = (!a || !b) && c;
675         return left == right;
676     }
677     public static bool DeMorganLaw64(bool a, bool b, bool c)
678     {
679         // !(a || b) || c == (!a && !b) || c
680         bool left = !(a || b) || c;
681         bool right = (!a && !b) || c;
682         return left == right;
683     }
684     public static bool DeMorganLaw65(bool a, bool b, bool c)
685     {
686         // !(!a && !b) && c == (a || b) && c
687         bool left = !(!a && !b) && c;
688         bool right = (a || b) && c;
689         return left == right;
690     }
691     public static bool DeMorganLaw66(bool a, bool b, bool c)
692     {
693         // !(!a || !b) || c == (a && b) || c
694         bool left = !(!a || !b) || c;
695         bool right = (a && b) || c;
696         return left == right;
697     }
698     public static bool DeMorganLaw67(bool a, bool b, bool c)
699     {
700         // !(a && b && c) == !a || !b || !c
701         bool left = !(a && b && c);
702         bool right = !a || !b || !c;
703         return left == right;
704     }
705     public static bool DeMorganLaw68(bool a, bool b, bool c)
706     {
707         // !(a || b || c) == !a && !b && !c
708         bool left = !(a || b || c);
709         bool right = !a && !b && !c;
710         return left == right;
711     }
712     public static bool DeMorganLaw69(bool a, bool b, bool c)
713     {
714         // !(!a && !b && !c) == a || b || c
715         bool left = !(!a && !b && !c);
716         bool right = a || b || c;
717         return left == right;
718     }
719     public static bool DeMorganLaw70(bool a, bool b, bool c)
720     {
721         // !(!a || !b || !c) == a && b && c
722         bool left = !(!a || !b || !c);
723         bool right = a && b && c;
724         return left == right;
725     }
726     public static bool DeMorganLaw71(bool a, bool b, bool c)
727     {
728         // !(a && b) && c == (!a || !b) && c
729         bool left = !(a && b) && c;
730         bool right = (!a || !b) && c;
731         return left == right;
732     }
733     public static bool DeMorganLaw72(bool a, bool b, bool c)
734     {
735         // !(a || b) || c == (!a && !b) || c
736         bool left = !(a || b) || c;
737         bool right = (!a && !b) || c;
738         return left == right;
739     }
740     public static bool DeMorganLaw73(bool a, bool b, bool c)
741     {
742         // !(!a && !b) && c == (a || b) && c
743         bool left = !(!a && !b) && c;
744         bool right = (a || b) && c;
745         return left == right;
746     }
747     public static bool DeMorganLaw74(bool a, bool b, bool c)
748     {
749         // !(!a || !b) || c == (a && b) || c
750         bool left = !(!
```

```

135 {
136     // a || (b && c) == (a || b) && (a || c)
137     bool left = a || (b && c);
138     bool right = (a || b) && (a || c);
139     return left == right;
140 }
141

```

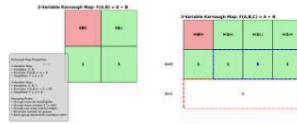
Mathematical formulation:

$$\neg(a \wedge b) = \neg a \vee \neg b \quad (\text{De Morgan's Law})$$

$$\neg(a \vee b) = \neg a \wedge \neg b$$

$$\bar{a} \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c) \quad (\text{Distributive Law})$$

Visual output: A Karnaugh map showing the minimized boolean function with grouped terms and the resulting simplified expression.



14.4 Combinatorics: Permutations, Combinations

Listing 55: Combinatorics in C#

```

1 public class Combinatorics
2 {
3     public static long Factorial(int n)
4     {
5         if (n < 0) throw new ArgumentException("Factorial not
6             defined for negative numbers");
7         if (n == 0 || n == 1) return 1;
8         long result = 1;
9         for (int i = 2; i <= n; i++)
10             result *= i;
11         return result;
12     }
13
14     public static long Permutation(int n, int r)
15     {
16         if (r > n) return 0;
17         return Factorial(n) / Factorial(n - r);
18     }
19
20     public static long Combination(int n, int r)
21     {
22         if (r > n) return 0;
23         if (r > n / 2) r = n - r; // Use symmetry
24         long result = 1;
25         for (int i = 0; i < r; i++)
26             result *= (n - i);
27         result /= (i + 1);
28         return result;
29     }
30
31     public static List<int[]> GeneratePermutations(int[] elements)
32     {
33         var permutations = new List<int[]>();
34         GeneratePermutationsHelper(elements, 0, permutations);
35         return permutations;
36     }
37
38     private static void GeneratePermutationsHelper(int[] elements,
39         int start, List<int[]> permutations)
40     {
41         if (start == elements.Length - 1)
42         {
43             permutations.Add((int[])elements.Clone());
44             return;
45         }
46
47         for (int i = start; i < elements.Length; i++)
48         {
49             // Swap elements
50             int temp = elements[start];
51             elements[start] = elements[i];
52             elements[i] = temp;
53
54             GeneratePermutationsHelper(elements, start + 1,
55                 permutations);
56
57             // Backtrack
58             temp = elements[start];
59             elements[start] = elements[i];
60             elements[i] = temp;
61         }
62     }
63
64     public static List<int[]> GenerateCombinations(int n, int r)
65     {
66         var combinations = new List<int[]>();
67         GenerateCombinationsHelper(n, r, new int[r], 0,
68             combinations);
69         return combinations;
70     }
71
72     private static void GenerateCombinationsHelper(int n, int r,
73         int[] start, int[] current, int index, List<int[]>
74             combinations)
75     {
76         if (index == r)
77         {
78             combinations.Add((int[])current.Clone());
79             return;
80         }
81         for (int i = start; i < n; i++)
82         {
83             current[index] = i;
84             GenerateCombinationsHelper(n, r, i + 1, current, index
85                 + 1, combinations);
86         }
87     }
88
89     public static long CatalanNumber(int n)
90     {
91         return Combination(2 * n, n) / (n + 1);
92     }
93
94     public static long StirlingNumberSecondKind(int n, int k)
95     {
96         if (k == 0 || k > n) return 0;
97         if (k == 1 || k == n) return 1;
98         return k * StirlingNumberSecondKind(n - 1, k) +
99             StirlingNumberSecondKind(n - 1, k - 1);
100     }
101
102     public static long BellNumber(int n)
103     {
104         long sum = 0;
105         for (int k = 0; k <= n; k++)
106             sum += StirlingNumberSecondKind(n, k);
107         return sum;
108     }
109 }

```

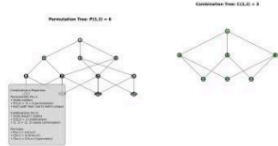
Mathematical formulation:

$$P(n, r) = \frac{n!}{(n - r)!}$$

$$C(n, r) = \binom{n}{r} = \frac{n!}{r!(n - r)!}$$

$$C_n = \frac{1}{n + 1} \binom{2n}{n} \quad (\text{Catalan Numbers})$$

Visual output: A tree diagram showing the generation of permutations and combinations, with each level representing a choice and branches showing the remaining options.



14.5 Mathematical Induction

Listing 56: Mathematical Induction in C#

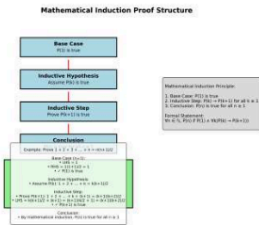
```
21 public class MathematicalInduction
22 {
23     public static bool ProveSumOfFirstN(int n)
24     {
25         // Prove: 1 + 2 + 3 + ... + n = n(n+1)/2
26
27         // Base case: n = 1
28         if (n == 1)
29         {
30             int sum = 1;
31             int formula = 1 * (1 + 1) / 2;
32             return sum == formula;
33         }
34
35         // Inductive step: assume true for n-1, prove for n
36         bool inductiveHypothesis = ProveSumOfFirstN(n - 1);
37         if (!inductiveHypothesis) return false;
38
39         // Calculate sum using formula
40         int sumUsingFormula = n * (n + 1) / 2;
41
42         // Calculate sum using inductive hypothesis
43         int sumUsingInduction = (n - 1) * n / 2 + n;
44
45         return sumUsingFormula == sumUsingInduction;
46     }
47
48     public static bool ProveSumOfSquares(int n)
49     {
50         // Prove: 1^2 + 2^2 + 3^2 + ... + n^2 = n(n+1)(2n+1)/6
51
52         if (n == 1)
53         {
54             int sum = 1;
55             int formula = 1 * (1 + 1) * (2 * 1 + 1) / 6;
56             return sum == formula;
57         }
58
59         bool inductiveHypothesis = ProveSumOfSquares(n - 1);
60         if (!inductiveHypothesis) return false;
61
62         int sumUsingFormula = n * (n + 1) * (2 * n + 1) / 6;
63         int sumUsingInduction = (n - 1) * n * (2 * (n - 1) + 1) / 6 + n * n;
64
65         return sumUsingFormula == sumUsingInduction;
66     }
67
68     public static bool ProveFibonacciFormula(int n)
69     {
70         // Prove: F(n) = (phi^n - (-phi)^(-n))/sqrt(5) where phi = (1+sqrt(5))/2
71
72         if (n <= 1) return true;
73
74         double phi = (1 + Math.Sqrt(5)) / 2;
75         double phiNeg = (1 - Math.Sqrt(5)) / 2;
76
77         double formulaValue = (Math.Pow(phi, n) - Math.Pow(phiNeg, n)) / Math.Sqrt(5);
78         int actualFibonacci = Fibonacci(n);
79
80         return Math.Abs(formulaValue - actualFibonacci) < 1e-10;
81     }
82
83     private static int Fibonacci(int n)
84     {
85         if (n <= 1) return n;
86         return Fibonacci(n - 1) + Fibonacci(n - 2);
87     }
88
89     public static bool ProvePowerSetSize(int n)
90     {
91         // Prove: |P(S)| = 2^|S| where P(S) is the power set of S
92     }
93 }
```

```
72 if (n == 0)
73 {
74     int powerSetSize = 1; // Empty set has power set (empty)
75     int formula = (int)Math.Pow(2, 0);
76     return powerSetSize == formula;
77 }
78
79 bool inductiveHypothesis = ProvePowerSetSize(n - 1);
80 if (!inductiveHypothesis) return false;
81
82 int powerSetSizeUsingFormula = (int)Math.Pow(2, n);
83 int powerSetSizeUsingInduction = (int)Math.Pow(2, n - 1) * 2;
84
85 return powerSetSizeUsingFormula == powerSetSizeUsingInduction;
86 }
87
88 public static bool ProveBinomialTheorem(int n, int k)
89 {
90     // Prove: (a+b)^n = sum(k=0 to n) C(n,k) * a^(n-k) * b^k
91
92     if (n == 0)
93     {
94         return k == 0 && Combination(0, 0) == 1;
95     }
96
97     // Use Pascal's identity: C(n,k) = C(n-1,k) + C(n-1,k-1)
98     long leftSide = Combination(n, k);
99     long rightSide = Combination(n - 1, k) + Combination(n - 1, k - 1);
100
101     return leftSide == rightSide;
102 }
103
104 }
```

Mathematical formulation:

Base Case: $P(1)$ is true
Inductive Step: $P(k) \Rightarrow P(k+1)$ for all $k \geq 1$
Conclusion: $P(n)$ is true for all $n \geq 1$

Visual output: A proof structure showing the base case verification, inductive hypothesis, and inductive step, with arrows connecting each step of the proof.



15 Graph Theory

Graph theory provides mathematical structures for modeling relationships and connectivity in data [23]. These concepts are fundamental for network analysis, social network modeling, and graph neural networks [6].

15.1 Graphs: Vertices, Edges, Directed/Undirected

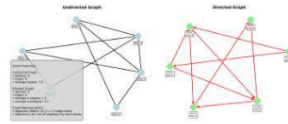
Listing 57: Graph Representation in C#

```
1 public class Graph
2 {
3     public int Vertices { get; }
4     public List<int>[] AdjacencyList { get; }
5     public bool IsDirected { get; }
6
7     public Graph(int vertices, bool isDirected = false)
8     {
9         Vertices = vertices;
10        IsDirected = isDirected;
11        AdjacencyList = new List<int>[Vertices];
12        for (int i = 0; i < Vertices; i++)
13            AdjacencyList[i] = new List<int>();
14    }
15
16    public void AddEdge(int source, int destination)
17    {
18        AdjacencyList[source].Add(destination);
19        if (!IsDirected)
20            AdjacencyList[destination].Add(source);
21    }
22
23    public void RemoveEdge(int source, int destination)
24    {
25        AdjacencyList[source].Remove(destination);
26        if (!IsDirected)
27            AdjacencyList[destination].Remove(source);
28    }
29
30    public bool HasEdge(int source, int destination)
31    {
32        return AdjacencyList[source].Contains(destination);
33    }
34
35    public int GetDegree(int vertex)
36    {
37        return AdjacencyList[vertex].Count;
38    }
39
40    public int GetInDegree(int vertex)
41    {
42        if (!IsDirected) return GetDegree(vertex);
43        int inDegree = 0;
44        for (int i = 0; i < Vertices; i++)
45        {
46            if (AdjacencyList[i].Contains(vertex))
47                inDegree++;
48        }
49        return inDegree;
50    }
51
52    public int GetOutDegree(int vertex)
53    {
54        return AdjacencyList[vertex].Count;
55    }
56
57    public List<int> GetNeighbors(int vertex)
58    {
59        return new List<int>(AdjacencyList[vertex]);
60    }
61
62 }
63
64 public bool IsConnected()
65 {
66     var visited = new bool[Vertices];
67     DFS(0, visited);
68     return visited.All(v => v);
69 }
70
71 private void DFS(int vertex, bool[] visited)
72 {
73     visited[vertex] = true;
74     foreach (int neighbor in AdjacencyList[vertex])
75     {
76         if (!visited[neighbor])
77             DFS(neighbor, visited);
78     }
79 }
80
81 public int CountConnectedComponents()
82 {
83     var visited = new bool[Vertices];
84     int components = 0;
85     for (int i = 0; i < Vertices; i++)
86     {
87         if (!visited[i])
88         {
89             DFS(i, visited);
90             components++;
91         }
92     }
93     return components;
94 }
95
96 }
97 }
```

Mathematical formulation:

$$G = (V, E)$$
$$\deg(v) = |\{u : (v, u) \in E\}|$$
$$\text{indeg}(v) = |\{u : (u, v) \in E\}|$$
$$\text{outdeg}(v) = |\{u : (v, u) \in E\}|$$

Visual output: A graph diagram showing vertices as circles and edges as lines connecting them, with arrows indicating direction for directed graphs.



15.2 Adjacency Matrices

Listing 58: Adjacency Matrix in C#

```
1 public class AdjacencyMatrix
2 {
3     public int[,] Matrix { get; }
4     public int Vertices { get; }
5     public bool IsDirected { get; }
6
7     public AdjacencyMatrix(int vertices, bool isDirected = false)
8     {
9         Vertices = vertices;
10        IsDirected = isDirected;
11        Matrix = new int[vertices, vertices];
12    }
13
14    public void AddEdge(int source, int destination)
15    {
16        Matrix[source, destination] = 1;
17        if (!IsDirected)
18            Matrix[destination, source] = 1;
19    }
20 }
```

```

20 public void RemoveEdge(int source, int destination)
21 {
22     Matrix[source, destination] = 0;
23     if (!IsDirected)
24         Matrix[destination, source] = 0;
25 }
26
27 public bool HasEdge(int source, int destination)
28 {
29     return Matrix[source, destination] != 0;
30 }
31
32 public int GetDegree(int vertex)
33 {
34     int degree = 0;
35     for (int i = 0; i < Vertices; i++)
36     {
37         degree += Matrix[vertex, i];
38     }
39     return degree;
40 }
41
42 public int[,] Power(int n)
43 {
44     if (n == 0)
45     {
46         var identity = new int[Vertices, Vertices];
47         for (int i = 0; i < Vertices; i++)
48             identity[i, i] = 1;
49         return identity;
50     }
51     if (n == 1)
52         return (int[,])Matrix.Clone();
53     var result = Power(n / 2);
54     result = MultiplyMatrices(result, result);
55     if (n % 2 == 1)
56         result = MultiplyMatrices(result, Matrix);
57     return result;
58 }
59
60 private int[,] MultiplyMatrices(int[,] a, int[,] b)
61 {
62     var result = new int[Vertices, Vertices];
63     for (int i = 0; i < Vertices; i++)
64     {
65         for (int j = 0; j < Vertices; j++)
66         {
67             for (int k = 0; k < Vertices; k++)
68             {
69                 result[i, j] += a[i, k] * b[k, j];
70             }
71         }
72     }
73     return result;
74 }
75
76 public int CountPaths(int source, int destination, int length)
77 {
78     var powerMatrix = Power(length);
79     return powerMatrix[source, destination];
80 }
81
82
83
84
85
86
87
88

```

```

89 public bool IsReachable(int source, int destination)
90 {
91     var reachability = new int[Vertices, Vertices];
92     for (int i = 0; i < Vertices; i++)
93     {
94         for (int j = 0; j < Vertices; j++)
95         {
96             reachability[i, j] = Matrix[i, j];
97         }
98     }
99
100     // Floyd-Warshall algorithm
101     for (int k = 0; k < Vertices; k++)
102     {
103         for (int i = 0; i < Vertices; i++)
104         {
105             for (int j = 0; j < Vertices; j++)
106             {
107                 reachability[i, j] = reachability[i, k] &
108                     (reachability[i, k] &
109                     reachability[k, j]);
110             }
111         }
112     }
113     return reachability[source, destination] != 0;
114 }
115
116

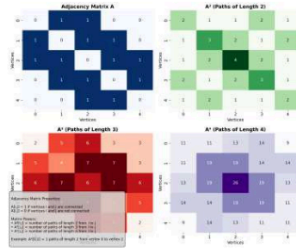
```

Mathematical formulation:

$$A_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

$(A^n)_{ij}$ = number of paths of length n from i to j

Visual output: A matrix representation showing 1s for connected vertices and 0s for disconnected vertices, with matrix powers showing path counts.



15.3 Breadth-First Search (BFS), Depth-First Search (DFS)

Listing 59: Graph Traversal in C#

```

1 public class GraphTraversal
2 {
3     public static int[] BFS(Graph graph, int startVertex)
4     {
5         var visited = new bool[graph.Vertices];
6         var distance = new int[graph.Vertices];
7         var queue = new Queue<int>();
8
9         visited[startVertex] = true;
10        distance[startVertex] = 0;
11        queue.Enqueue(startVertex);
12
13        while (queue.Count > 0)
14        {
15            int current = queue.Dequeue();
16
17            foreach (int neighbor in graph.GetNeighbors(current))
18            {
19                if (!visited[neighbor])

```

```

20                {
21                    visited[neighbor] = true;
22                    distance[neighbor] = distance[current] + 1;
23                    queue.Enqueue(neighbor);
24                }
25            }
26        }
27        return distance;
28    }
29
30    public static List<int> DFS(Graph graph, int startVertex)
31    {
32        var visited = new bool[graph.Vertices];
33        var traversal = new List<int>();
34
35        DFSRecursive(graph, startVertex, visited, traversal);
36        return traversal;
37    }
38
39    private static void DFSRecursive(Graph graph, int vertex, bool
40        [] visited, List<int> traversal)
41    {

```

```

42 visited[vertex] = true;
43 traversal.Add(vertex);
44
45 foreach (int neighbor in graph.GetNeighbors(vertex))
46 {
47     if (!visited[neighbor])
48         DFSRecursive(graph, neighbor, visited, traversal);
49 }
50 }
51
52 public static List<int> TopologicalSort(Graph graph)
53 {
54     if (!graph.IsDirected)
55         throw new InvalidOperationException("Topological sort
56         requires directed graph");
57
58     var visited = new bool[graph.Vertices];
59     var result = new List<int>();
60
61     for (int i = 0; i < graph.Vertices; i++)
62     {
63         if (!visited[i])
64             TopologicalSortDFS(graph, i, visited, tempVisited,
65             result);
66     }
67     result.Reverse();
68     return result;
69 }
70
71 private static void TopologicalSortDFS(Graph graph, int vertex,
72     bool[] visited, bool[] tempVisited, List<int> result)
73 {
74     if (tempVisited[vertex])
75         throw new InvalidOperationException("Graph contains
76         cycle");
77
78     if (visited[vertex])
79         return;
80
81     tempVisited[vertex] = true;
82
83     foreach (int neighbor in graph.GetNeighbors(vertex))
84     {
85         TopologicalSortDFS(graph, neighbor, visited,
86         tempVisited, result);
87     }
88
89     tempVisited[vertex] = false;
90     visited[vertex] = true;
91     result.Add(vertex);
92 }
93
94 public static bool HasCycle(Graph graph)
95 {
96     var visited = new bool[graph.Vertices];
97     var recStack = new bool[graph.Vertices];

```

```

96     for (int i = 0; i < graph.Vertices; i++)
97     {
98         if (!visited[i] && HasCycleDFS(graph, i, visited,
99         recStack))
100             return true;
101     }
102     return false;
103 }
104
105 private static bool HasCycleDFS(Graph graph, int vertex, bool[]
106     visited, bool[] recStack)
107 {
108     visited[vertex] = true;
109     recStack[vertex] = true;
110
111     foreach (int neighbor in graph.GetNeighbors(vertex))
112     {
113         if (!visited[neighbor] && HasCycleDFS(graph, neighbor,
114         visited, recStack))
115             return true;
116         else if (recStack[neighbor])
117             return true;
118     }
119     recStack[vertex] = false;
120     return false;
121 }

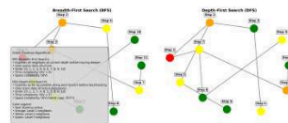
```

Mathematical formulation:

$$\text{BFS: } d[v] = \min_{u \in N(v)} d[u] + 1$$

$$\text{DFS: } \text{visit}(v) = \text{mark } v \text{ visited, then} \\ \forall u \in N(v) \text{ visit}(u)$$

Visual output: A graph with traversal order highlighted, showing BFS exploring in layers and DFS exploring deep into branches first.



15.4 Dijkstra's Algorithm

Listing 60: Dijkstra's Algorithm in C#

```

1 public class WeightedGraph
2 {
3     public int Vertices { get; }
4     public Dictionary<int, Dictionary<int, double>> AdjacencyList {
5         get; }
6
7     public WeightedGraph(int vertices)
8     {
9         Vertices = vertices;
10        AdjacencyList = new Dictionary<int, Dictionary<int, double>
11        >>();
12        for (int i = 0; i < vertices; i++)
13            AdjacencyList[i] = new Dictionary<int, double>();
14    }
15
16    public void AddEdge(int source, int destination, double weight)
17    {
18        AdjacencyList[source][destination] = weight;
19    }
20
21    public double[] Dijkstra(int startVertex)
22    {
23        var distances = new double[Vertices];
24        var visited = new bool[Vertices];
25        var pq = new SortedSet<double distance, int vertex>();
26
27        for (int i = 0; i < Vertices; i++)
28            distances[i] = double.MaxValue;

```

```

28        distances[startVertex] = 0;
29        pq.Add((0, startVertex));
30
31        while (pq.Count > 0)
32        {
33            var (currentDistance, currentVertex) = pq.Min;
34            pq.Remove(pq.Min);
35
36            if (visited[currentVertex])
37                continue;
38
39            visited[currentVertex] = true;
40
41            foreach (var neighbor in AdjacencyList[currentVertex])
42            {
43                int neighborVertex = neighbor.Key;
44                double weight = neighbor.Value;
45
46                if (!visited[neighborVertex])
47                {
48                    double newDistance = currentDistance + weight;
49                    if (newDistance < distances[neighborVertex])
50                    {
51                        distances[neighborVertex] = newDistance;
52                        pq.Add((newDistance, neighborVertex));
53                    }
54                }
55            }
56        }
57        return distances;
58    }
59 }
60

```

```

61 public (double[], int[]) predecessors
62 {
63     DijkstraWithPath(startVertex)
64     {
65         var distances = new double[Vertices];
66         var predecessors = new int[Vertices];
67         var visited = new bool[Vertices];
68         var pq = new SortedSet<(double distance, int vertex)>();
69         for (int i = 0; i < Vertices; i++)
70         {
71             distances[i] = double.MaxValue;
72             predecessors[i] = -1;
73         }
74         distances[startVertex] = 0;
75         pq.Add(0, startVertex);
76         while (pq.Count > 0)
77         {
78             var (currentDistance, currentVertex) = pq.Min;
79             pq.Remove(pq.Min);
80             if (visited[currentVertex])
81                 continue;
82             visited[currentVertex] = true;
83             foreach (var neighbor in AdjacencyList[currentVertex])
84             {
85                 int neighborVertex = neighbor.Key;
86                 double weight = neighbor.Value;
87                 if (!visited[neighborVertex])
88                 {
89                     double newDistance = currentDistance + weight;
90                     if (newDistance < distances[neighborVertex])
91                     {
92                         distances[neighborVertex] = newDistance;
93                         predecessors[neighborVertex] = currentVertex;
94                         pq.Add((newDistance, neighborVertex));
95                     }
96                 }
97             }
98         }
99         return (distances, predecessors);
100     }
101 }
102
103 public List<int> GetShortestPath(int startVertex, int endVertex)
104 {
105     var (distances, predecessors) = DijkstraWithPath(

```

```

112 startVertex);
113 if (distances[endVertex] == double.MaxValue)
114     return new List<int>(); // No path exists
115 var path = new List<int>();
116 int current = endVertex;
117 while (current != -1)
118 {
119     path.Add(current);
120     current = predecessors[current];
121 }
122 path.Reverse();
123 return path;
124 }
125 }
126 }
127 }
128 }

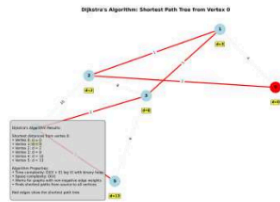
```

Mathematical formulation:

$$d[v] = \min(d[v], d[u] + w(u, v))$$

where u is the vertex with minimum $d[u]$ among unvisited vertices.

Visual output: A weighted graph showing the shortest path tree with distances labeled at each vertex and the path highlighted.



15.5 Graph Representation for GNNs

Listing 61: Graph Neural Network Representation in C#

```

1 public class GraphNeuralNetwork
2 {
3     public int Vertices { get; }
4     public int FeatureDimension { get; }
5     public double[,] AdjacencyMatrix { get; }
6     public double[,] NodeFeatures { get; }
7     public GraphNeuralNetwork(int vertices, int featureDimension)
8     {
9         Vertices = vertices;
10        FeatureDimension = featureDimension;
11        AdjacencyMatrix = new double[vertices, vertices];
12        NodeFeatures = new double[vertices, featureDimension];
13    }
14    public void AddEdge(int source, int destination, double weight = 1.0)
15    {
16        AdjacencyMatrix[source, destination] = weight;
17        AdjacencyMatrix[destination, source] = weight; // Undirected
18    }
19    public void SetNodeFeature(int vertex, double[] features)
20    {
21        for (int i = 0; i < FeatureDimension; i++)
22        {
23            NodeFeatures[vertex, i] = features[i];
24        }
25    }
26    public double[,] GraphConvolution(double[,] weights, double bias = 0.0)

```

```

32 // h = sigma(Axw + b)
33 var result = new double[Vertices, weights.GetLength(1)];
34
35 // Compute Ax
36 var ax = new double[Vertices, FeatureDimension];
37 for (int i = 0; i < Vertices; i++)
38 {
39     for (int j = 0; j < FeatureDimension; j++)
40     {
41         for (int k = 0; k < Vertices; k++)
42         {
43             ax[i, j] += AdjacencyMatrix[i, k] * NodeFeatures[k, j];
44         }
45     }
46 }
47 // Compute Axw
48 for (int i = 0; i < Vertices; i++)
49 {
50     for (int j = 0; j < weights.GetLength(1); j++)
51     {
52         for (int k = 0; k < FeatureDimension; k++)
53         {
54             result[i, j] += ax[i, k] * weights[k, j];
55         }
56     }
57     result[i, j] += bias;
58 }
59 }
60 // Apply activation function (ReLU)
61 for (int i = 0; i < Vertices; i++)
62 {
63     for (int j = 0; j < weights.GetLength(1); j++)
64     {
65         result[i, j] = Math.Max(0, result[i, j]);
66     }
67 }

```

```

68     }
69     return result;
70 }
71
72 public double[,] AttentionMechanism(double[,] query, double[,]
73     key, double[,] value)
74 {
75     // Attention(Q,K,V) = softmax(QK^T/sqrt(d_k))V
76     int dk = key.GetLength(1);
77     var attention = new double[Vertices, value.GetLength(1)];
78
79     // Compute QK^T
80     var qkt = new double[Vertices, Vertices];
81     for (int i = 0; i < Vertices; i++)
82     {
83         for (int j = 0; j < Vertices; j++)
84         {
85             for (int k = 0; k < dk; k++)
86             {
87                 qkt[i, j] += query[i, k] * key[j, k];
88             }
89             qkt[i, j] /= Math.Sqrt(dk);
90         }
91     }
92
93     // Apply softmax
94     for (int i = 0; i < Vertices; i++)
95     {
96         double maxVal = qkt[i, 0];
97         for (int j = 1; j < Vertices; j++)
98         {
99             if (qkt[i, j] > maxVal)
100                 maxVal = qkt[i, j];
101         }
102         double sum = 0;
103         for (int j = 0; j < Vertices; j++)
104         {
105             qkt[i, j] = Math.Exp(qkt[i, j] - maxVal);
106             sum += qkt[i, j];
107         }
108         for (int j = 0; j < Vertices; j++)
109         {
110             qkt[i, j] /= sum;
111         }
112     }
113
114     // Compute final attention
115     for (int i = 0; i < Vertices; i++)
116     {
117         for (int j = 0; j < value.GetLength(1); j++)
118         {
119             for (int k = 0; k < Vertices; k++)
120             {
121                 attention[i, j] += qkt[i, k] * value[k, j];
122             }
123         }
124     }

```

```

124     }
125 }
126
127 public double[,] NodeClassification(double[,] finalFeatures,
128     double[,] classifier)
129 {
130     var predictions = new double[Vertices];
131     for (int i = 0; i < Vertices; i++)
132     {
133         for (int j = 0; j < classifier.GetLength(1); j++)
134         {
135             for (int k = 0; k < finalFeatures.GetLength(1); k++)
136             {
137                 predictions[i] += finalFeatures[i, k] *
138                     classifier[k, j];
139             }
140         }
141     }
142     return predictions;
143 }
144
145 }
146
147 }
148

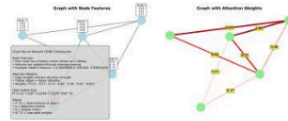
```

Mathematical formulation:

$$H^{(l+1)} = \sigma(D^{-\frac{1}{2}} A D^{-\frac{1}{2}} H^{(l)} W^{(l)})$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Visual output: A graph with node features as vectors and attention weights as edge thickness, showing how information flows through the graph structure.



16 Computability and Complexity

Computational complexity theory provides mathematical frameworks for analyzing algorithm efficiency and computational limits [2]. Understanding these concepts is essential for designing efficient machine learning algorithms and understanding their theoretical limitations [11].

16.1 Big O, Big Omega, Big Theta Notation

Listing 62: Complexity Analysis in C#

```

1 public class ComplexityAnalysis
2 {
3     public static int LinearSearch(int[] array, int target)
4     {
5         for (int i = 0; i < array.Length; i++)
6         {
7             if (array[i] == target)
8                 return i;
9         }
10        return -1; // O(n) time complexity
11    }
12
13    public static int BinarySearch(int[] array, int target)
14    {
15        int left = 0;
16        int right = array.Length - 1;
17        while (left <= right)
18        {
19            int mid = left + (right - left) / 2;
20

```

```

21            if (array[mid] == target)
22                return mid;
23            else if (array[mid] < target)
24                left = mid + 1;
25            else
26                right = mid - 1;
27        }
28        return -1; // O(log n) time complexity
29    }
30
31    public static void BubbleSort(int[] array)
32    {
33        int n = array.Length;
34        for (int i = 0; i < n - 1; i++)
35        {
36            for (int j = 0; j < n - i - 1; j++)
37            {
38                if (array[j] > array[j + 1])
39                {
40                    int temp = array[j];
41                    array[j] = array[j + 1];
42                    array[j + 1] = temp;
43                }
44            }
45        }
46    }

```

```

44     }
45     } // O(n^2) time complexity
46 }
47
48 public static void MergeSort(int[] array, int left, int right)
49 {
50     if (left < right)
51     {
52         int mid = left + (right - left) / 2;
53         MergeSort(array, left, mid);
54         MergeSort(array, mid + 1, right);
55         Merge(array, left, mid, right);
56     } // O(n log n) time complexity
57 }
58
59 private static void Merge(int[] array, int left, int mid, int
60 right)
61 {
62     int n1 = mid - left + 1;
63     int n2 = right - mid;
64     int[] leftArray = new int[n1];
65     int[] rightArray = new int[n2];
66
67     for (int i = 0; i < n1; i++)
68         leftArray[i] = array[left + i];
69     for (int j = 0; j < n2; j++)
70         rightArray[j] = array[mid + 1 + j];
71
72     int i1 = 0, j1 = 0, k = left;
73
74     while (i1 < n1 && j1 < n2)
75     {
76         if (leftArray[i1] <= rightArray[j1])
77         {
78             array[k] = leftArray[i1];
79             i1++;
80         }
81         else
82         {
83             array[k] = rightArray[j1];
84             j1++;
85         }
86         k++;
87     }
88
89     while (i1 < n1)
90     {
91         array[k] = leftArray[i1];
92         i1++;
93         k++;
94     }
95
96     while (j1 < n2)
97     {
98         array[k] = rightArray[j1];
99         j1++;
100         k++;
101     }
102 }
103
104 public static long FibonacciRecursive(int n)
105 {
106     if (n <= 1) return n;
107     return FibonacciRecursive(n - 1) + FibonacciRecursive(n -
108 2);
109 } // O(2^n) time complexity
110
111 public static long FibonacciDynamic(int n)
112 {
113     if (n <= 1) return n;
114
115     long[] fib = new long[n + 1];
116     fib[0] = 0;
117     fib[1] = 1;
118     for (int i = 2; i <= n; i++)
119     {
120         fib[i] = fib[i - 1] + fib[i - 2];
121     }
122 }

```

```

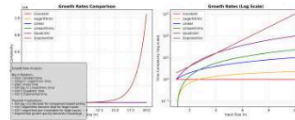
123
124     return fib[n]; // O(n) time complexity
125 }
126
127 public static bool IsPrime(int n)
128 {
129     if (n <= 1) return false;
130     if (n <= 3) return true;
131     if (n % 2 == 0 || n % 3 == 0) return false;
132
133     for (int i = 5; i + i <= n; i += 6)
134     {
135         if (n % i == 0 || n % (i + 2) == 0)
136             return false;
137     }
138     return true; // O(sqrt(n)) time complexity
139 }
140
141 public static int[] GeneratePrimes(int n)
142 {
143     bool[] isPrime = new bool[n + 1];
144     for (int i = 2; i <= n; i++)
145         isPrime[i] = true;
146
147     for (int i = 2; i + i <= n; i++)
148     {
149         if (isPrime[i])
150         {
151             for (int j = i + i; j <= n; j += i)
152                 isPrime[j] = false;
153         }
154     }
155
156     var primes = new List<int>();
157     for (int i = 2; i <= n; i++)
158     {
159         if (isPrime[i])
160             primes.Add(i);
161     }
162     return primes.ToArray(); // O(n log log n) time complexity
163 }
164
165 }
166
167 }

```

Mathematical formulation:

$$\begin{aligned}
 f(n) &= O(g(n)) && \text{if } \exists c > 0, n_0 > 0 : \\
 &f(n) \leq c \cdot g(n) && \text{for } n \geq n_0 \\
 f(n) &= \Omega(g(n)) && \text{if } \exists c > 0, n_0 > 0 : \\
 &f(n) \geq c \cdot g(n) && \text{for } n \geq n_0 \\
 f(n) &= \Theta(g(n)) && \text{if } f(n) = O(g(n)) \\
 &\text{and } f(n) = \Omega(g(n))
 \end{aligned}$$

Visual output: A plot showing different growth rates: constant, logarithmic, linear, linearithmic, quadratic, and exponential functions.



16.2 Polynomial vs Exponential Time

Listing 63: Polynomial vs Exponential Algorithms in C#

```

1 public class ComplexityComparison
2 {
3     public static int PolynomialAlgorithm(int n)
4     {
5         int result = 0;
6         for (int i = 0; i < n; i++)

```

```

7         {
8             for (int j = 0; j < n; j++)
9             {
10                 result += i + j; // O(n^2) polynomial time
11             }
12         }
13         return result;
14     }
15
16     public static long ExponentialAlgorithm(int n)
17     {

```

```

18 if (n <= 1) return 1;
19 return ExponentialAlgorithm(n - 1) + ExponentialAlgorithm(n
20 // O(2^n) exponential time
21
22
23 public static int SubsetSum(int[] numbers, int target)
24 {
25     int n = numbers.Length;
26     int totalSubsets = 1 << n; // 2^n
27
28     for (int i = 0; i < totalSubsets; i++)
29     {
30         int sum = 0;
31         for (int j = 0; j < n; j++)
32         {
33             if ((i & (1 << j)) != 0)
34             {
35                 sum += numbers[j];
36             }
37             if (sum == target)
38                 return i;
39         }
40     }
41     return -1; // O(2^n) exponential time
42 }
43
44 public static bool TravelingSalesman(int[,] distances, int
45 startCity)
46 {
47     int n = distances.GetLength(0);
48     int[] cities = new int[n];
49     for (int i = 0; i < n; i++)
50         cities[i] = i;
51
52     double minCost = double.MaxValue;
53
54     do
55     {
56         double cost = 0;
57         int currentCity = startCity;
58         for (int i = 0; i < n; i++)
59         {
60             int nextCity = cities[i];
61             cost += distances[currentCity, nextCity];
62             currentCity = nextCity;
63         }
64         cost += distances[currentCity, startCity];
65
66         if (cost < minCost)
67             minCost = cost;
68     } while (NextPermutation(cities)); // O(n!) factorial time
69
70     return minCost < double.MaxValue;
71 }
72
73
74 private static bool NextPermutation(int[] array)
75 {
76     int i = array.Length - 2;
77     while (i >= 0 && array[i] >= array[i + 1])
78         i--;
79
80     if (i < 0)
81         return false;
82
83     int j = array.Length - 1;
84     while (array[j] <= array[i])
85         j--;

```

```

87 int temp = array[i];
88 array[i] = array[j];
89 array[j] = temp;
90
91 Array.Reverse(array, i + 1, array.Length - i - 1);
92 return true;
93 }
94
95 public static int Knapsack(int[] weights, int[] values, int
96 capacity)
97 {
98     int n = weights.Length;
99     int[,] dp = new int[n + 1, capacity + 1];
100     for (int i = 1; i <= n; i++)
101     {
102         for (int w = 0; w <= capacity; w++)
103         {
104             if (weights[i - 1] <= w)
105             {
106                 dp[i, w] = Math.Max(dp[i - 1, w],
107                                     dp[i - 1, w - weights[i - 1]] + values[i - 1]);
108             }
109             else
110             {
111                 dp[i, w] = dp[i - 1, w];
112             }
113         }
114     }
115     return dp[n, capacity]; // O(nW) pseudo-polynomial time
116 }
117
118 }

```

Mathematical formulation:

Polynomial: $O(n^k)$

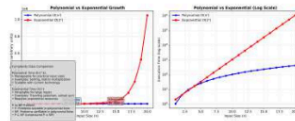
for some constant k

Exponential: $O(2^n), O(n!)$

Pseudo-polynomial: $O(nW)$

where W is input
value

Visual output: A comparison chart showing execution time growth for polynomial algorithms (manageable) vs exponential algorithms (intractable) as input size increases.



16.3 P vs NP Concepts

Listing 64: P vs NP Problems in C#

```

1 public class PvsNPProblems
2 {
3     // P Problems (Polynomial time)
4     public static bool IsPrimeP(int n)
5     {
6         if (n <= 1) return false;
7         if (n <= 3) return true;
8         if (n % 2 == 0 || n % 3 == 0) return false;
9
10        for (int i = 5; i * i <= n; i += 6)
11        {
12            if (n % i == 0 || n % (i + 2) == 0)
13                return false;
14        }
15        return true; // O(sqrt(n)) ~ ln P
16    }
17
18    public static int[] ShortestPath(int[,] graph, int start, int
19    end)
20    {
21        int n = graph.GetLength(0);
22        int[] distances = new int[n];
23        bool[] visited = new bool[n];
24        for (int i = 0; i < n; i++)
25            distances[i] = int.MaxValue;
26
27        distances[start] = 0;
28
29        for (int count = 0; count < n - 1; count++)
30        {
31            int minDistance = int.MaxValue;
32            int minIndex = -1;
33
34            for (int v = 0; v < n; v++)
35            {
36                if (!visited[v] && distances[v] <= minDistance)
37

```

```

38 minDistance = distances[v];
39 minIndex = v;
40 }
41 }
42 visited[minIndex] = true;
43
44 for (int v = 0; v < n; v++)
45 {
46     if (!visited[v] && graph[minIndex, v] != 0 &&
47         distances[minIndex] != Int.MaxValue &&
48         distances[minIndex] + graph[minIndex, v] <
49         distances[v])
50     {
51         distances[v] = distances[minIndex] + graph[
52             minIndex, v];
53     }
54 }
55
56 return distances; // O(V^2) - in P
57 }
58
59 // NP Problems (Non-deterministic Polynomial time)
60 public static bool SubsetSumNP(int[] numbers, int target)
61 {
62     int n = numbers.Length;
63     int totalSubsets = 1 << n;
64     for (int i = 0; i < totalSubsets; i++)
65     {
66         int sum = 0;
67         for (int j = 0; j < n; j++)
68         {
69             if ((i & (1 << j)) != 0)
70             {
71                 sum += numbers[j];
72             }
73         }
74         if (sum == target)
75             return true;
76     }
77     return false; // O(2^n) - in NP
78 }
79
80 public static bool ThreeColorability(int[,] graph)
81 {
82     int n = graph.GetLength(0);
83     int totalColorings = (int)Math.Pow(3, n);
84     for (int coloring = 0; coloring < totalColorings; coloring++)
85     {
86         int[] colors = new int[n];
87         int temp = coloring;
88         for (int i = 0; i < n; i++)
89         {
90             colors[i] = temp % 3;
91             temp /= 3;
92         }
93         bool isValid = true;
94         for (int i = 0; i < n; i++)
95         {
96             for (int j = i + 1; j < n; j++)
97             {
98                 if (graph[i, j] == 1 && colors[i] == colors[j])
99                 {
100                     isValid = false;
101                     break;
102                 }
103             }
104             if (!isValid) break;
105         }
106         if (isValid)
107             return true;
108     }
109     return false; // O(3^n) - in NP
110 }
111
112 // NP-Complete Problems
113 public static bool BooleanSatisfiability(string expression)
114 {
115     // Simplified 3-SAT problem
116     // Expression format: "(x1 OR x2 OR x3) AND (x1 OR NOT x2
117     // OR x4)"
118     var clauses = ParseExpression(expression);
119     int numVars = CountVariables(expression);
120     int totalAssignments = 1 << numVars;
121     for (int assignment = 0; assignment < totalAssignments;
122         assignment++)
123     {

```

```

129 bool[] values = new bool[numVars];
130 for (int i = 0; i < numVars; i++)
131 {
132     values[i] = (assignment & (1 << i)) != 0;
133 }
134
135 bool satisfiesAll = true;
136 foreach (var clause in clauses)
137 {
138     bool satisfiesClause = false;
139     foreach (var literal in clause)
140     {
141         int varIndex = Math.Abs(literal) - 1;
142         bool varValue = literal > 0 ? values[varIndex]
143             : !values[varIndex];
144         if (varValue)
145         {
146             satisfiesClause = true;
147             break;
148         }
149     }
150     if (!satisfiesClause)
151     {
152         satisfiesAll = false;
153         break;
154     }
155 }
156 if (satisfiesAll)
157     return true;
158
159 return false; // O(2^n) - NP-Complete
160
161 private static List<List<int>> ParseExpression(string
162     expression)
163 {
164     // Simplified parser for demonstration
165     var clauses = new List<List<int>>();
166     // Implementation would parse the expression into clauses
167     return clauses;
168 }
169
170 private static int CountVariables(string expression)
171 {
172     // Simplified variable counter
173     return 4; // For demonstration
174 }
175
176 }

```

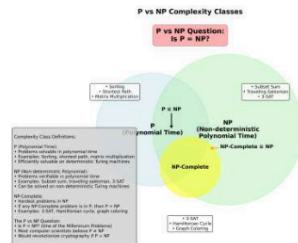
Mathematical formulation:

$P = \{L : L \text{ can be decided in polynomial time}\}$

$NP = \{L : L \text{ can be verified in polynomial time}\}$

$NP\text{-Complete} = \{L : L \in NP \text{ and } L \text{ is NP-hard}\}$

Visual output: A Venn diagram showing the relationship between P, NP, and NP-Complete problems, with the famous P vs NP question highlighted.



16.4 Reducibility

Listing 65: Problem Reductions in C#

```
1 public class ProblemReductions
2 {
3     // Reduction from Subset Sum to Knapsack
4     public static bool SubsetSumToKnapsack(int[] numbers, int
5         target)
6     {
7         // Transform Subset Sum to Knapsack:
8         // weights = values = numbers, capacity = target
9         int maxValue = Knapsack(numbers, numbers, target);
10        return maxValue == target;
11    }
12
13    private static int Knapsack(int[] weights, int[] values, int
14        capacity)
15    {
16        int n = weights.Length;
17        int[] dp = new int[n + 1, capacity + 1];
18        for (int i = 1; i <= n; i++)
19        {
20            for (int w = 0; w <= capacity; w++)
21            {
22                if (weights[i - 1] <= w)
23                {
24                    dp[i, w] = Math.Max(dp[i - 1, w],
25                        dp[i - 1, w - weights[i - 1]] + values[i - 1]);
26                }
27                else
28                {
29                    dp[i, w] = dp[i - 1, w];
30                }
31            }
32        }
33        return dp[n, capacity];
34    }
35
36    // Reduction from 3-SAT to Independent Set
37    public static bool ThreeSATtoIndependentSet(string expression)
38    {
39        var clauses = ParseThreeSAT(expression);
40        var graph = CreateIndependentSetGraph(clauses);
41        int k = clauses.Count;
42        return HasIndependentSetOfSize(graph, k);
43    }
44
45    private static List<List<int>> ParseThreeSAT(string expression)
46    {
47        // Simplified parser for 3-SAT
48        var clauses = new List<List<int>>();
49        // Implementation would parse the expression
50        return clauses;
51    }
52
53    private static int[,] CreateIndependentSetGraph(List<List<int>>
54        clauses)
55    {
56        // Create graph where each literal in each clause is a
57        // vertex
58        // Add edges between literals in same clause and between
59        // complementary literals
60        int numVertices = clauses.Count * 3; // 3 literals per
61        clause
62        var graph = new int[numVertices, numVertices];
63        // Implementation would create the graph structure
64        return graph;
65    }
66
67    private static bool HasIndependentSetOfSize(int[,] graph, int k)
68    {
69        int n = graph.GetLength(0);
70        int totalSubsets = 1 << n;
71        for (int subset = 0; subset < totalSubsets; subset++)
72        {
73            if (CountBits(subset) == k)
74            {
75                bool isIndependent = true;
76                for (int i = 0; i < n; i++)
77                {
78                    if ((subset & (1 << i)) != 0)
79                    {
80                        for (int j = i + 1; j < n; j++)
81                        {
82                            if ((subset & (1 << j)) != 0 && graph[i,
83                                j] == 1)
84                                break;
85                        }
86                        if (!isIndependent) break;
87                    }
88                }
89                if (isIndependent)
90                    return true;
91            }
92        }
93        return false;
94    }
95
96    private static int CountBits(int n)
97    {
98        int count = 0;
99        while (n > 0)
100        {
101            count += n & 1;
102            n >>= 1;
103        }
104        return count;
105    }
106
107    // Reduction from Hamiltonian Cycle to Traveling Salesman
108    public static bool HamiltonianCycleToTSP(int[,] graph)
109    {
110        int n = graph.GetLength(0);
111        int[,] tspGraph = new int[n, n];
112        // Transform: if edge exists in original graph, distance =
113        // 1, else distance = n-1
114        for (int i = 0; i < n; i++)
115        {
116            for (int j = 0; j < n; j++)
117            {
118                if (i == j)
119                    tspGraph[i, j] = 0;
120                else if (graph[i, j] == 1)
121                    tspGraph[i, j] = 1;
122                else
123                    tspGraph[i, j] = n - 1;
124            }
125        }
126        // Check if TSP has solution with cost = n
127        return TravelingSalesmanTSP(tspGraph, n);
128    }
129
130    private static bool TravelingSalesmanTSP(int[,] distances, int
131        targetCost)
132    {
133        int n = distances.GetLength(0);
134        int[] cities = new int[n];
135        for (int i = 0; i < n; i++)
136            cities[i] = i;
137        do
138        {
139            int cost = 0;
140            int currentCity = 0;
141            for (int i = 0; i < n; i++)
142            {
143                int nextCity = cities[i];
144                cost += distances[currentCity, nextCity];
145                currentCity = nextCity;
146            }
147            cost += distances[currentCity, 0];
148            if (cost == targetCost)
149                return true;
150        } while (NextPermutation(cities));
151        return false;
152    }
153
154    private static bool NextPermutation(int[] array)
155    {
156        int i = array.Length - 2;
157        while (i >= 0 && array[i] >= array[i + 1])
158            i--;
159        if (i < 0)
160            return false;
161        int j = array.Length - 1;
162        while (array[j] <= array[i])
163            j--;
164        int temp = array[i];
165        array[i] = array[j];
166        array[j] = temp;
167    }
168}
```

```

177 Array.Reverse(array, 1 * 3, array.Length - 1 * 3);
178     return true;
179 }
180 }

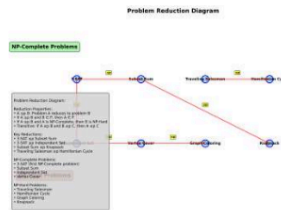
```

Mathematical formulation:

$$A \leq_p B \quad \text{iff} \quad \exists f : \text{polynomial time reduction} \\ \text{where } x \in A \iff f(x) \in B$$

Visual output: A reduction diagram showing how one problem can be transformed into another, with arrows indicating the direction of reduction and polynomial-

time transformations.



References

- [1] Charu C. Aggarwal and Chandan K. Reddy. Data clustering: Algorithms and applications. *CRC Press*, 2014.
- [2] Sanjeev Arora and Boaz Barak. Computational complexity: A modern approach. *Cambridge University Press*, 2009.
- [3] Christopher M. Bishop. Pattern recognition and machine learning. *Springer*, 2006.
- [4] George E. P. Box, Gwilym M. Jenkins, and Gregory C. Reinsel. Time series analysis: Forecasting and control. *Wiley Series in Probability and Statistics*, 2015.
- [5] Stephen Boyd and Lieven Vandenberghe. Convex optimization. *Cambridge University Press*, 2004.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to algorithms. *MIT Press*, 2009.
- [7] Thomas M. Cover and Joy A. Thomas. Elements of information theory. *Wiley*, 2006.
- [8] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. Mathematics for machine learning. *Cambridge University Press*, 2020.
- [9] Julian J. Faraway. Linear models with r. *Chapman & Hall/CRC*, 2014.
- [10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. *MIT Press*, 2016.
- [11] Aurélien Géron. Hands-on machine learning with scikit-learn, keras, and tensorflow. *O'Reilly Media*, 2019.
- [12] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. The elements of statistical learning: Data mining, inference, and prediction. *Springer Series in Statistics*, 2009.
- [13] David J. C. MacKay. Information theory, inference, and learning algorithms. *Cambridge University Press*, 2003.
- [14] Michael A. Nielsen. Neural networks and deep learning. *Determination Press*, 2015.
- [15] Jorge Nocedal and Stephen J. Wright. Numerical optimization. *Springer Series in Operations Research*, 2006.
- [16] Mitchel Resnick. Lifelong kindergarten: Cultivating creativity through projects, passion, peers, and play. *MIT Press*, 2017.
- [17] Kenneth H. Rosen. Discrete mathematics and its applications. *McGraw-Hill Education*, 2018.
- [18] Sheldon M. Ross. Introduction to probability and statistics. *Academic Press*, 2014.
- [19] Marina Sokolova and Guy Lapalme. Evaluation metrics for machine learning. *Journal of Machine Learning Research*, 10:2075–2089, 2009.
- [20] James Stewart. Calculus: Early transcendentals. *Cengage Learning*, 2015.
- [21] Gilbert Strang. Linear algebra and its applications. *Academic Press*, 2006.
- [22] Laurens Van der Maaten and Geoffrey Hinton. Dimensionality reduction: A comparative review. *Journal of Machine Learning Research*, 9:2579–2605, 2008.

- [23] Douglas B. West. Introduction to graph theory. *Prentice Hall*, 2001.
- [24] Jeannette M. Wing. Computational thinking. *Communications of the ACM*, 49(3):33–35, 2006.

Mathematical Foundations for Machine Learning_compressed.pdf

ORIGINALITY REPORT

23%

SIMILARITY INDEX

19%

INTERNET SOURCES

22%

PUBLICATIONS

11%

STUDENT PAPERS

PRIMARY SOURCES

1

turhancoban.com

Internet Source

4%

2

nozdr.ru

Internet Source

3%

3

www.javatips.net

Internet Source

2%

4

fivedots.coe.psu.ac.th

Internet Source

1%

5

archive.org

Internet Source

1%

6

gitlab.eduxiji.net

Internet Source

1%

7

Waldemar Dos Passos. "Numerical Methods, Algorithms and Tools in C#", CRC Press, 2019

Publication

1%

8

Submitted to University of Ghana

Student Paper

1%

9

doc.lagout.org

Internet Source

1%

10

Submitted to CSU, San Jose State University

Student Paper

1%

11

Daniel J. Duffy, Andrea Germani. "C# for Financial Markets", Wiley, 2013

Publication

1%

12	Yonghui Wu, Jiande Wang. "Algorithm Design Practice for Collegiate Programming Contests and Education", CRC Press, 2018 Publication	1 %
13	Antti Laaksonen. "Guide to Competitive Programming", Springer Science and Business Media LLC, 2024 Publication	1 %
14	Daniel Duffy. "Financial Instrument Pricing Using C++ 2e + Website", Wiley, 2018 Publication	1 %
15	Steinmetz, Shannon. "Music Information Geometry – Algebraic Group, Metric, and Equivalence for A-Priori, Real-Time Sensation of Audio.", University of Colorado at Denver Publication	<1 %
16	Submitted to University of Southern California Student Paper	<1 %
17	"A Concise and Practical Introduction to Programming Algorithms in Java", Springer Science and Business Media LLC, 2009 Publication	<1 %
18	Submitted to Oxford Brookes University Student Paper	<1 %
19	Submitted to University of Hong Kong Student Paper	<1 %
20	de Carvalho, Tiago Diogo Ribeiro. "Programming and Mapping Strategies for Embedded Computing Runtime Adaptability", Universidade do Porto (Portugal), 2024 Publication	<1 %
21	Rubin H. Landau, Manuel J. Páez, Cristian C. Bordeianu. "Computational Physics", Wiley, 2007	<1 %

-
- 22 Geraldles, Carlos José Brás. "Aplicação das Redes Neurais Aditivas Generalizadas à Medicina", Universidade NOVA de Lisboa (Portugal), 2024
Publication
-
- 23 Yair Shapira. "Mathematical Objects in C++ - Computational Tools in A Unified Object-Oriented Approach", CRC Press, 2019
Publication
-
- 24 Vladimir O. Safonov. "Appendix: Examples of Aspect.NET Aspects", Wiley, 2008
Publication
-
- 25 Adams, Andrew Bensley. "High-Dimensional Gaussian Filtering for Computational Photography.", Stanford University, 2020
Publication
-
- 26 Przybyla, Craig P.. "Methodology for Determining the Variance of the Taylor Factor: Application in Fe-3%Si.", Brigham Young University, 2020
Publication
-
- 27 Yonghui Wu, Jiande Wang. "Data Structure Practice - for Collegiate Programming Contests and Education", CRC Press, 2019
Publication
-
- 28 Felipe de Lima Peressim. "Machine learning for the prediction of 28-day cement compressive strength", Universidade de São Paulo. Agência de Bibliotecas e Coleções Digitais, 2025
Publication
-
- 29 Ali Kashefi, Tapan Mukerji. "ChatGPT FOR PROGRAMMING NUMERICAL METHODS",

Journal of Machine Learning for Modeling and Computing, 2023

Publication

30

Campbell, Daniel . "Classification and Clustering of Functional Eye-Tracking Data for Autism Spectrum Disorders", Proquest, 2012.

Publication

<1 %

31

Submitted to Griffth University

Student Paper

<1 %

32

Michael Roth. "Modeling and Simulation of Everyday Things", CRC Press, 2025

Publication

<1 %

33

Submitted to Sri Lanka Institute of Information Technology

Student Paper

<1 %

34

Dos Passos, . "Random Numbers", Numerical Methods Algorithms and Tools in C#, 2009.

Publication

<1 %

35

Tzoannopoulos, Konstantinos-Dimitrios. "Development of a Multi-Agent System for the Support of Group Decisions Utilizing Argumentation and Multicriteria Methods", Technical University of Crete (Greece)

Publication

<1 %

36

Submitted to St. Xavier's College

Student Paper

<1 %

37

Khamta Pongsanun, Tadsorn Apirak, Leklerdsiriwong Aekaluck, Sanphet Chunithipaisan, Chaidate Inchaisri. "Classification of Individual Dairy Cow Behaviors Using Accelerometer, Gyroscope, and Integrated Sensor Models", Springer Science and Business Media LLC, 2025

Publication

<1 %

38 Sebastian Dörn. "Programmieren für Ingenieure und Naturwissenschaftler", Springer Science and Business Media LLC, 2016
Publication

39 Granados, Jose Maria. "General Purpose GPU Benchmarks for Neural Networks", The University of Texas at El Paso, 2025
Publication

40 George S. Tselikis. "Introduction to C++", CRC Press, 2022
Publication

41 Frédéric Cazals, Edoardo Sarti. "AlphaFold predictions on whole genomes at a glance", Cold Spring Harbor Laboratory, 2024
Publication

42 Frank Vega. "NP on Logarithmic Space", Cambridge University Press (CUP), 2023
Publication

43 Lucas Matheus Silva Pereira. "Otimização estrutural mediante redes adversárias generativas.", Universidade de São Paulo. Agência de Bibliotecas e Coleções Digitais, 2024
Publication

44 Xu, Dafeng. "A Critical Evaluation of Risk on Large Scale Logistics Projects", University of Pretoria (South Africa), 2023
Publication

45 Juanru Guo, Simona Sarafinovska, Ryan A. Hagenson, Mark C. Valentine, Joseph D. Dougherty, Robi D. Mitra, Brian D. Muegge. "SMURF Reconstructs Single-Cells from Visium HD Data to Reveal Zonation of Transcriptional

Programs in the Intestine", Cold Spring Harbor Laboratory, 2025

Publication

46

Kamel Lahouel, Mete Mulazimuglu, Jorge Soria-Bustos, Kameron Bates et al. "Learned Geometry, Predicted Binding: Structurally-Based Prediction of Peptide:MHC Binding Using AlphaFold 3 Enables CD4 T Cell Epitope Prediction", Cold Spring Harbor Laboratory, 2025

Publication

<1 %

47

Lubos Brieda. "Plasma Simulations by Example", CRC Press, 2019

Publication

<1 %

Exclude quotes Off

Exclude matches Off

Exclude bibliography Off